

UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN
Y TELECOMUNICACIONES



SOFTWARE PARA LA GESTIÓN Y OPTIMIZACIÓN INTELIGENTE
DE PROCESOS DE NEGOCIO MEDIANTE INTELIGENCIA
ARTIFICIAL Y PROCESAMIENTO DE LENGUAJE NATURAL

Universitario: Mamani Meza Beimar

Registro: 220001741

Materia: Ingeniería de Software I

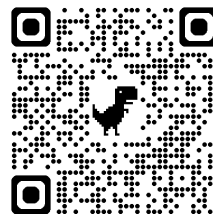
Grupo: SB

Docente: Ing. Rolando Antoni Martinez Canedo

Gestión: Semestre 1 – 2026



Software en linea



Repositorio de github

Contenido	
1. Perfil	6
1.1. Introducción.....	6
1.2. Objetivo General	7
1.3. Objetivos Específicos.....	7
1.4. Descripción del problema.....	8
1.5. Alcance	10
1. Módulo de Modelado y Constructor de Procesos	10
2. Módulo de Gestión Documental y Repositorio Colaborativo	11
3. Módulo de Ejecución, Movilidad y Disponibilidad Offline	11
4. Módulo de Inteligencia Artificial, Agente Predictivo y Análisis de Riesgo	11
5. Módulo de Backend, Persistencia e Infraestructura de Nube	12
PARTE 1: FUNDAMENTACION TEORICA	13
1.6. Marco teórico	13
CAPÍTULO 1: INGENIERÍA DE SOFTWARE ASISTIDA POR COMPUTADORA (CASE)	13
CAPÍTULO 2: DESARROLLO DE SOFTWARE BASADO EN COMPONENTES (CBSE)	13
CAPÍTULO 3: ARQUITECTURA DE SOFTWARE Y PATRONES DE DISEÑO.....	14
CAPÍTULO 4: EL PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE (PUDS).....	15
CAPÍTULO 5: METODOLOGÍA DE MODELADO: UML Y C4 MODEL	15
CAPÍTULO 6: GESTIÓN DOCUMENTAL DIGITAL Y FILOSOFÍA PAPERLESS	16
CAPÍTULO 7: INTELIGENCIA ARTIFICIAL, DEEP LEARNING Y PLN MULTIMODAL	16
CAPÍTULO 8: CALIDAD DE SOFTWARE, DEVOPS Y ENTORNO CLOUD	17
CAPÍTULO 9: ANÁLISIS COMPARATIVO DE HERRAMIENTAS.....	17
2. FLUJO DE TRABAJO: CAPTURA DE REQUISITOS	18
2.1. IDENTIFICAR ACTORES Y CASOS DE USO	18
2.1.1. Actores	18
2.1.2. Lista De Casos De uso	19
2.1.3. Priorización de casos de uso	20

Ciclo # 1	20
Ciclo # 2	21
2.1.4. Detallar casos de uso	21
Ciclo # 1	21
Ciclo # 2	36
3.FLUJO DE TRABAJO: ANALISIS	47
3.1. ANALISIS DE ARQUITECTURA.....	47
3.1.1. IDENTIFICAR PAQUETES.....	47
Paquete # 1. Gestión y configuración inteligente.....	47
Paquete #2. Operación, Movilidad y Seguimiento de tramites	48
Paquete #3: Gestión Documental Concurrente	49
Paquete #4: Servicios Cognitivos y Analítica Predictiva.....	49
3.1.2. RELACIONAR PAQUETES Y CASOS DE USO	50
Paquete # 1. Gestión y configuración inteligente.....	50
Paquete #2. Operación, Movilidad y Seguimiento de tramites	51
Paquete #3: Gestión Documental Concurrente	51
Paquete #4: Servicios Cognitivos y Analítica Predictiva.....	52
3.1.3. VISTA DE CASOS DE USO	52
Paquete # 1. Gestión y configuración inteligente.....	52
Paquete #2. Operación, Movilidad y Seguimiento de tramites	53
Paquete #3: Gestión Documental Concurrente	53
Paquete #4: Servicios Cognitivos y Analítica Predictiva.....	54
3.2. ANALISIS DE CASOS DE USO.....	54
DIAGRAMA DE COMUNICACIÓN.....	54
Ciclo # 1.....	54
Ciclo # 2.....	57
3.3. ANALISIS DE CLASES.....	58
Ciclo # 1.....	58
Ciclo # 2.....	62
3.4. ANALISIS DE PAQUETES.....	63
4. FLUJO DE TRABAJO: DISEÑO.....	64
4.1. DISEÑO DE ARQUITECTURA	64
4.1.1. DISEÑO FISICO	64

DIAGRAMA DE DESPLIEGUE.....	64
4.1.2. DISEÑO LOGICO	65
DIAGRAMA ORGANIZADO EN CAPAS.....	65
4.1.3. MODELO C4.....	65
5. FLUJO DE TRABAJO: DISEÑO DE DATOS.....	70
5.1. DIAGRAMA DE CLASES	70
5.2. DISEÑO FISICO	70
TABLA DE VOLUMEN.....	70
SCRIP.....	75
5.3. DISEÑAR CASOS DE USO	81
DIAGRAMA DE SECUENCIA	81
Ciclo # 1.....	81
Ciclo # 2.....	86
6.FLUJO DE TRABAJO: IMPLEMENTACION	90
6.1. Elección de la plataforma de desarrollo de software.....	90
6.1.1. Lenguajes de programación	90
6.1.2. Base de datos.....	91
6.1.3. Sistemas operativos.....	91
6.1.4. Otros	92
FRAMEWORK Y LIBRERIAS	92
6.1.5. Entornos de desarrollo (IDEs).....	93
6.1.6. Servidores y alojamiento (AWS)	94
6.2. Implementación de la arquitectura de software.....	94
Diagrama de componentes Principal del sistema	94
6.3. Implantación e la arquitectura de subsistema	95
Diagrama de componentes de cada paquete	95
Subsistema: Paquete 1.....	95
Subsistema: Paquete 2.....	95
Subsistema: Paquete 3.....	96
Subsistema: Paquete 4.....	96
7. CONCLUSION	97
8. RECOMENDACIÓN	98
9. BIBLIOGRAFIA.....	99

10.	CODIGO	100
10.1.	Backend	100
10.2.	fontend	¡Error! Marcador no definido.
10.3.	microservicio_ia	¡Error! Marcador no definido.
10.4.	proyecto desplegado.....	101

1. Perfil

1.1. Introducción

En la actualidad, la agilidad organizacional depende directamente de la capacidad de las instituciones para gestionar sus procesos de negocio (BPM). Sin embargo, los sistemas tradicionales de gestión de flujos de trabajo (*workflows*) enfrentan una barrera crítica: la rigidez estructural, la dependencia de procesos manuales basados en papel y el modelado técnico propenso a errores, lo que genera una brecha significativa entre las necesidades operativas y la implementación tecnológica final.

El presente proyecto de grado propone el desarrollo de un **"Software para la Gestión y Optimización Inteligente de Procesos de Negocio mediante Inteligencia Artificial y Procesamiento de Lenguaje Natural"**. Esta solución no solo busca automatizar y digitalizar trámites bajo una filosofía estrictamente *Paperless* (cero papel), sino transformar radicalmente la manera en que se conciben, diseñan y ejecutan las políticas de negocio, integrando capacidades cognitivas avanzadas que permiten evolucionar de diagramas estáticos a ecosistemas de trabajo completamente dinámicos, autónomos y asistidos.

Para responder a la alta complejidad que exige una solución de esta naturaleza, la propuesta técnica y arquitectónica se fundamenta en los siguientes pilares de innovación e infraestructura:

- **Modelado Natural y Automatización:** Capacidad de traducir lenguaje humano directamente en diagramas de actividades ejecutables y formularios dinámicos asociados a cada punto de atención.
- **Enrutamiento Inteligente y Análisis de Riesgo:** Implementación de modelos de *Deep Learning* y Procesamiento de Lenguaje Natural (PLN) que actúan como un agente inteligente, prediciendo la mejor ruta para cada trámite, anticipando cuellos de botella y detectando anomalías en tiempo real.
- **Gestión Documental Concurrente y Escalable:** Un repositorio digital avanzado que soporta múltiples formatos de archivos con control estricto de privilegios y capacidades de edición colaborativa simultánea.
- **Arquitectura Multilenguaje e Infraestructura Cloud:** Despliegue híbrido y altamente eficiente que combina la robustez de un backend en **Spring Boot**, la agilidad de microservicios cognitivos en **Python con FastAPI** para los módulos de IA, interfaces móviles en **Flutter con soporte offline**, y aplicaciones web progresivas (**PWA**); todo esto orquestado y alojado en la nube de **Amazon Web Services (AWS)** aprovechando instancias **EC2** y almacenamiento optimizado en **Amazon S3**.

Bajo la rigurosidad metodológica del **Proceso Unificado (PUDS)**, el estándar de modelado **UML 2.5+** y el diseño arquitectónico estructurado mediante **C4 Model**,

este trabajo busca demostrar que la convergencia entre la ingeniería de software clásica, las buenas prácticas de código limpio (SOLID/Clean Code) y la inteligencia artificial multimodal puede elevar exponencialmente los estándares de eficiencia, transparencia y usabilidad en la gestión de servicios corporativos y universitarios.

1.2. Objetivo General

Desarrollar un software para la gestión y optimización inteligente de procesos de negocio mediante inteligencia artificial y procesamiento de lenguaje natural

1.3. Objetivos Específicos

- **Analizar y modelar** los requerimientos funcionales y no funcionales de todo el sistema bajo el marco conceptual del Proceso Unificado (PUDS), garantizando una estricta trazabilidad entre el modelado de negocio (calles/flujo) y los diagramas e ingeniería de software generados mediante herramientas CASE.
- **Diseñar** la arquitectura macro del software utilizando el enfoque de C4 Model (niveles de contexto, contenedor y componentes), definiendo la interacción síncrona y asíncrona entre el núcleo transaccional en Spring Boot, los microservicios cognitivos de Inteligencia Artificial en FastAPI (Python) y la persistencia de datos NoSQL/Relacional.
- **Implementar** el Agente Inteligente de atención y el motor de enrutamiento predictivo mediante técnicas de *Deep Learning* y Procesamiento de Lenguaje Natural (PLN), capacitando al sistema para procesar comandos de voz y texto con el fin de realizar la asignación automatizada de políticas de negocio, detección de anomalías y análisis de riesgos en tiempo real.
- **Desarrollar** el repositorio de Gestión Documental digital (*Paperless*) acoplado a los módulos core de *Workflow* y *Form Builder*, implementando un control estricto de privilegios por nodo y asegurando la edición colaborativa concurrente y síncrona de archivos en tiempo real.

- **Construir** las interfaces de usuario del ecosistema (Web y Móvil) utilizando Angular y Flutter, integrando capacidades de Aplicaciones Web Progresivas (PWA) y persistencia local para asegurar el funcionamiento 100% offline y la sincronización automática de datos mediante WebSockets.
- **Desarrollar** el motor de Reportes Dinámicos asistido por inteligencia artificial conversacional, permitiendo al sistema interpretar las solicitudes complejas del administrador por texto o voz para estructurar reportes automatizados basados en datos, criterios de filtrado y formatos de salida bajo demanda (Excel, Word, PDF).
- **Integrar** los servicios de asistencia por voz (síntesis y reconocimiento) en las interfaces de usuario, diseñando una estrategia de usabilidad y mecanismos de aprendizaje intuitivos que faciliten al usuario final la adopción de las características cognitivas del software.
- **Garantizar** la calidad, robustez y mantenibilidad del código fuente aplicando principios de diseño SOLID, estándares de Clean Code y la ejecución de entornos de pruebas automatizadas (Unit Tests) con métricas de cobertura integradas directamente en el IDE de desarrollo.
- **Consolidar** el despliegue e integración continua (CI/CD) de toda la solución integrada en la infraestructura de nube de Amazon Web Services (AWS), mediante el aprovisionamiento de instancias Amazon EC2 para los backends, almacenamiento optimizado de archivos en Amazon S3, y gestionando el versionamiento del proyecto al 100% a través de GitHub.

1.4. Descripción del problema

En el entorno corporativo y administrativo actual, la eficiencia operativa depende directamente de la capacidad de las organizaciones para ejecutar sus procesos de negocio de manera ágil, transparente y controlada. Sin embargo, los sistemas tradicionales de gestión de flujos de trabajo (*workflows*) e ingeniería de software arrastran una serie de limitaciones críticas que impactan negativamente la productividad, la toma de decisiones y el ciclo de vida del software, las cuales se detallan a continuación:

- **Rigidez en el Modelado de Políticas de Negocio:** El diseño de flujos operativos sigue siendo una tarea manual, técnica y restrictiva. El Diseñador se enfrenta a interfaces de modelado estáticas que resultan lentas y propensas a errores lógicos. No existen mecanismos cognitivos naturales que traduzcan las peticiones verbales o escritas del negocio directamente en flujos de trabajo dinámicos, lo que frena drásticamente la automatización de nuevos trámites.
- **Dependencia de Procesos Físicos e Ineficiencia Documental (*Falta de Paperless*):** Las organizaciones sufren una persistente dependencia de documentos físicos o archivos dispersos sin centralización eficiente. Los repositorios tradicionales carecen de capacidades colaborativas síncronas, impidiendo que múltiples funcionarios editen en tiempo real un mismo archivo durante un trámite paralelo. Asimismo, almacenar grandes volúmenes de documentación híbrida (Word, PDFs, imágenes, videos) satura los servidores locales y degrada severamente los tiempos de respuesta del sistema, incrementando los costos operativos.
- **Ausencia de Enrutamiento Inteligente y Análisis Predictivo de Riesgos:** La mayoría de los sistemas BPM se limitan a registrar bitácoras estáticas de datos históricos, pero carecen de una capa analítica proactiva basada en *Deep Learning*. Identificar por qué un trámite excede los límites de tiempo establecidos (SLA) o predecir cuellos de botella antes de que ocurran es una tarea que hoy se realiza mediante auditorías manuales exhaustivas. La falta de un motor inteligente que determine la ruta óptima de un flujo y detecte anomalías o riesgos en tiempo real impide la optimización de recursos y daña la experiencia del Usuario Final.
- **Sobrecarga Cognitiva en la Atención al Usuario y Reportería Operativa:** El Funcionario encargado de ejecutar los trámites lidia con interfaces saturadas de formularios extensos y manuales de usuario estáticos e ineficientes. La carencia de un asistente inteligente con capacidades de síntesis y reconocimiento de voz que comprenda el contexto y guíe al operador eleva el margen de error. Por otro lado, cuando la gerencia o el Administrador necesita evaluar el rendimiento global, se enfrenta a la rigidez de reportes predefinidos; la imposibilidad de generar reportes dinámicos al vuelo mediante lenguaje natural conversacional frena la toma de decisiones oportunas.

- **Pérdida de Disponibilidad Operativa en Entornos Inestables (*Falta de Soporte Offline*):** Las aplicaciones corporativas tradicionales quedan completamente inutilizadas ante cortes de energía o fallas de conectividad a Internet. La ausencia de arquitecturas pensadas para el funcionamiento en modo offline en dispositivos móviles y de aplicaciones web progresivas (PWA) paraliza la atención de trámites en campo, provocando la pérdida de datos y retrasos masivos en la sincronización del estado de los flujos.
- **Desafío Arquitectónico en la Mantenibilidad, Calidad del Código y Despliegue:** Uno de los problemas más graves en proyectos de gran envergadura es la degradación arquitectónica y la obsolescencia de la documentación técnica (diagramas que no reflejan el código real). El desarrollo sin un marco estricto de desacoplamiento (como microservicios o patrones de diseño limpios) y la ausencia de entornos de pruebas unitarias automatizadas en el IDE incrementan los "bugs" en producción. A esto se suma la complejidad de desplegar de forma monolítica, perdiendo la escalabilidad elástica que proveen los entornos de nube nativos.

1.5. Alcance

El sistema para la Gestión y Optimización Inteligente de Procesos de Negocio se desarrollará como un ecosistema tecnológico multiplataforma integrado (Web y Móvil) orientado a la automatización de flujos de trabajo (*workflows*) dinámicos bajo una filosofía corporativa *Paperless*. El software integrará microservicios distribuidos de Inteligencia Artificial (IA) y Procesamiento de Lenguaje Natural (PLN) para permitir que la asignación, ejecución, auditoría y análisis de procesos sean asistidos de manera proactiva, eliminando la rigidez operativa y arquitectónica de los sistemas BPM tradicionales.

El proyecto abarcará los siguientes módulos funcionales e ingeniería de software:

1. Módulo de Modelado y Constructor de Procesos

- **Diagramador de Clases y Actividades Colaborativo:**
 - Interfaz visual para el diseño de flujos operativos, incluyendo nodos de atención, aristas, atributos y reglas de negocio dinámicas.
 - Soporte completo para notación UML 2.5+ con especificación de multiplicidades, estereotipos y calles (*swimlanes*) de negocio.
- **Gestión de Formularios Dinámicos (*Form Builder*):**
 - Constructor visual para el diseño e instanciación de interfaces de captura de datos asociadas de forma unívoca a cada nodo de atención dentro del flujo.

- Configuración de esquemas y campos variables que adaptan su comportamiento en tiempo real según el tipo de trámite e interacciones previas.

2. Módulo de Gestión Documental y Repositorio Colaborativo

- **Base Documental Digital Corporativa (*Paperless*):**
 - Repositorio centralizado capaz de admitir múltiples formatos digitales híbridos (archivos Word, planillas Excel, documentos PDF, imágenes y videos) generados a lo largo del ciclo de vida del trámite.
 - Control estricto de seguridad, perfiles y asignación de privilegios de acceso (lectura, subida, modificación) parametrizables por nodo y por funcionario.
- **Simultaneidad y Edición Concurrente:**
 - Soporte para la edición colaborativa simultánea de múltiples funcionarios sobre un mismo archivo en tiempo real, implementado mediante canales de comunicación bidireccional asíncrona (*WebSockets*) e integración de librerías de concurrencia.

3. Módulo de Ejecución, Movilidad y Disponibilidad Offline

- **Bandeja de Trabajo (*Worklist*) Priorizada:**
 - Interfaz unificada para el Funcionario que organiza de forma inteligente las tareas pendientes según su criticidad, nivel de prioridad y acuerdos de nivel de servicio (SLA).
 - Control dinámico del ciclo de vida de los trámites mediante estados obligatorios y auditorías completas: *En Proceso*, *Terminado*, *Retenido* y *Cancelado*.
- **Disponibilidad y Resiliencia en Redes (*Modo Offline*):**
 - Arquitectura con persistencia local de datos en la aplicación móvil de **Flutter** y capacidades de Aplicación Web Progresiva (**PWA**), permitiendo la continuidad de la atención de trámites sin conectividad a Internet y ejecutando la sincronización diferida automática al restablecerse el canal de red.
- **Seguimiento y Portabilidad para el Usuario Final:**
 - Portal de autoservicio que permite al cliente consultar el estado de su solicitud, visualizar el historial detallado de movimientos e interactuar con el flujo mediante notificaciones *push*.

4. Módulo de Inteligencia Artificial, Agente Predictivo y Análisis de Riesgo

- **Agente Inteligente de Enrutamiento Cognitivo:**

- Procesamiento de comandos de voz y texto a través de modelos de Procesamiento de Lenguaje Natural (PLN) que interpretan las intenciones del usuario, automatizando la interacción con la plataforma sin requerir un operador humano inicial.
 - Motor predictivo basado en algoritmos de *Deep Learning* para determinar la ruta óptima de procesamiento para cada trámite, anticipar retrasos y realizar análisis de riesgo operacional en tiempo real.
- **Asistencia al Operador por Síntesis de Voz:**
 - Integración de servicios cognitivos de voz para guiar de manera interactiva y audible al Funcionario en momentos de alta carga laboral, optimizando los tiempos de llenado de formularios y mitigando los errores en la captura de datos.
- **Generador Conversacional de Reportes Dinámicos:**
 - Capa analítica avanzada que procesa solicitudes complejas del Administrador expresadas en lenguaje natural. El motor es capaz de decodificar la instrucción para generar reportes dinámicos al vuelo, abstrayendo de forma automática los datos solicitados, los criterios de filtrado/agrupación y estructurando el archivo de salida en formatos Excel, Word o PDF.

5. Módulo de Backend, Persistencia e Infraestructura de Nube

- **Persistencia No-Relacional Documental:**
 - Almacenamiento elástico de los proyectos, versionamiento de diagramas de actividades, metadatos y esquemas de trámites bajo estructuras de colecciones documentales dinámicas.
- **Aprovisionamiento de Infraestructura Cloud en AWS:**
 - Despliegue y publicación productiva de la solución completa en la nube de **Amazon Web Services (AWS)** utilizando instancias virtuales **Amazon EC2** para el alojamiento y ejecución de los backends y microservicios distributivos.
 - Almacenamiento masivo externo de los documentos de la gestión documental del sistema mediante **Amazon S3**, asegurando alta velocidad de transferencia, redundancia de datos y una reducción sustancial de costos en comparación con sistemas de almacenamiento local.

- **Gestión del Ciclo de Vida y Versionamiento:**
 - Administración de perfiles, roles y niveles de acceso diferenciados para todos los actores (*Diseñador, Funcionario, Administrador y Usuario Final*).
 - Gestión de la configuración y control total de versiones del código de ingeniería del sistema centralizado al 100% mediante repositorios colaborativos en **GitHub**.

PARTE 1: FUNDAMENTACION TEORICA

1.6. Marco teórico

CAPÍTULO 1: INGENIERÍA DE SOFTWARE ASISTIDA POR COMPUTADORA (CASE)

La ingeniería CASE (*Computer-Aided Software Engineering*) es la aplicación de métodos y herramientas de software orientados a automatizar de manera productiva las actividades del ciclo de vida de desarrollo de software (SDLC).

- **Upper CASE (U-CASE):** Facilita la captura de requisitos y el diseño conceptual mediante el modelado visual. En este sistema, el Diseñador modela flujos y formularios dinámicos estructurados en calles (*swimlanes*) a través de la interfaz visual de Angular.
- **Lower CASE (L-CASE):** Se enfoca en la automatización de la generación de código, pruebas y despliegue. El entorno de desarrollo del proyecto actúa como una herramienta L-CASE al integrar ingeniería inversa y directa mediante herramientas CASE de modelado para sincronizar el diseño con el código real.
- **Integrated CASE (I-CASE):** Representa la integración total de herramientas para cubrir todo el SDLC, persiguiendo la **Sincronía Total**. En la solución, se logra asegurando que las reglas de negocio modeladas visualmente correspondan unívocamente con el motor de estados transaccionales en Spring Boot y los esquemas dinámicos del repositorio.

CAPÍTULO 2: DESARROLLO DE SOFTWARE BASADO EN COMPONENTES (CBSE)

La CBSE (*Component-Based Software Engineering*) propone la construcción de sistemas complejos mediante la integración de piezas de software independientes, permitiendo reducir costos, mitigar errores y acelerar el tiempo de comercialización.

- **Naturaleza del Componente:** Son unidades de composición independientes con interfaces contractualmente especificadas. El núcleo transaccional en Spring Boot y los servicios distributivos de Inteligencia Artificial en FastAPI operan como componentes de software desacoplados, autónomos y con responsabilidades unívocas.
- **Interfaces y Contratos:** La comunicación interna y externa entre componentes se establece mediante protocolos REST para peticiones asíncronas tradicionales y canales bidireccionales de **WebSockets** para soportar la interactividad colaborativa en tiempo real de la gestión documental.
- **Composición de Servicios Externos:** El ecosistema integra componentes externos especializados como servicios cognitivos de voz para potenciar la accesibilidad y usabilidad del sistema, actuando como proveedores de servicios externos (*Software as a Service* - SaaS) dentro de la arquitectura modular.

CAPÍTULO 3: ARQUITECTURA DE SOFTWARE Y PATRONES DE DISEÑO

La arquitectura de software es la organización fundamental de un sistema, definida por sus componentes, las relaciones entre ellos, su entorno y los principios que guían su diseño y evolución.

- **Arquitectura de Microservicios Distribuidos:** Se divide la lógica de negocio en servicios autónomos para mejorar la mantenibilidad y escalabilidad. El backend en Spring Boot orquesta los flujos de trabajo (*workflows*) corporativos, mientras que los microservicios en FastAPI gestionan las tareas intensivas de Inteligencia Artificial, NLP y analítica predictiva.
- **Patrón de Persistencia No-Relacional Documental:** Se emplea un enfoque NoSQL documental para manejar esquemas dinámicos variables, almacenando la configuración de diagramas, formularios dinámicos e históricos de trámites sin las restricciones de integridad rígida de los modelos relacionales.

- **Patrón Observer con WebSockets:** Implementado para la sincronización de estado y la propagación de eventos en tiempo real. Este patrón es el núcleo que asegura que múltiples usuarios visualicen las actualizaciones de un archivo simultáneamente durante la **edición colaborativa concurrente** en la base documental.

CAPÍTULO 4: EL PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE (PUDS)

El PUDS es el marco metodológico de ingeniería de software (basado en la literatura de Jacobson, Booch y Rumbaugh) caracterizado por ser iterativo, incremental, centrado en la arquitectura y dirigido por casos de uso.

- **Fase de Inicio (*Inception*):** Se delimita el alcance global del sistema, la visión del producto y la mitigación de los primeros riesgos tecnológicos asociados a la IA y la concurrencia.
- **Fase de Elaboración:** Análisis profundo del dominio del problema y establecimiento de la **Línea Base de la Arquitectura**, mitigando riesgos mediante el diseño de prototipos ejecutables integrados.
- **Fase de Construcción:** Desarrollo iterativo y secuencial de los componentes del software mediante sprints, transformando los modelos de análisis y diseño en código fuente funcional.
- **Fase de Transición:** Pruebas finales de calidad global, validación de la usabilidad y despliegue productivo de la solución integrada en la nube de **Amazon Web Services (AWS)**.

CAPÍTULO 5: METODOLOGÍA DE MODELADO: UML Y C4 MODEL

La documentación y visualización técnica del ecosistema de software requiere la combinación de dos estándares industriales complementarios de modelado:

- **C4 Model para Arquitectura de Software:** Exigido de manera estricta para documentar la estructura de la solución a través de niveles de abstracción jerárquica. Se implementa para modelar el **Contexto** del sistema, los **Contenedores** tecnológicos (Angular, Flutter, Spring Boot, FastAPI) y los **Componentes** internos de cada servicio.

- **Lenguaje Unificado de Modelado (UML 2.5+):** Utilizado para el modelado detallado de la lógica y la ingeniería del software. Incluye **Diagramas de Casos de Uso** para los requisitos, **Diagramas de Clases y Paquetes** para el diseño lógico, **Diagramas de Secuencia** para la comunicación asíncrona, y **Diagramas de Estado** para controlar el ciclo de vida del trámite (*En Proceso, Terminado, Retenido, Cancelado*).

CAPÍTULO 6: GESTIÓN DOCUMENTAL DIGITAL Y FILOSOFÍA PAPERLESS

Sustenta la transformación de flujos administrativos físicos en entornos totalmente digitalizados y eficientes.

- **Flujos Documentales Híbridos:** Concepto de indexación y almacenamiento de metadatos para archivos multimediales de diferente naturaleza (Word, Excel, PDF, imágenes), asociados directamente al estado de un flujo de atención.
- **Control de Acceso Basado en Roles (RBAC) por Nodo:** Mecanismo de seguridad que restringe o habilita de forma dinámica las operaciones documentales de un Funcionario (subir, leer, modificar) dependiendo estrictamente del punto de atención en el que se encuentre el trámite.

CAPÍTULO 7: INTELIGENCIA ARTIFICIAL, DEEP LEARNING Y PLN MULTIMODAL

Justifica científicamente el uso de modelos avanzados de IA embebidos en el software como optimizadores analíticos del negocio y facilitadores de la usabilidad.

- **Modelos de Lenguaje Basados en Transformers (NLP):** Redes neuronales implementadas en FastAPI para procesar texto libre y audio transcrito, abstrayendo las intenciones del usuario para la ejecución de comandos y la **generación automatizada de reportes dinámicos** bajo demanda.
- **Redes Neuronales Profundas (Deep Learning) para Análisis de Riesgo:** Algoritmos predictivos encargados de evaluar en tiempo real las métricas históricas de ejecución del *workflow*, determinando la ruta óptima de enrutamiento, estimando retrasos (SLA) y detectando anomalías o riesgos operacionales.

- **Síntesis y Reconocimiento de Voz:** Herramientas de audio conversacional integradas para guiar proactivamente al Funcionario y al Usuario Final en el llenado de datos, mitigando la sobrecarga cognitiva y reduciendo errores.

CAPÍTULO 8: CALIDAD DE SOFTWARE, DEVOPS Y ENTORNO CLOUD

Establece los estándares de ingeniería y las estrategias de despliegue que garantizan la alta disponibilidad y la robustez del código fuente del producto final.

- **Principios SOLID y Clean Code:** Estándares de programación aplicados estrictamente para asegurar que el código sea desacoplado, legible, extensible y mantenible, facilitando la escalabilidad de los microservicios.
- **Pruebas Automatizadas (*Unit Tests*):** Configuración de entornos de pruebas unitarias directamente en el IDE de desarrollo, asegurando la verificación de la lógica del negocio antes de pasar a fases de distribución.
- **Infraestructura Cloud en Amazon Web Services (AWS):** Estrategia de despliegue productivo basada en dos servicios mandatorios de la nube de Amazon:
 - **Amazon EC2:** Suministro de instancias virtuales elásticas para la ejecución y alojamiento estable de los backends transaccionales y cognitivos.
 - **Amazon S3:** Almacenamiento masivo externo de los archivos de la gestión documental, garantizando alta velocidad de transferencia, persistencia redundante y minimización drástica de costos de infraestructura.
- **Gestión de Configuración con GitHub:** Uso obligatorio de repositorios de control de versiones distribuidos para centralizar y auditar el desarrollo colaborativo del código fuente del equipo.

CAPÍTULO 9: ANÁLISIS COMPARATIVO DE HERRAMIENTAS

Justificación crítica de las decisiones tecnológicas adoptadas frente a alternativas de la industria para asegurar la viabilidad del software:

- **Persistencia Documental NoSQL vs. Modelo Relacional:** Se prioriza el motor documental NoSQL frente a soluciones relacionales clásicas (como PostgreSQL) debido a la necesidad mandatoria de manejar colecciones de metadatos elásticas, formularios variables y configuraciones de flujos altamente dinámicas.
- **Amazon S3 vs. Servidor de Archivos Local:** Se selecciona la infraestructura de AWS S3 frente al almacenamiento local debido a que un sistema *Paperless* masivo requiere alta disponibilidad, escalabilidad elástica y un costo por gigabyte infinitamente menor, evitando la saturación del servidor de aplicaciones.
- **FastAPI vs. Flask/Django:** Se adopta FastAPI en Python para los componentes de IA debido a su soporte nativo para programación asíncrona de alto rendimiento y su velocidad en el manejo de peticiones de alta concurrencia con grandes modelos de lenguaje.

2. FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

2.1. IDENTIFICAR ACTORES Y CASOS DE USO

2.1.1. Actores

Actor	Identidad	Funcionalidad Principal
Diseñador de Procesos	A1	• Utiliza la interfaz NLP para generar diagramas de actividades y clases mediante lenguaje natural. • Define la lógica de los trámites, formularios dinámicos y reglas de negocio en el motor de workflows . • Configura los niveles de prioridad y los acuerdos de nivel de servicio (SLA) para el control de tiempos.

Funcionario	A2	• Administra su Bandeja de Trabajo (Worklist) para gestionar los trámites asignados según prioridad. • Ejecuta las tareas operativas completando los formularios dinámicos procesados por el backend . • Interactúa con el Asistente de Voz (vía ElevenLabs) para recibir guía proactiva en procesos complejos . • Actualiza los estados del trámite (En proceso, Terminado, Retenido) y coordina el avance del flujo.
Usuario Final	A3	• Inicia solicitudes de trámites o servicios a través de la aplicación móvil (Flutter) o web . • Adjunta requisitos y completa la información inicial solicitada por el esquema del proceso • Realiza el seguimiento en tiempo real del estado de su solicitud y visualiza el historial de movimientos . • * Recibe notificaciones automáticas sobre la finalización o requerimientos adicionales de su trámite.

2.1.2. Lista De Casos De uso

Nro	Caso de Uso
CU1	Generar diagrama de proceso mediante NLP (Voz/Texto)
CU2	Editar política de negocio colaborativamente en tiempo real
CU3	Diseñar formulario dinámico para actividades específicas
CU4	Generar proyecto base
CU5	Visualizar dashboard de analítica y cuellos de botella
CU6	Aplicar recomendación de optimización sugerida por la IA

CU7	Iniciar un nuevo trámite desde plataforma web o móvil
CU8	Gestionar bandeja de trabajo y actualizar estados (SLA)
CU9	Consultar al asistente virtual proactivo
CU10	Consultar trazabilidad de trámite y notificaciones
CU11	Gestionar seguridad, perfiles y acceso multitenant
CU12	Subir requisitos digitales al trámite
CU13	Editar documentos colaborativamente en tiempo real
CU14	Respaldar archivos masivos en infraestructura externa
CU15	Generar reportes dinámicos mediante lenguaje natural
CU16	Evaluar enrutamiento predictivo y análisis de riesgos
CU17	Procesar tareas pendientes en modo fuera de línea
CU18	Sincronizar transacciones diferidas acumuladas en el dispositivo

2.1.3. Priorización de casos de uso

Ciclo # 1

Nro	Caso de Uso	Estado	Prioridad	Riesgo	Actor
CU-01	Generar diagrama de proceso mediante NLP (Voz/Texto)	Definido	Alta	Alto	A1
CU-02	Editar política de negocio colaborativamente en tiempo real	Definido	Alta	Medio	A1
CU-03	Diseñar formulario dinámico para actividades específicas	Definido	Alta	Bajo	A1
CU-04	Generar proyecto base	Definido	Alta	Alto	A1
CU-05	Visualizar dashboard de analítica y cuellos de botella	Definido	Media	Medio	A1
CU-06	Aplicar recomendación de optimización sugerida por la IA	Definido	Media	Alto	A1
CU-07	Iniciar un nuevo trámite desde plataforma web o móvil	Definido	Alta	Bajo	A3
CU-08	Gestionar bandeja de trabajo y actualizar estados (SLA)	Definido	Alta	Bajo	A2
CU-09	Consultar al asistente virtual proactivo (ElevenLabs)	Definido	Media	Alto	A2
CU-10	Consultar trazabilidad de trámite y notificaciones	Definido	Alta	Bajo	A3
CU-11	Gestionar seguridad, perfiles y acceso multitenant	Definido	Alta	Medio	A1

Ciclo # 2

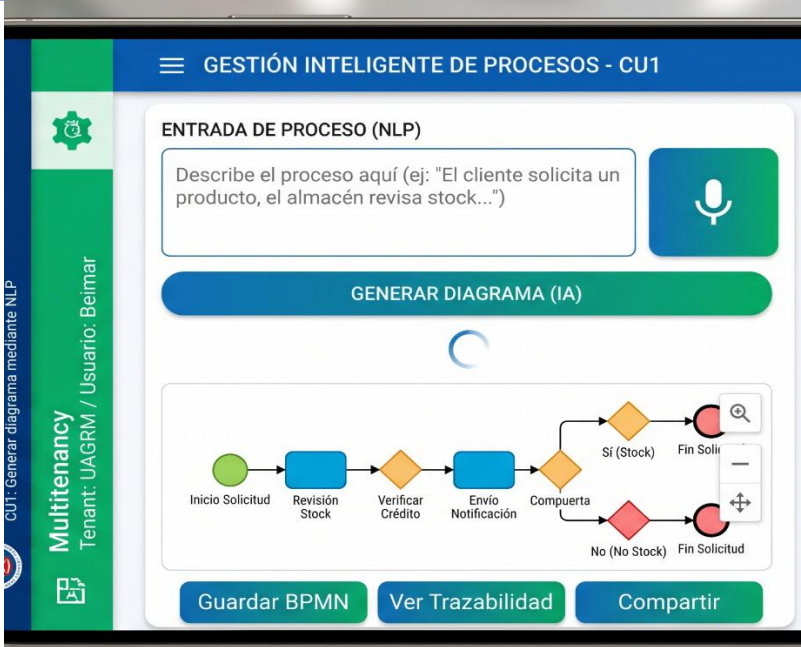
Nro	Caso de Uso	Estado	Prioridad	Riesgo	Actor
CU12	Subir requisitos digitales al trámite	Definido	Alta	Bajo	A2, A3
CU13	Editar documentos colaborativamente en tiempo real	Definido	Alta	Alto	A2
CU14	Respalidar archivos masivos en infraestructura externa	Definido	Alta	Medio	A2, A3
CU15	Generar reportes dinámicos mediante lenguaje natural (FastAPI)	Definido	Alta	Alto	A1
CU16	Evaluar enrutamiento predictivo y análisis de riesgos	Definido	Media	Alto	A2, A3
CU17	Procesar tareas pendientes en modo fuera de línea	Definido	Alta	Alto	A2
CU18	Sincronizar transacciones diferidas acumuladas en el dispositivo	Definido	Alta	Medio	A2

2.1.4. Detallar casos de uso

Ciclo # 1

CU-01 Generar diagrama de proceso mediante NLP (Voz/Texto)

Elemento	Descripción
Propósito	Automatizar la creación de la estructura lógica de un proceso de negocio a partir de descripciones en lenguaje natural, reduciendo el tiempo de modelado manual y garantizando la coherencia técnica inicial.
Actores	A1
Actor Iniciador	A1.
Precondición	El usuario debe estar autenticado, tener un proyecto activo y conexión estable con el microservicio de IA en FastAPI.
Flujo Principal	1. El diseñador accede al lienzo de modelado y activa la función de "Asistente de Diseño NLP". 2. El sistema habilita la entrada de datos mediante campo de texto o activación de micrófono para dictado de voz. 3. El diseñador ingresa la descripción semántica del proceso. 4. El sistema envía el <i>prompt</i> al microservicio de FastAPI para su procesamiento semántico. 5. La IA identifica las entidades (clases), actividades (nodos) y relaciones (aristas) pertinentes.

	6. El sistema traduce el esquema lógico a un formato BPMN/JSON compatible con el lienzo. 7. El sistema renderiza automáticamente el diagrama UML en el lienzo en tiempo real. 8. El diseñador revisa la propuesta, realiza ajustes manuales y selecciona "Guardar y Sincronizar".
Postcondición	El diagrama y sus metadatos quedan almacenados en MongoDB, habilitando los componentes de Ingeniería Forward para la generación de código en Spring Boot.
Excepciones	• E1: El prompt es ambiguo; el sistema solicita mayor detalle al usuario. • E2: Fallo de comunicación o <i>timeout</i> con el microservicio de IA. • E3: Incoherencia lógica detectada; el sistema genera una alerta de validación UML.
Prototipo	

CU-02 Editar política de negocio colaborativamente en tiempo real

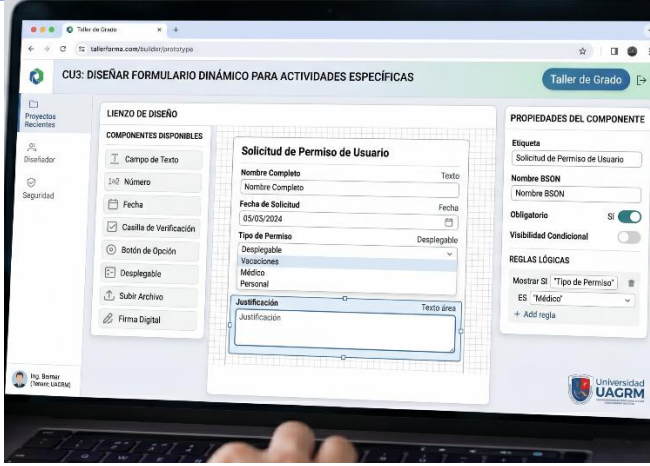
Elemento	Descripción
Propósito	Permitir que múltiples diseñadores (A1) trabajen simultáneamente sobre el mismo diagrama de procesos, asegurando que cualquier modificación sea visible instantáneamente para todos los participantes, eliminando conflictos de versiones y mejorando la eficiencia del modelado.

Actores	A1
Actor Iniciador	A1
Precondición	El usuario debe estar autenticado, el proyecto debe estar configurado para acceso compartido y el servicio de mensajería bidireccional (WebSockets) debe estar activo en el backend de Spring Boot.
Flujo Principal	1. El diseñador abre un diagrama de procesos existente desde el repositorio en MongoDB. 2. El sistema establece una conexión persistente mediante el protocolo WebSocket para la sesión actual. 3. El sistema muestra los indicadores de presencia de otros diseñadores conectados al mismo lienzo. 4. El diseñador realiza una acción sobre el diagrama (ej: mover un nodo de actividad, cambiar un estado o editar una regla de negocio). 5. El sistema captura el evento y lo transmite en formato JSON al servidor de orquestación. 6. El servidor valida la acción y redistribuye el evento de actualización a todos los clientes suscritos en tiempo real. 7. El sistema de cada participante renderiza automáticamente el cambio en la interfaz de Angular sin necesidad de recargar la página. 8. El sistema persiste el cambio en la base de datos de forma asíncrona para mantener la integridad del modelo.
Postcondición	El diagrama de procesos se mantiene sincronizado y actualizado para todos los usuarios concurrentes, reflejando el estado actual de la política de negocio en MongoDB.
Excepciones	• E1: Interrupción de la conexión de red; el sistema intenta restablecer el canal WebSocket y notifica al usuario sobre el estado <i>offline</i>. • E2: Conflictos de edición simultánea sobre el mismo objeto; el sistema aplica una estrategia de "bloqueo de nodo" o "último cambio prevalece" según la política configurada. • E3: Error de persistencia en la base de datos; el sistema revierte el cambio local y genera una alerta de error de sincronización.



CU-03 Diseñar formulario dinámico para actividades específicas

Elemento	Descripción
Propósito	Proveer una herramienta visual (Form Builder) que permita al diseñador definir los campos de entrada de datos, validaciones y reglas de visibilidad para cada actividad específica dentro de un flujo de trabajo.
Actores	A1
Actor Iniciador	A1
Precondición	El diseñador debe haber seleccionado una actividad específica dentro de un diagrama de procesos previamente modelado en el lienzo web (Angular).
Flujo Principal	1. El diseñador selecciona una actividad (nodo) en el diagrama y activa la opción "Configurar Formulario". 2. El sistema despliega el panel del Constructor Dinámico de Formularios con una paleta de componentes (texto, fecha, selección, archivos, etc.). 3. El diseñador arrastra y suelta los componentes necesarios en el área de diseño del formulario. 4. 5. El diseñador define las propiedades de cada campo: nombre técnico, etiqueta visual, obligatoriedad y validaciones (Regex). 6. El diseñador establece reglas de visibilidad condicional si el formulario lo requiere.

	- El sistema genera una vista previa instantánea del formulario para su validación visual. - El diseñador confirma el diseño y selecciona "Vincular a Actividad". - El sistema genera un esquema en formato JSON que representa la estructura del formulario y lo asocia al ID de la actividad.
Postcondición	El esquema del formulario queda persistido en MongoDB vinculado a la política de negocio, quedando disponible para ser renderizado dinámicamente cuando un Funcionario (A2) ejecute la tarea.
Excepciones	- E1: El diseñador intenta guardar un formulario sin campos definidos; el sistema solicita al menos un elemento de entrada. - E2: Error de sincronización con la base de datos; el sistema notifica que el esquema no pudo ser persistido en MongoDB. - E3: Nombres técnicos de campos duplicados; el sistema emite una alerta de validación para evitar conflictos en la persistencia de datos.
Prototipo	

CU-04 Generar proyecto base

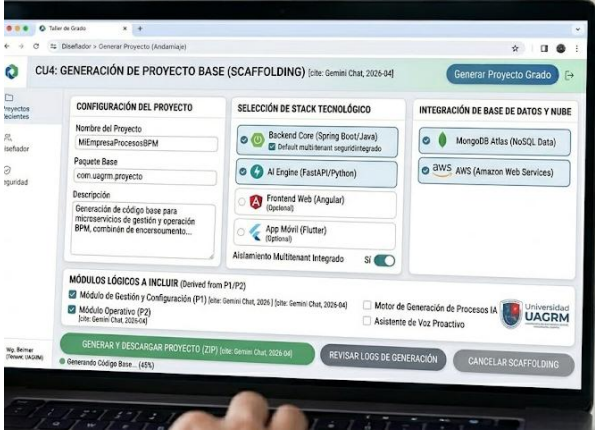
Elemento	Descripción
Propósito	Automatizar la creación de la infraestructura de backend del sistema mediante la generación de un proyecto base en Spring Boot, basado en los diagramas de clases y procesos

	diseñados, para reducir el error humano y acelerar el desarrollo.
Actores	A1
Actor Iniciador	A1.
Precondición	El diagrama de procesos y el modelo de datos deben estar validados, guardados en MongoDB y no presentar inconsistencias lógicas (ej. clases sin atributos o flujos sin fin).
Flujo Principal	1. El diseñador selecciona la opción "Generar Ingeniería Forward" desde el panel de administración del proyecto. 2. El sistema analiza el metamodelo almacenado en la base de datos documental. 3. El sistema realiza el mapeo de las clases UML hacia documentos Spring Data MongoDB. 4. El sistema genera los componentes de persistencia (Repositories) y la lógica de orquestación (Services) para cada actividad del flujo. 5. El sistema crea los controladores REST (Controllers) y los objetos de transferencia de datos (DTOs) necesarios para la integración con el frontend. 6. El sistema empaqueta toda la estructura de carpetas, archivos .java y archivos de configuración (pom.xml, application.properties) en un archivo comprimido (.ZIP). 7. El sistema habilita el botón de descarga del proyecto generado. 8. El diseñador descarga el archivo para su posterior despliegue o personalización en un entorno local.
Postcondición	Se obtiene un artefacto de software (código fuente) que refleja exactamente el diseño visual, listo para ser ejecutado en un entorno de desarrollo o virtualizado con Docker.
Excepciones	• E1: Errores en la estructura del diagrama (nodos huérfanos); el sistema interrumpe la generación y emite un reporte de errores de validación UML. • E2: Fallo en la escritura del sistema de archivos del servidor al crear el archivo .ZIP.

	E3: Incompatibilidad de tipos de datos entre el formulario dinámico y el modelo de datos; el sistema solicita corregir el mapeo.
Prototipo	

CU-05 Visualizar dashboard de analítica y cuellos de botella

Elemento	Descripción
Propósito	Proporcionar una visión integral y en tiempo real del rendimiento de los procesos de negocio, utilizando modelos de Machine Learning para identificar retrasos críticos y recomendar áreas de mejora en la eficiencia operativa.
Actores	A1
Actor Iniciador	A1.
Precondición	El sistema debe haber registrado datos históricos de ejecución de trámites en MongoDB y el servicio de analítica en FastAPI debe estar operativo.
Flujo Principal	1. El diseñador accede al módulo de "Analítica e Inteligencia de Negocio" desde la plataforma web. 2. El sistema consulta en MongoDB los tiempos de ejecución, estados de los trámites y transiciones entre actividades. 3. El sistema envía los datos estructurados al microservicio de FastAPI para el procesamiento con modelos de analítica. 4. El Agente IA analiza los tiempos de respuesta frente a los acuerdos de nivel de servicio (SLA) configurados. 5. El sistema identifica nodos donde el tiempo de permanencia excede el promedio histórico (Cuellos de Botella). 6. El sistema renderiza un panel de control interactivo con mapas de calor, gráficos de tendencia y alertas de criticidad. 7. El diseñador interactúa con los indicadores para profundizar en la causa raíz de los retrasos detectados.
Postcondición	El diseñador obtiene información estratégica procesada por la IA para la toma de decisiones sobre la optimización del modelo de negocio.
Excepciones	E1: Datos insuficientes para generar tendencias estadísticas; el sistema muestra un mensaje informativo solicitando mayor volumen de trámites.

	• E2: Error de sincronización con el servicio de Machine Learning; el sistema muestra los datos base sin el análisis predictivo de la IA. • E3: Inconsistencia en los metadatos de tiempo; el sistema genera una alerta de integridad de datos.
Prototipo	

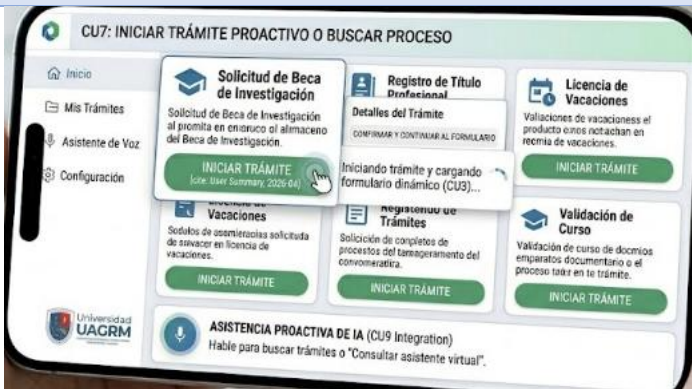
CU-06Aplicar recomendación de optimización sugerida por la IA

Elemento	Descripción
Propósito	Implementar de manera asistida las mejoras estructurales propuestas por el motor de inteligencia artificial para reducir tiempos de ciclo, eliminar redundancias y optimizar la eficiencia operativa del proceso de negocio.
Actores	A1
Actor Iniciador	A1.
Precondición	El sistema debe haber identificado previamente un cuello de botella o ineficiencia (CU-05) y generado una propuesta de optimización válida.
Flujo Principal	1. El diseñador selecciona una alerta de optimización desde el dashboard de analítica. 2. El sistema despliega una comparación visual (split-view) entre el flujo de trabajo actual y la versión optimizada propuesta por la IA.

	3. El Agente IA detalla la justificación del cambio (ej: "Se recomienda paralelizar las actividades A y B para reducir el tiempo total en un 15%"). 4. El diseñador evalúa el impacto proyectado y la factibilidad de la recomendación. 5. El diseñador selecciona la opción "Ejecutar Optimización Automática". 6. El sistema modifica dinámicamente el diagrama de procesos en el lienzo, reconfigurando los nodos y aristas afectados. 7. El sistema actualiza los metadatos de la política en MongoDB. 8. El sistema genera una nueva versión del proceso y notifica que los cambios están listos para la sincronización con el código (Ingeniería Forward).
Postcondición	La estructura del proceso de negocio es actualizada con la nueva lógica optimizada, mejorando el rendimiento de los futuros trámites iniciados en el sistema.
Excepciones	• E1: El diseñador descarta la recomendación por criterios institucionales; el sistema registra el rechazo para ajustar futuros modelos de aprendizaje. • E2: Conflicto de estructura; la optimización propuesta rompe una regla de negocio obligatoria y el sistema bloquea la aplicación automática. • E3: Error al persistir la nueva versión del diagrama en la base de datos documental.
Prototipo	

CU-07 Iniciar un nuevo trámite desde plataforma web o móvil

Elemento	Descripción
Propósito	Permitir que el usuario final solicite formalmente un servicio o proceso, proporcionando la información y requisitos necesarios de acuerdo al diseño establecido por la organización.
Actores	A3
Actor Iniciador	A3.
Precondición	El usuario debe estar autenticado en la aplicación móvil (Flutter) o plataforma web, y el proceso de negocio debe estar en estado "Publicado" en el catálogo de servicios.
Flujo Principal	1. El usuario navega por el catálogo de trámites disponibles y selecciona el proceso deseado. 2. El sistema recupera el esquema JSON del formulario dinámico (configurado en CU-03) desde MongoDB. 3. El sistema renderiza automáticamente la interfaz de captura de datos en el dispositivo del usuario. 4. El usuario completa los campos obligatorios y adjunta los documentos digitales requeridos. 5. El sistema valida en tiempo real que los datos cumplan con las reglas de negocio definidas (formatos, obligatoriedad). 6. El usuario selecciona la opción "Enviar Solicitud". 7. El sistema genera un código único de seguimiento (ID de Trámite) y persiste la instancia del proceso en la base de datos. 8. El sistema notifica al usuario el éxito de la operación y asigna el trámite a la bandeja del funcionario correspondiente (CU-08).
Postcondición	Se crea una instancia de trámite activa dentro del flujo de trabajo, iniciando el conteo de los tiempos de servicio (SLA).

Excepciones	• E1: El usuario no cuenta con los requisitos mínimos; el sistema bloquea el envío y detalla los documentos faltantes. • E2: Error en la carga de archivos por límite de tamaño o formato no permitido. • E3: Fallo en la persistencia del trámite por pérdida de conexión; el sistema guarda un borrador local si la plataforma lo permite.
Prototipo	

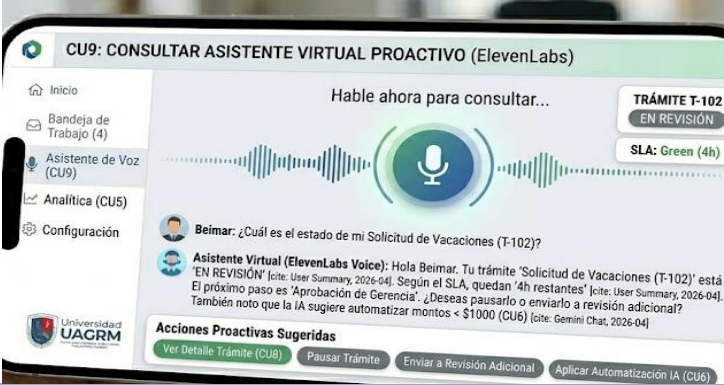
CU-08 Gestionar bandeja de trabajo y actualizar estados (SLA)

Elemento	Descripción
Propósito	Permitir que el funcionario gestione eficientemente las tareas asignadas, priorizando su ejecución según los Acuerdos de Nivel de Servicio (SLA) y actualizando los estados del trámite.
Actores	A2
Actor Iniciador	A2.
Precondición	El funcionario debe estar autenticado y existir trámites en su bandeja con estados "Pendiente" o "En proceso".
Flujo Principal	1. El funcionario accede a su "Bandeja de Trabajo" en la plataforma web. 2. El sistema recupera de MongoDB la lista de tareas asignadas a su perfil o grupo. 3. El sistema calcula y muestra visualmente el tiempo restante según el SLA (colores: verde, amarillo, rojo).

	4. El funcionario selecciona un trámite para trabajar. 5. El sistema renderiza el formulario dinámico con la información cargada por el Usuario Final (CU-07). 6. El funcionario completa los campos operativos y adjunta observaciones o documentos. 7. El funcionario selecciona el nuevo estado del trámite: "Terminado", "Retenido" o "Derivado". 8. El sistema persiste los cambios y mueve el trámite a la siguiente etapa del flujo.
Postcondición	El estado del trámite se actualiza en la base de datos y se libera el espacio en la bandeja del funcionario si la tarea fue finalizada.
Excepciones	• E1: El trámite ya fue tomado por otro funcionario (colisión de acceso); el sistema bloquea la edición. • E2: El SLA ha expirado; el sistema genera una alerta de criticidad automática. • *E3: Error al intentar guardar datos obligatorios incompletos.
Prototipo	

CU-09 Consultar al asistente virtual proactivo (ElevenLabs)

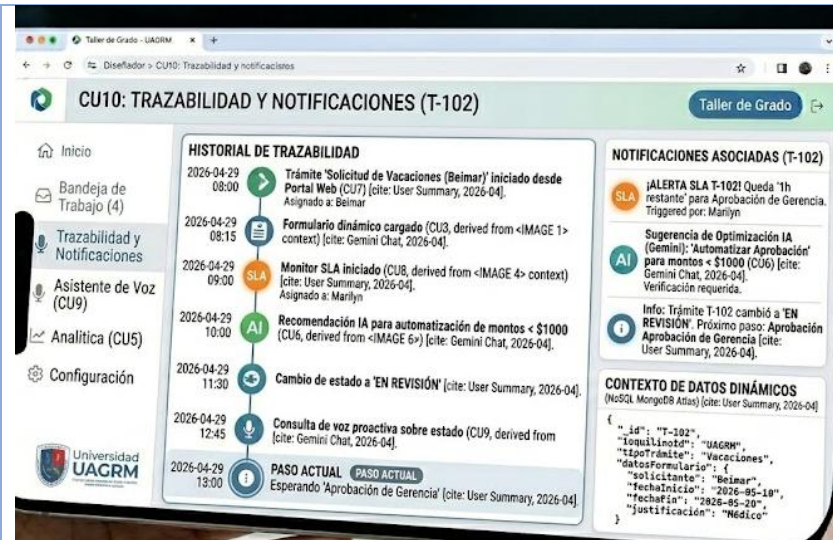
Elemento	Descripción
Propósito	Brindar asistencia cognitiva y guía técnica al funcionario mediante síntesis de voz en tiempo real, facilitando la ejecución de procesos complejos y reduciendo errores operativos.

Actores	A2
Actor Iniciador	A2.
Precondición	El funcionario debe estar dentro de una tarea activa y contar con salida de audio habilitada.
Flujo Principal	1. El funcionario activa el "Asistente de Guía Proactiva" en la interfaz de trabajo. 2. El sistema detecta el contexto de la actividad actual y las reglas de negocio asociadas. 3. El sistema envía el texto de ayuda al microservicio de FastAPI. 4. FastAPI procesa la solicitud y utiliza la API de ElevenLabs para generar el audio de guía. 5. El sistema reproduce la instrucción por voz (ej: "Para este trámite, asegúrese de verificar el sello de la notaría en el documento adjunto"). 6. El funcionario sigue las instrucciones mientras mantiene las manos libres para operar la interfaz. 7. El sistema permite repetir o pausar la instrucción mediante comandos rápidos.
Postcondición	El funcionario recibe asistencia auditiva sin interrumpir su flujo de trabajo visual.
Excepciones	* E1: Fallo de conexión con la API de ElevenLabs; el sistema muestra la ayuda únicamente en formato texto. * E2: El contexto de la tarea no tiene guía configurada; el sistema informa que no hay instrucciones disponibles.
Prototipo	

CU-10 Consultar trazabilidad de trámite y notificaciones

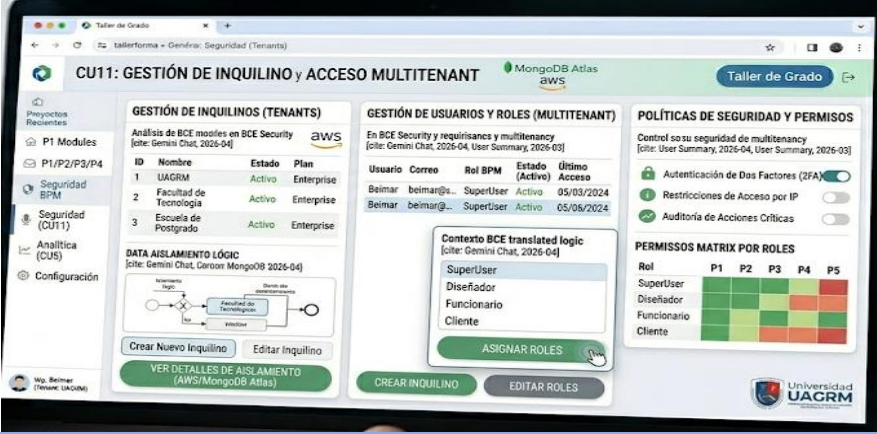
Elemento	Descripción
Propósito	Garantizar la transparencia del proceso permitiendo que el usuario final conozca en todo momento la ubicación, estado y responsable actual de su solicitud.
Actores	A3: Usuario Final.
Actor Iniciador	A3.
Precondición	El usuario debe tener al menos un trámite iniciado o haber recibido un código de seguimiento válido.
Flujo Principal	1. El usuario accede a la sección "Mis Trámites" en la App móvil (Flutter) o Web. 2. El sistema consulta en MongoDB el historial de movimientos y estados de la instancia del trámite. 3. El sistema muestra una línea de tiempo (Timeline) con las fechas, funcionarios y acciones realizadas. 4. El sistema destaca el estado actual (ej: "En revisión por Asesoría Legal"). 5. Si hay una notificación pendiente (ej: "Requisito rechazado"), el sistema resalta la acción requerida. 6. El usuario visualiza los comentarios públicos dejados por los funcionarios.
Postcondición	El usuario queda informado sobre el progreso de su solicitud, reduciendo la necesidad de consultas presenciales.
Excepciones	* E1: El código de trámite no existe; el sistema solicita verificar los datos. * E2: El trámite se encuentra en una etapa confidencial; el sistema muestra información limitada según la política de seguridad.

Prototipo



CU-11 Gestionar seguridad, perfiles y acceso multitenant

Elemento	Descripción
Propósito	Administrar el acceso al sistema bajo una arquitectura Multitenant, asegurando que los datos de diferentes organizaciones estén aislados y que cada usuario tenga los permisos adecuados según su rol.
Actores	A1
Actor Iniciador	A1.
Precondición	El administrador debe tener privilegios de "SuperUser" o "Admin de Organización".
Flujo Principal	1. El administrador accede al módulo de "Gestión de Identidades y Organización" 2. El sistema permite crear, editar o dar de baja usuarios vinculándolos a un Tenant_ID específico. 3. El administrador asigna roles (Diseñador, Funcionario, Auditor) y permisos granulares sobre los procesos. 4. El sistema configura las políticas de seguridad (complejidad de contraseñas, sesiones simultáneas). 5. El sistema asegura que las consultas a MongoDB incluyan siempre el filtro de Tenant_ID para garantizar el aislamiento de datos. 6. El administrador audita los logs de acceso y cambios realizados en las políticas de negocio.

Postcondición	La configuración de seguridad queda persistida, garantizando la integridad y privacidad de la información de la organización.
Excepciones	* E1: Intento de crear un usuario con un correo ya existente en otro tenant. * E2: El administrador intenta eliminarse a sí mismo; el sistema bloquea la acción para evitar la orfandad del sistema.
Prototipo	

Ciclo # 2

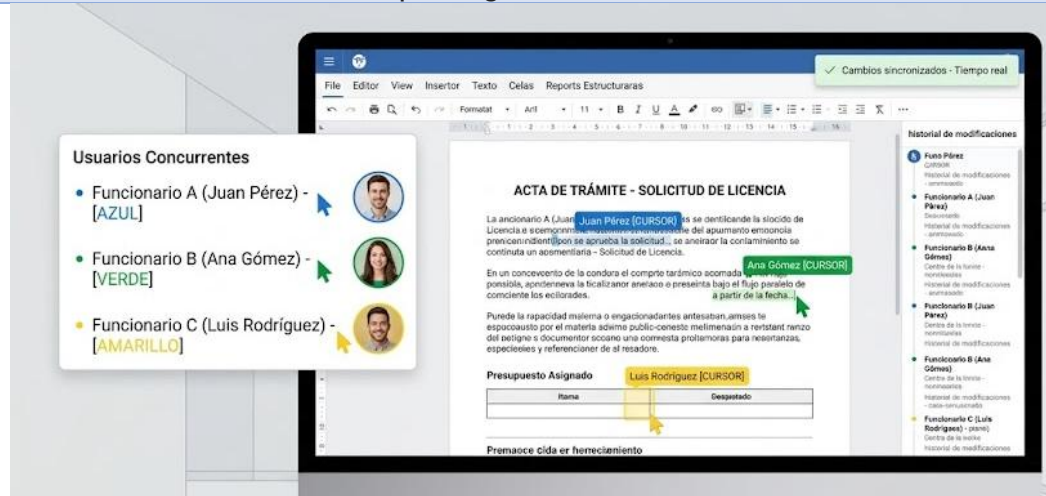
CU12 Subir requisitos digitales al trámite

Elemento	Descripción
Propósito	Permitir que el Funcionario (A2) o el Usuario Final (A3) adjunten requisitos en formatos digitales híbridos asociados de forma unívoca a un nodo de atención dentro del flujo de trabajo, sustituyendo por completo el uso de papel y centralizando la documentación en un repositorio elástico y seguro.
Actores	A2, A3
Actor Iniciador	A2, A3
Precondición	El usuario debe estar autenticado, la instancia del trámite debe encontrarse en estado "En Proceso", el nodo actual debe tener configurados requisitos documentales obligatorios u opcionales por el Diseñador, y las credenciales de conexión con el servicio de infraestructura externa de Amazon S3 deben estar activas.
Flujo Principal	1. El usuario accede a la tarea o bandeja del trámite actual desde la interfaz web (Angular) o móvil (Flutter). 2. El sistema lee el esquema dinámico de la actividad y renderiza la lista de requisitos documentales solicitados para ese nodo.

	3. El usuario selecciona la opción de adjuntar archivo y elige un documento digital híbrido desde el almacenamiento local de su dispositivo (formatos admitidos: Word, Excel, PDF, imágenes o videos). 4. El sistema valida localmente las restricciones de tipo de archivo y que el tamaño no exceda el límite máximo configurado. 5. El usuario confirma la subida; el sistema captura el archivo y lo transmite en un flujo binario multipartes (MultipartFile) hacia el backend en Spring Boot. 6. El servidor de Spring Boot procesa la petición y delega de manera asíncrona la transferencia del archivo masivo hacia el bucket externo en Amazon S3, el cual retorna una URL única y segura de acceso. 7. El sistema registra y actualiza los metadatos del trámite en la base de datos documental (identificador, nombre, formato, fecha y la URL de Amazon S3), asociándolo al nodo y actor correspondiente. 8. El sistema notifica al usuario el éxito de la operación y marca el requisito como "Cargado" en la interfaz visual.
Postcondición	Los requisitos digitales quedan respaldados con persistencia redundante en la nube de Amazon S3 y los metadatos quedan indexados de forma elástica en la base de datos documental, quedando disponibles para auditoría o revisión en los siguientes nodos del flujo.
Excepciones	• E1: Tipo de archivo o tamaño no permitido • E2: Falla de red durante la transferencia en modo offline • E3: Error de autenticación o de conexión con Amazon S3
Prototipo	

CU13 Editar documentos colaborativamente en tiempo real

Elemento	Descripción
Propósito	Permitir que múltiples funcionarios (A2) asignados al mismo trámite operen, modifiquen y redacten simultáneamente el contenido de un documento digital adjunto (ej: Word, Excel o actas de trámite), visualizando los cambios de forma instantánea y síncrona, eliminando los conflictos de versiones e incrementando la productividad bajo flujos paralelos.
Actores	A2
Actor Iniciador	A2
Precondición	El usuario debe estar autenticado, poseer permisos de edición (RBAC) asignados por el Diseñador para el nodo actual del trámite, el documento debe estar pre-cargado (alojado en Amazon S3) y el canal de comunicación bidireccional asíncrona (WebSockets) debe estar activo y estable en el backend de Spring Boot.
Flujo Principal	1. El funcionario accede al repositorio documental del trámite y selecciona la opción de edición remota sobre un documento existente. 2. El sistema web/móvil inicializa una conexión de red persistente mediante el protocolo WebSocket suscrita al canal (topic) exclusivo del documento. 3. El sistema renderiza un lienzo de edición interactiva y muestra indicadores visuales de presencia y cursores con los nombres de otros funcionarios concurrentes integrados a la sesión. 4. El funcionario realiza una modificación en el documento (ej: insertar texto, modificar una celda o estructurar un informe técnico). 5. El sistema captura la mutación del archivo de forma asíncrona en milisegundos y transmite el delta del cambio en formato JSON (u operaciones operacionales transformadas) a través del WebSocket hacia el servidor. 6. El backend en Spring Boot valida los tokens de acceso, procesa la colisión de eventos mediante algoritmos de sincronización y redistribuye instantáneamente la actualización a todos los clientes concurrentes suscritos al canal. 7. La interfaz del sistema (Angular/Flutter) de cada participante renderiza automáticamente el cambio en tiempo real reflejando la actualización exacta sin necesidad de recargar la pantalla. 8. El sistema persiste el estado actualizado del documento de forma diferida o al cerrar la sesión directamente en la infraestructura elástica.

Postcondición	El documento digital del trámite se mantiene sincronizado, consistente y actualizado en tiempo real para todos los funcionarios en concurrencia, resguardando el historial unificado de modificaciones.
Excepciones	• E1: Interrupción abrupta del canal WebSocket (Pérdida de red) • E2: Conflicto extremo por edición simultánea en el mismo caracter/celda • E3: Intento de edición sin privilegios en el nodo
Prototipo	 The screenshot displays a web application interface. On the left, a panel titled 'Usuarios Concurrentes' (Concurrent Users) lists three users: 'Funcionario A (Juan Pérez) - [AZUL]', 'Funcionario B (Ana Gómez) - [VERDE]', and 'Funcionario C (Luis Rodríguez) - [AMARILLO]'. Each user is accompanied by a circular profile picture and a colored cursor icon. The main area shows a document titled 'ACTA DE TRÁMITE - SOLICITUD DE LICENCIA'. The document text includes fields for 'Presupuesto Asignado' and 'Prestación de servicio'. A yellow cursor is visible over the 'Presupuesto Asignado' field. On the right, a sidebar titled 'Historial de modificaciones' (Modification History) shows a list of changes made by the three users, including actions like 'Crear', 'Actualizar', and 'Eliminar'.

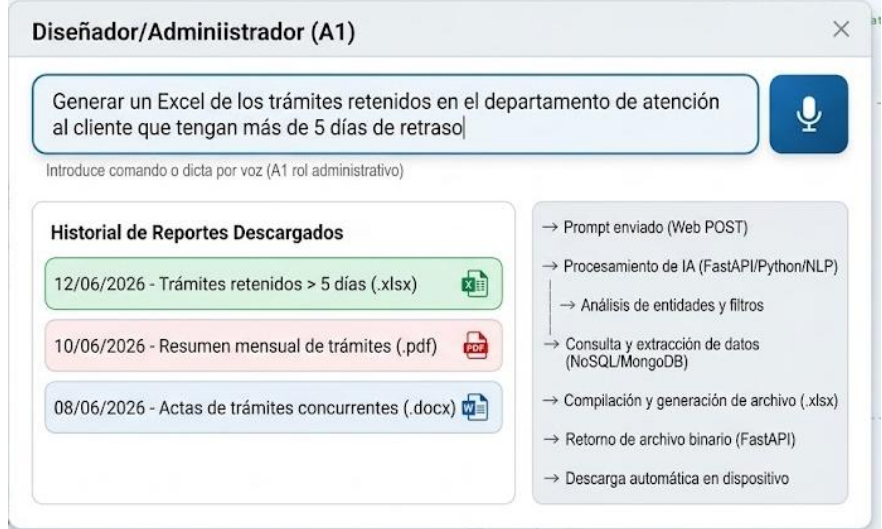
CU14 Respaldo archivos masivos en infraestructura externa

Elemento	Descripción
Propósito	Delegar de forma transparente el almacenamiento físico y persistencia de todos los archivos digitales e híbridos pesados (Word, Excel, PDF, imágenes o videos) cargados durante un trámite hacia un bucket elástico de Amazon S3, liberando espacio en el servidor de aplicaciones, optimizando los costos de infraestructura y garantizando una alta velocidad de transferencia y redundancia de datos.
Actores	A2 , A3
Actor Iniciador	A2, A3
Precondición	El usuario debe haber iniciado un proceso de carga de archivos (ej: en el CU12 o CU13), el backend en Spring Boot debe tener correctamente inicializado y autenticado el SDK de AWS (Amazon Web Services), y los permisos de escritura/lectura en las políticas del bucket de destino en Amazon S3 deben estar vigentes.
Flujo Principal	1. El usuario (A2/A3) confirma la subida o modificación de un archivo digital desde las interfaces del sistema (Angular/Flutter).

	2. El backend en Spring Boot recibe el flujo binario del archivo a través de una petición HTTP multipartes (MultipartFile). 3. El sistema extrae los metadatos iniciales del archivo (nombre, tamaño original, tipo MIME) y genera un identificador único global (UUID) para evitar colisiones de nombres en la nube. 4. El backend invoca al cliente de AWS S3 (S3Client) y abre un flujo de transferencia directa de datos empaquetados hacia el bucket configurado en Amazon Web Services. 5. La infraestructura de Amazon S3 procesa el flujo, almacena el objeto de manera replicada en múltiples zonas de disponibilidad y retorna un código de éxito HTTP 200 junto con la clave (key) del objeto y su URL pública/firmada de acceso. 6. El sistema captura la respuesta de AWS y actualiza de inmediato el repositorio documental en la base de datos NoSQL, persistiendo únicamente los metadatos livianos y la URL de acceso de S3 vinculados al trámite. 7. El sistema confirma la persistencia de manera exitosa y el archivo queda disponible para ser visualizado o descargado en las interfaces de usuario de forma inmediata.															
Postcondición	El archivo físico masivo queda resguardado de forma permanente y redundante en la infraestructura elástica de Amazon S3, mientras que la base de datos transaccional solo almacena la referencia lógica (metadatos livianos y URL), asegurando la escalabilidad del sistema.															
Excepciones	• E1: Error de red o timeout de conexión con los servidores de AWS • E2: Superación de cuota de almacenamiento o falta de privilegios en el bucket de AWS • E3: Archivo corrupto o interrumpido a mitad de transferencia															
Prototipo	Adjuntar archivo opcional Seleccionar archivo del dispositivo <table><thead><tr><th>Archivo</th><th>Actors</th><th>Tamaño</th><th>MIME</th><th>Check verde</th></tr></thead><tbody><tr><td> Plano.pdf</td><td>Required</td><td>13.0 MB</td><td>microoarchivo</td><td></td></tr><tr><td> Foto.jpg</td><td>Opcional</td><td>33.6 bytes</td><td>JPEG-intenal</td><td></td></tr></tbody></table>	Archivo	Actors	Tamaño	MIME	Check verde	 Plano.pdf	Required	13.0 MB	microoarchivo		 Foto.jpg	Opcional	33.6 bytes	JPEG-intenal	
Archivo	Actors	Tamaño	MIME	Check verde												
 Plano.pdf	Required	13.0 MB	microoarchivo													
 Foto.jpg	Opcional	33.6 bytes	JPEG-intenal													

CU15 Generar reportes dinámicos mediante lenguaje natural

Elemento	Descripción
Propósito	Permitir que el Diseñador/Administrador (A1) solicite la creación de reportes analíticos complejos y personalizados mediante comandos conversacionales de texto o voz, delegando en un motor de Inteligencia Artificial (NLP) la tarea de interpretar la petición, extraer los datos requeridos, aplicar criterios de filtrado y estructurar el archivo de salida en tiempo real.
Actores	A1
Actor Iniciador	A1
Precondición	El usuario debe estar autenticado con rol administrativo, la interfaz web de Angular debe tener acceso al micrófono (si es por voz) y el microservicio cognitivo de Inteligencia Artificial en FastAPI (Python) debe estar en línea y conectado a la persistencia de datos.
Flujo Principal	1. El administrador accede al panel de reportes analíticos del sistema. 2. El usuario introduce una instrucción en lenguaje natural (ejemplo por texto o dictado por voz: "Generar un Excel de los trámites retenidos en el departamento de atención al cliente que tengan más de 5 días de retraso"). 3. El sistema web captura la cadena de texto (o transcribe el audio a texto mediante un servicio de reconocimiento de voz) y envía el prompt mediante una petición HTTP POST al backend en FastAPI. 4. El motor de IA en FastAPI procesa el texto libre utilizando Modelos de Lenguaje (NLP) y ejecuta el análisis en tres dimensiones estrictas: a) Datos solicitados (entidades y atributos), b) Criterios y condiciones (filtros de tiempo, estados, agrupaciones), y c) Formato de salida (Excel, Word, PDF). 5. El backend de IA traduce esta abstracción cognitiva en una consulta estructurada y extrae la información requerida desde la base de datos documental de manera óptima. 6. El microservicio de Python procesa el conjunto de datos recuperado y, utilizando librerías nativas de automatización

	documental, compila y construye dinámicamente el archivo en el formato solicitado (ej: .xlsx, .docx, .pdf). 7. FastAPI retorna el archivo binario generado al frontend de Angular junto con los metadatos de la operación. 8. El sistema descarga automáticamente el reporte en el dispositivo del administrador y muestra un mensaje de éxito en la interfaz visual.
Postcondición	El administrador obtiene un reporte analítico estructurado y filtrado a medida directamente en su almacenamiento local, generado exclusivamente a partir de una instrucción conversacional libre.
Excepciones	• E1: Instrucción ambigua o indescifrable para la IA • E2: Búsqueda sin resultados concurrentes • E3: Error de formato no soportado
Prototipo	

CU16 Evaluar enrutamiento predictivo y análisis de riesgos

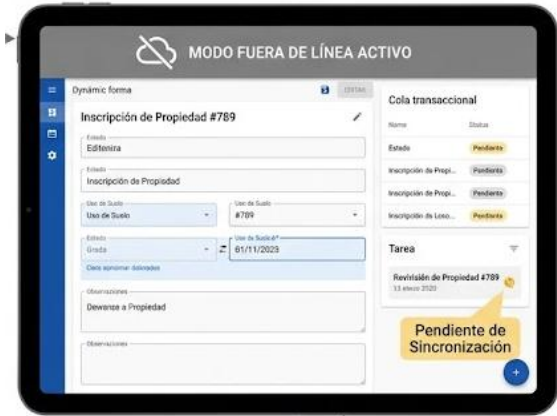
Elemento	Descripción
Propósito	Permitir que el Funcionario (A2) o el Usuario Final (A3) sometan un trámite al motor analítico de Inteligencia Artificial para que, mediante modelos de Deep Learning, se prediga automáticamente la ruta óptima de resolución, se evalúe el riesgo de retrasos o incumplimiento de SLA y se detecten anomalías en los datos, optimizando la asignación de recursos.
Actores	A2, A3
Actor Iniciador	A2, A3

Precondición	El usuario debe estar autenticado, existir un trámite activo con datos cargados o requisitos digitales adjuntos (CU12), y los modelos predictivos neuronales entrenados en el backend de FastAPI (Python) deben estar completamente operativos y comunicados con el repositorio de datos.
Flujo Principal	1. El usuario interactúa con la plataforma para enviar un trámite al siguiente nodo, o un cliente (A3) introduce un caso inicial mediante comandos de voz/texto con el Agente de IA. 2. El backend transaccional (Spring Boot) recopila el estado actual, el historial del trámite y el contenido de los formularios dinámicos asociados, enviando el payload al servicio de IA en FastAPI. 3. El motor de Deep Learning analiza los metadatos e historial en tiempo real frente a los patrones de ejecución del negocio. 4. El sistema evalúa el enrutamiento inteligente, prediciendo y seleccionando de forma automática cuál es la calle (swimlane) o la secuencia de nodos óptima para agilizar el trámite. 5. El sistema ejecuta el análisis predictivo de riesgos, calculando la probabilidad de que el trámite sufra un cuello de botella o quede en estado "Retenido" excediendo los tiempos estipulados (SLA). 6. El modelo de IA escanea los datos en busca de anomalías (inconsistencias en los documentos o fraude) y asigna un puntaje (score) de riesgo operativo al trámite. 7. FastAPI retorna la matriz de predicciones, alertas de cuellos de botella y la ruta sugerida al backend transaccional y a la interfaz (Angular/Flutter). 8. El sistema redirige automáticamente el flujo al nodo óptimo, prioriza la tarea en la bandeja del funcionario según el riesgo y visualiza las alertas analíticas en pantalla.
Postcondición	El trámite es enrutado de manera inteligente y priorizado dinámicamente en el sistema, quedando registrado el reporte predictivo de riesgos y alertas de SLA en la persistencia documental para auditorías del negocio.
Excepciones	• E1: Datos insuficientes para la predicción • E2: Detección crítica de anomalía o riesgo extremo de fraude • E3: Error de timeout en el microservicio de Deep Learning

Prototipo	Bandeja de Trabajo de Trámites			
	Trámites			
	ID	Trámite	Estado	Riesgo Predictivo
	#1234	Solicitud de Permiso Documento	Confirmado	78%
	#5678	Actualización de Datos Documento	Aprobado	50%
	#9012	Trámite de Licencia Documento	Conformado	78%
	#1234	Solicitud de Permiso Documento	Prevision	75%
	#5678	Actualización de Datos Documento	Rormato	100%
	#5302	Solicitud Pangu Documento	Medicina	90%
	#9012	Trámite de Licencia	Previsino	80%

CU17 Procesar tareas pendientes en modo fuera de línea

Elemento	Descripción
Propósito	Permitir que el Funcionario (A2) continúe operando, revisando tareas pendientes y llenando formularios dinámicos en su dispositivo móvil o cliente web aun cuando no exista conectividad a Internet, asegurando la resiliencia del software y evitando la paralización de la atención de trámites en campo.
Actores	A2
Actor Iniciador	A2
Precondición	El funcionario debe haber iniciado sesión previamente con conectividad activa para heredar la caché de seguridad (tokens JWT) y haber descargado los datos iniciales de su bandeja de trabajo (Worklist), esquemas de formularios y políticas de negocio asociadas.
Flujo Principal	1. El sistema detecta mediante sus servicios de monitoreo de red internos la pérdida total del enlace a Internet. 2. El sistema móvil (Flutter) o web (PWA) cambia automáticamente su estado interno a Modo Offline y despliega una sutil advertencia visual en la interfaz del funcionario. 3. El funcionario accede a su bandeja de trabajo; el sistema intercepta la petición HTTP y, en lugar de consultar al backend, recupera de forma inmediata las tareas pre-cargadas desde el motor de persistencia local en caché del dispositivo.

	4. El funcionario selecciona una tarea pendiente; el sistema lee el esquema del formulario dinámico almacenado localmente en formato JSON y lo renderiza de manera idéntica en pantalla. 5. El funcionario procesa la tarea llenando los campos correspondientes, modificando valores y ejecutando el avance del nodo en la calle de atención. 6. Al presionar "Confirmar", el sistema valida localmente que los datos ingresados cumplan con las reglas restrictivas de negocio del esquema. 7. El sistema congela la transacción localmente: empaqueta el payload en un JSON estructurado, lo almacena en una cola transaccional indexada dentro de la base de datos local embebida y cambia el estado visual de la tarea a "Pendiente de Sincronización". 8. El sistema notifica al operador que la tarea fue procesada localmente de manera exitosa y que se resguardó de forma segura hasta restaurar el canal de comunicación.
Postcondición	Las transacciones ejecutadas en desconexión quedan almacenadas con integridad criptográfica en la persistencia local del dispositivo, garantizando cero pérdida de datos y permitiendo al funcionario seguir operando de forma continua.
Excepciones	• E1: Intento de procesar una tarea que no fue pre-cargada en la sesión activa • E2: Fallo de almacenamiento por espacio insuficiente en el dispositivo físico • E3: Expiración local estricta del token de seguridad (SLA/Seguridad)
Prototipo	

CU18 Sincronizar transacciones diferidas acumuladas en el dispositivo

Elemento	Descripción
Propósito	Transmitir de manera transaccional, ordenada y asíncrona todas las operaciones, flujos de datos y cambios de estado de trámites que fueron procesados y congelados localmente en el dispositivo durante el modo fuera de línea, impactando el backend central para restablecer la consistencia global del sistema sin intervención manual del usuario.
Actores	A2
Actor Iniciador	A2
Precondición	El dispositivo debe contar con transacciones encoladas en su persistencia local en estado "Pendiente de Sincronización" (CU17), y los servicios de conectividad deben detectar un canal de red estable con los endpoints del backend de Spring Boot.
Flujo Principal	1. El servicio de monitoreo de red de la aplicación (Connectivity Lifecycle) detecta que el dispositivo ha recuperado una conexión a Internet estable y con ancho de banda suficiente. 2. El sistema remueve de forma sutil el estado visual de "Modo Offline" y activa en segundo plano (background thread) el motor de sincronización diferida. 3. La aplicación lee de manera secuencial la cola de transacciones indexadas en la base de datos local (SQLite/Hive/IndexedDB) respetando estrictamente el orden cronológico en que fueron creadas (enfoque FIFO: First In, First Out) para evitar colisiones lógicas. 4. El sistema empaqueta el payload de la primera transacción diferida e inyecta los tokens de seguridad vigentes (JWT), enviando la petición HTTP POST/PUT al servidor central. 5. El backend en Spring Boot recibe el paquete, valida la integridad de los datos frente a las reglas de negocio del Workflow, ejecuta la persistencia definitiva en la base de datos documental y dispara las alertas del ciclo de vida del trámite. 6. El servidor central retorna un código HTTP 200 de confirmación exitosa al dispositivo. 7. Al recibir la confirmación, el sistema local elimina de forma permanente dicha transacción de la caché interna del

	dispositivo y actualiza el estado visual de la tarea a "Sincronizado". 8. El sistema repite cíclicamente los pasos 4, 5, 6 y 7 hasta vaciar por completo la cola de transacciones diferidas, notificando al funcionario el éxito total del proceso mediante un sutil indicador en pantalla.
Postcondición	La base de datos central y el estado global de los trámites quedan completamente unificados, actualizados y consistentes con las acciones ejecutadas en campo; la memoria caché local del dispositivo se libera de cargas temporales.
Excepciones	• E1: Interrupción del canal de red a mitad del proceso de sincronización. • E2: Conflicto de concurrencia en el servidor (El trámite fue alterado en la base de datos central por otro usuario mientras el dispositivo estaba offline) • E3: Rechazo por token de seguridad expirado durante la desconexión
Prototipo	Sincronizando 3 tareas pendientes... 3/3 100% Indicador sutil de éxito - Revisión #456 [Check Verde] - Solicitud #789 [Check Verde] - Actualización #101 [Check Verde]

3.FLUJO DE TRABAJO: ANALISIS

3.1. ANALISIS DE ARQUITECTURA

3.1.1. IDENTIFICAR PAQUETES

Paquete # 1. Gestión y configuración inteligente

Gestión y configuración inteligente

Descripción: Este paquete constituye el núcleo administrativo y de diseño de la plataforma. Su función es proporcionar las herramientas avanzadas para que el Diseñador (A1) estructure las políticas de negocio en calles (*swimlanes*), orqueste las transacciones base en Spring Boot, configure los formularios dinámicos y garantice la seguridad del entorno multi-entidad (*Multi-tenancy*).

Paquete #2. Operación, Movilidad y Seguimiento de tramites

Operación, Movilidad y Seguimiento de tramites

Descripción: Este paquete representa la capa ejecutiva, móvil y de interacción con el usuario. Su objetivo es materializar las definiciones lógicas del sistema en instancias de trabajo funcionales para el Funcionario (A2) y el Usuario Final (A3). Incorpora el control de estados del trámite, el manejo de bandejas de trabajo con alertas de SLA y, críticamente, la arquitectura híbrida para soportar el procesamiento en modo fuera de línea (*Offline*) y la sincronización diferida.

Paquete #3: Gestión Documental Concurrente

Gestión Documental Concurrente

Descripción: Subsistema crítico introducido para digitalizar los flujos empresariales y eliminar por completo el uso de papel. Se encarga de administrar el repositorio de requisitos digitales híbridos (Word, Excel, PDF, imágenes, videos), controlar de forma estricta los privilegios de acceso (RBAC) por nodo de atención y resolver la simultaneidad mediante canales bidireccionales (*WebSockets*) para permitir la edición colaborativa concurrentes de archivos en tiempo real.

Paquete #4: Servicios Cognitivos y Analítica Predictiva

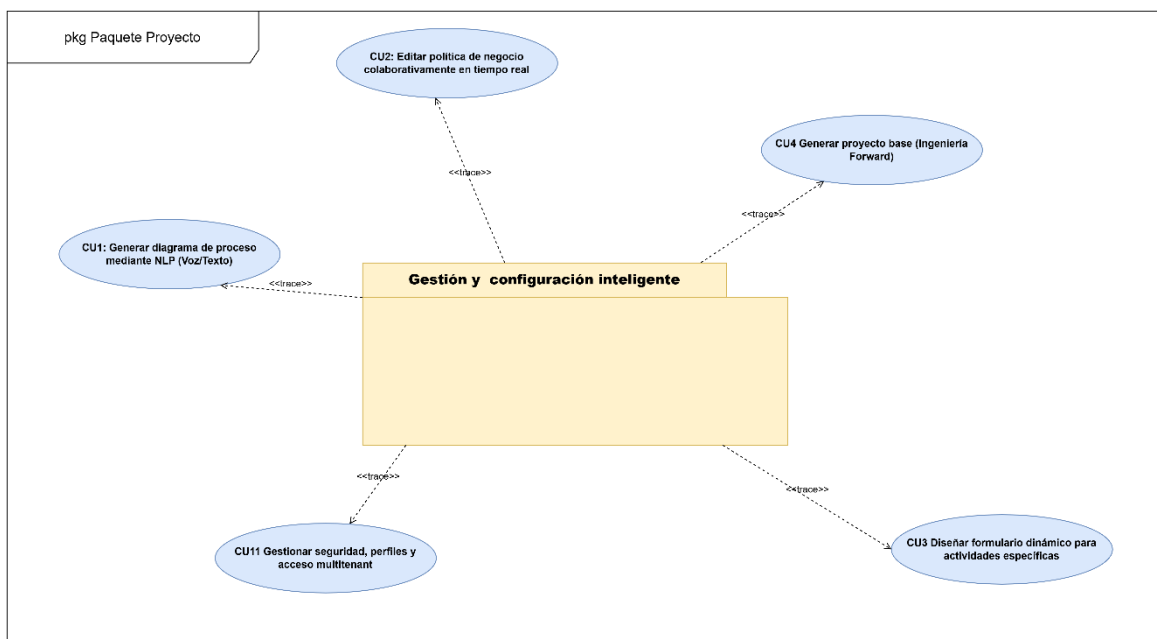
Servicios Cognitivos y Analítica Predictiva

Descripción: Representa la capa de Inteligencia Artificial avanzada del ecosistema, desarrollada de forma desacoplada en Python con FastAPI. Aloja el Agente Inteligente para la atención automatizada, los modelos de *Deep Learning*

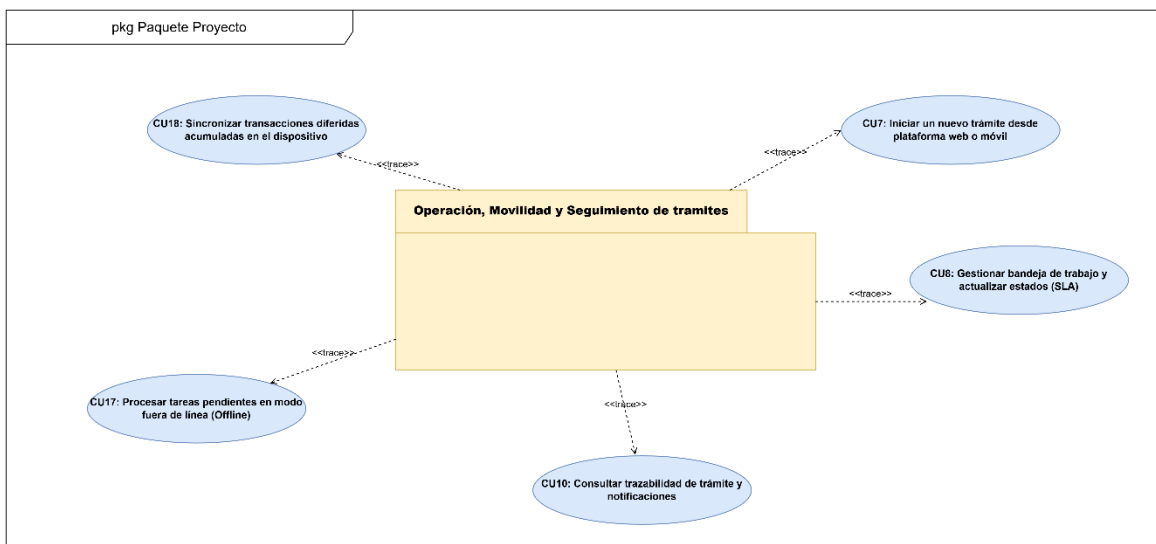
encargados de ejecutar el enrutamiento óptimo y análisis predictivo de riesgos/anomalías, los servicios conversacionales de audio y el motor de procesamiento de lenguaje natural (NLP) encargado de interpretar las peticiones del administrador para construir reportes dinámicos bajo demanda.

3.1.2. RELACIONAR PAQUETES Y CASOS DE USO

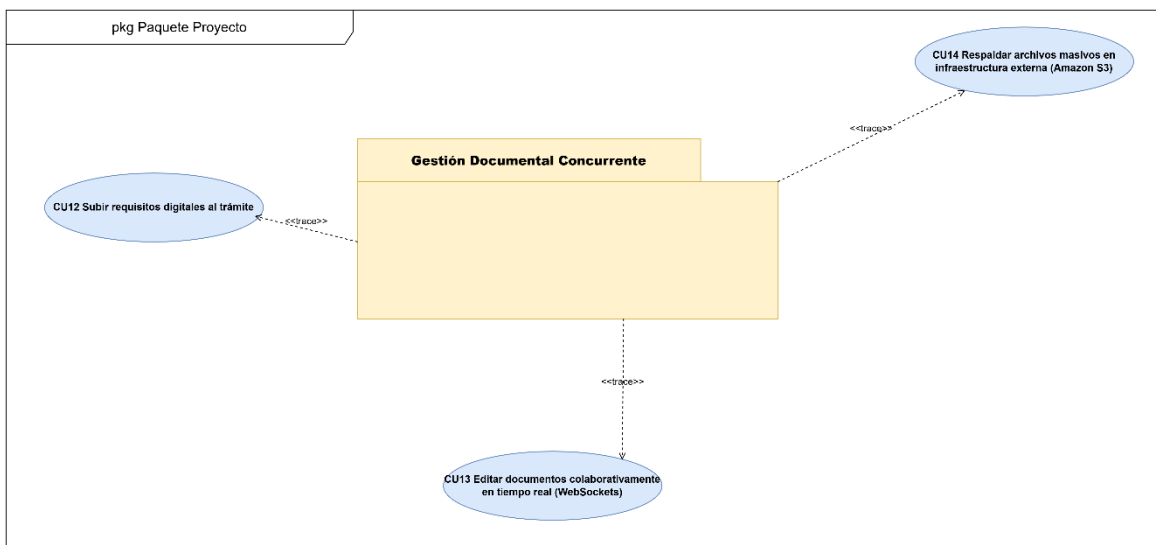
Paquete # 1. Gestión y configuración inteligente



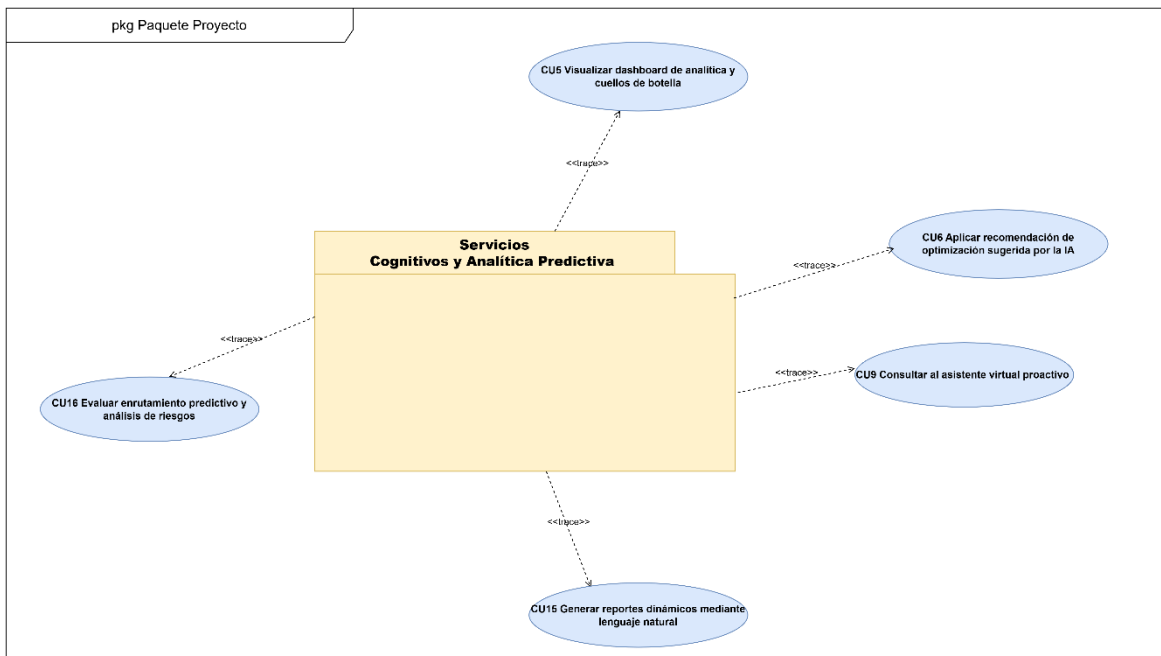
Paquete #2. Operación, Movilidad y Seguimiento de tramites



Paquete #3: Gestión Documental Concurrente

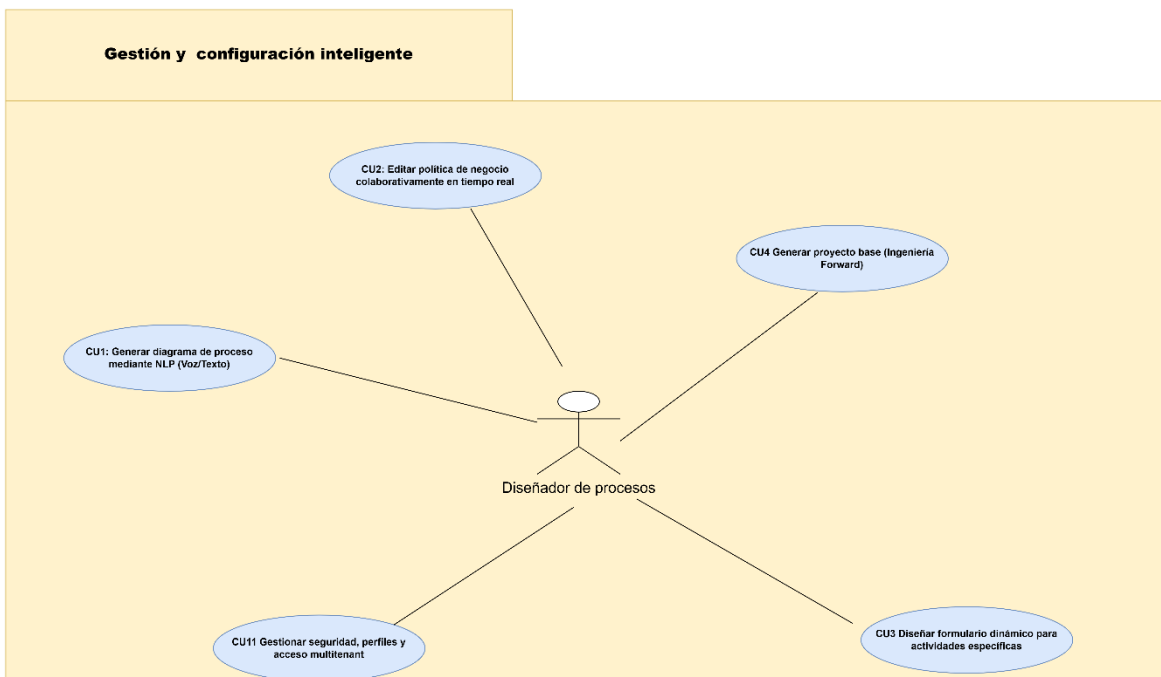


Paquete #4: Servicios Cognitivos y Analítica Predictiva

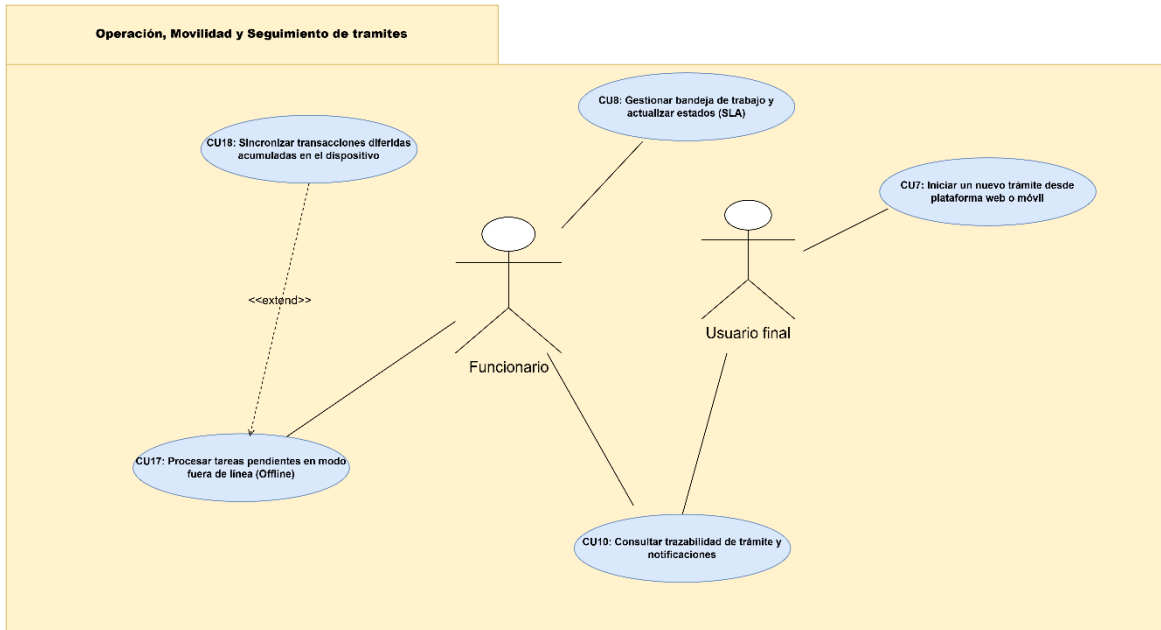


3.1.3. VISTA DE CASOS DE USO

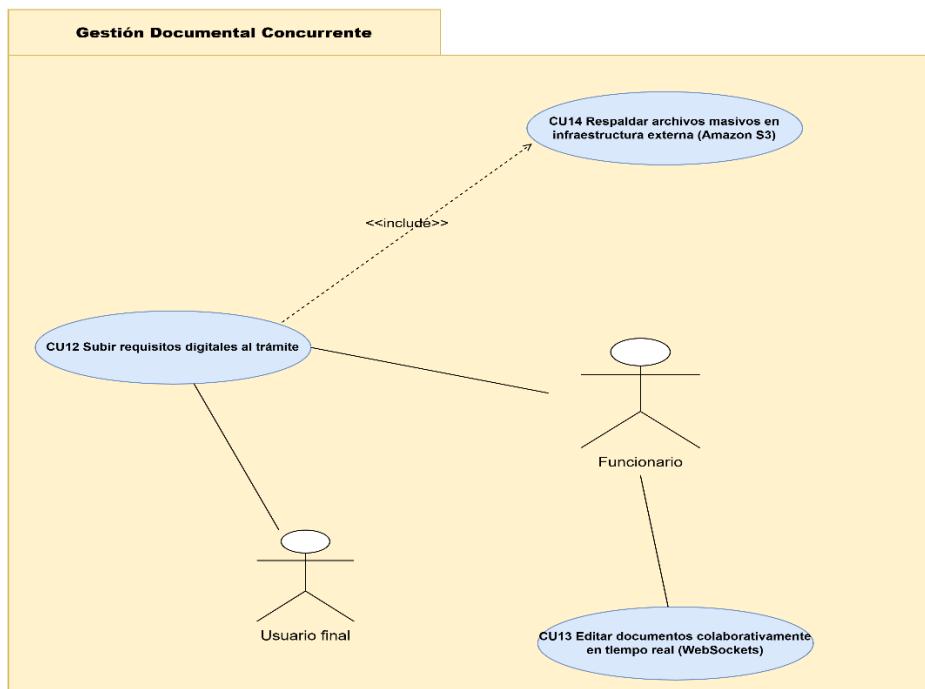
Paquete # 1. Gestión y configuración inteligente



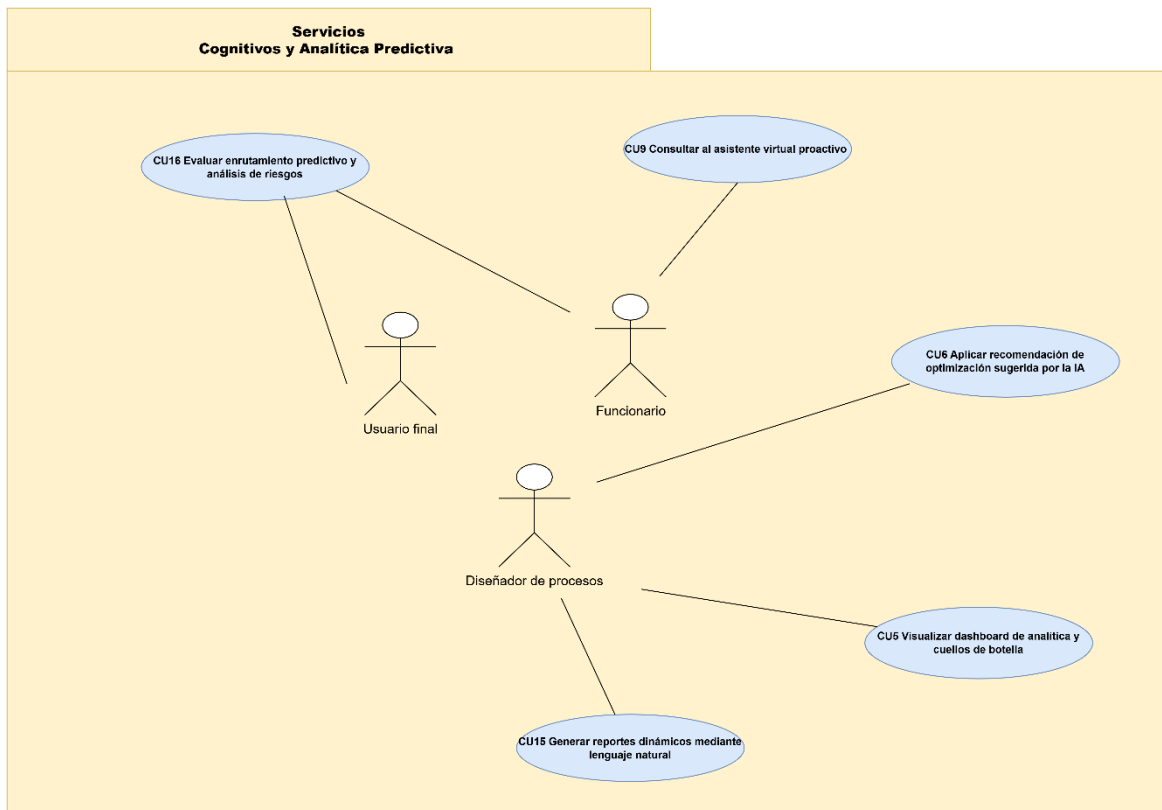
Paquete #2. Operación, Movilidad y Seguimiento de tramites



Paquete #3: Gestión Documental Concurrente



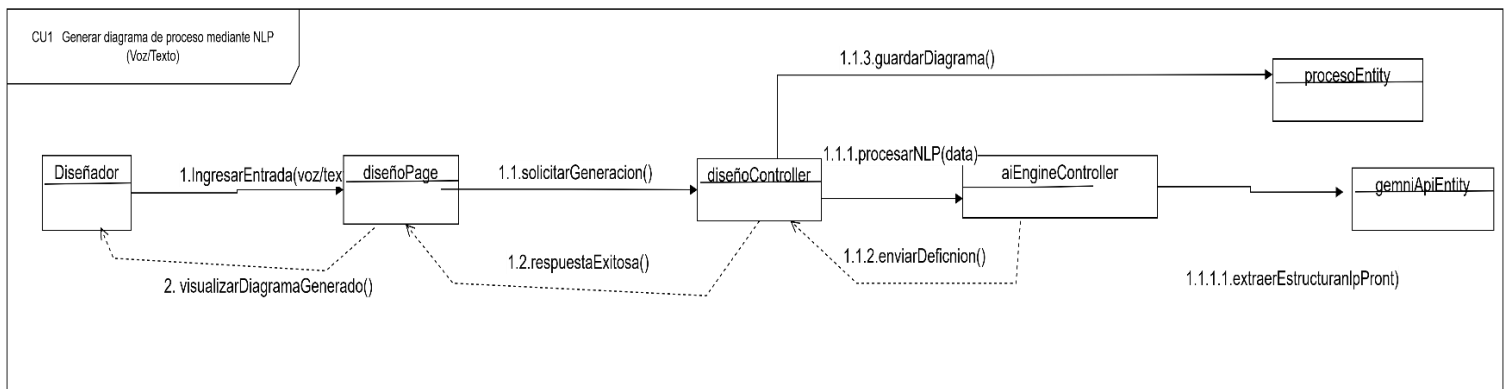
Paquete #4: Servicios Cognitivos y Analítica Predictiva



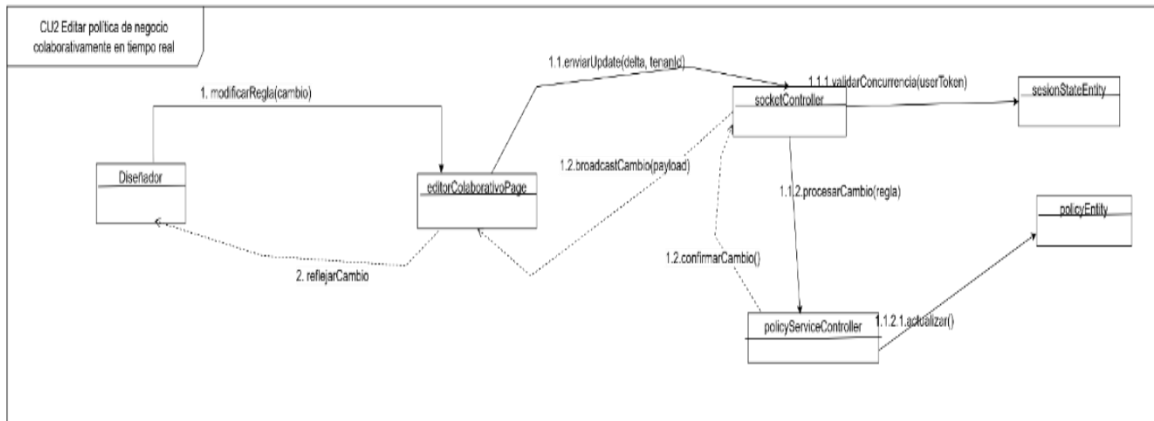
3.2. ANALISIS DE CASOS DE USO DIAGRAMA DE COMUNICACIÓN

Ciclo # 1

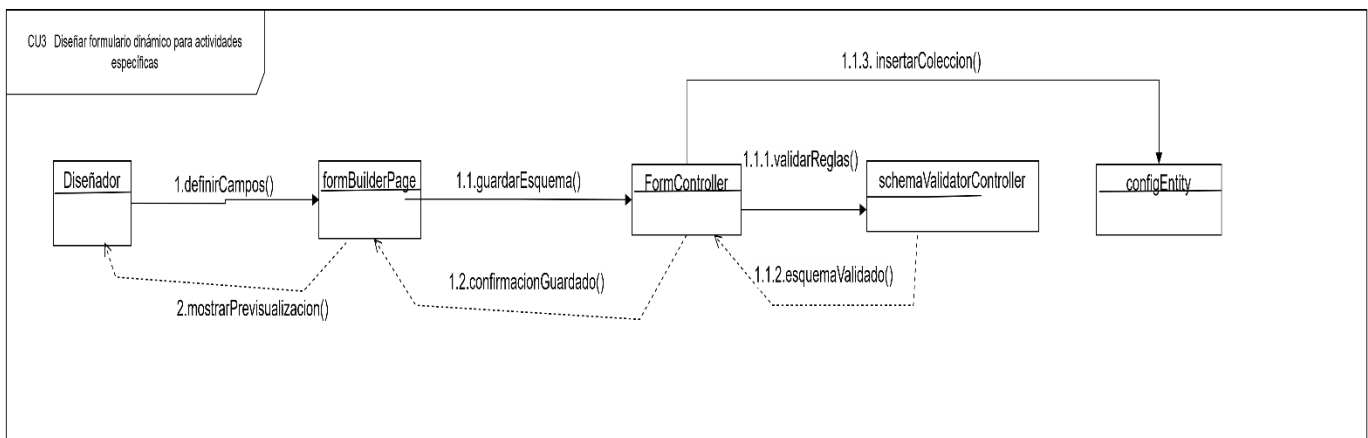
CU1 Generar diagrama de proceso mediante NLP (Voz/Texto)



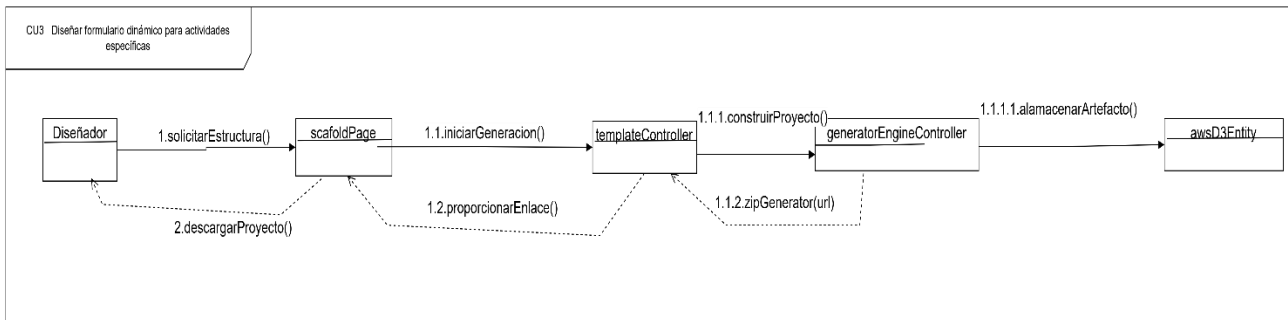
CU2 Editar política de negocio colaborativamente en tiempo real



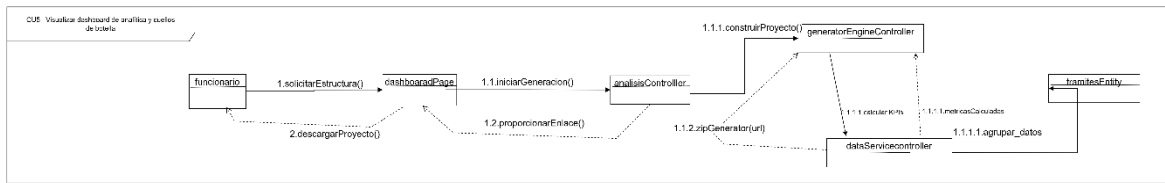
CU3 Diseñar formulario dinámico para actividades específicas



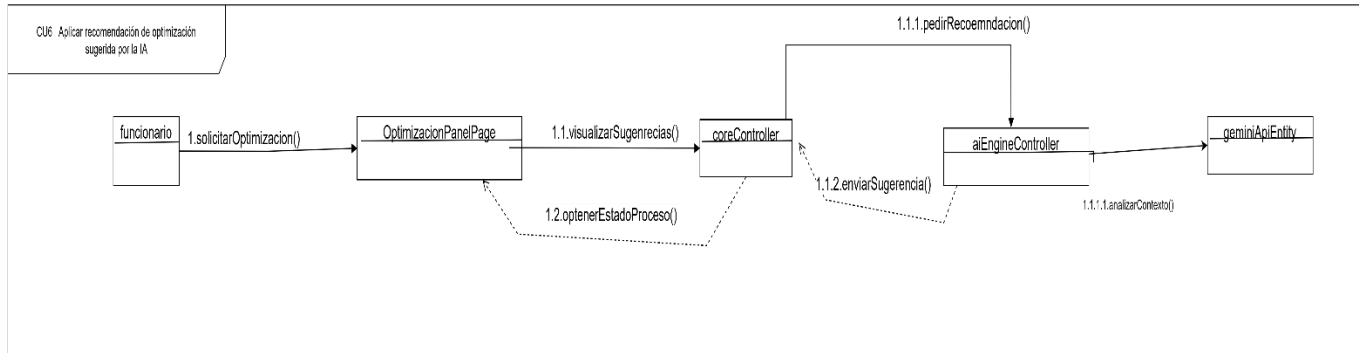
CU4 Generar proyecto base



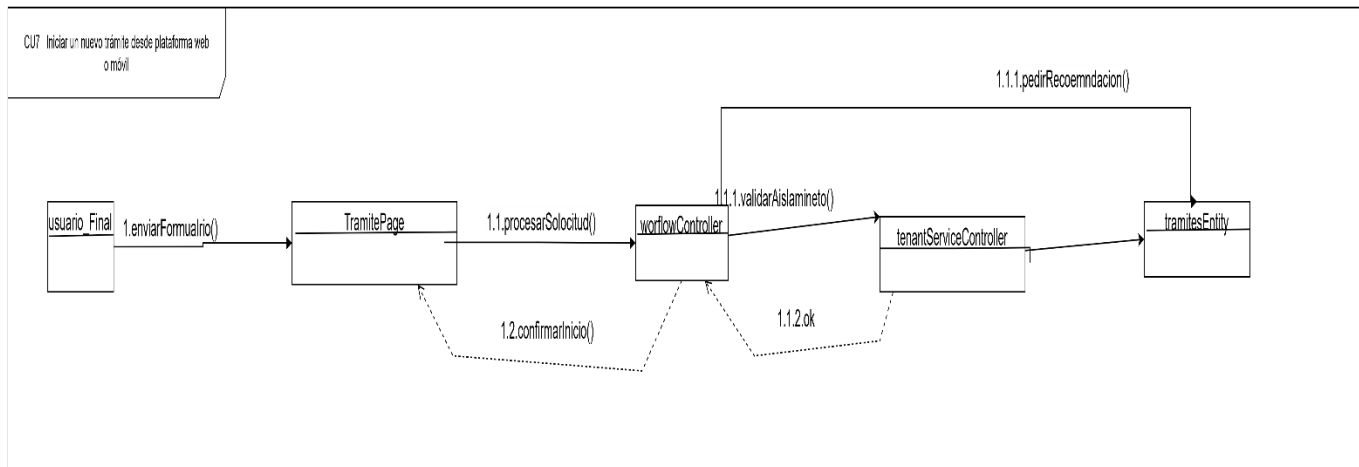
CU5 Visualizar dashboard de analítica y cuellos de botella



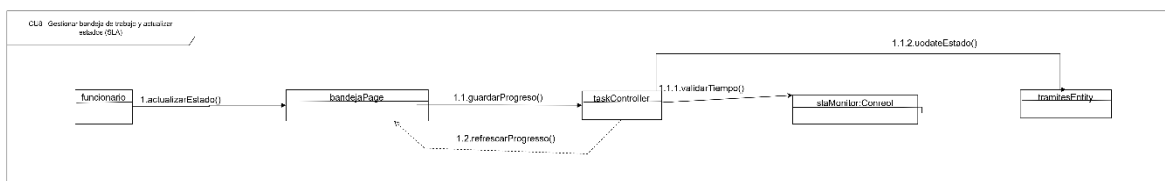
CU6 Aplicar recomendación de optimización sugerida por la IA



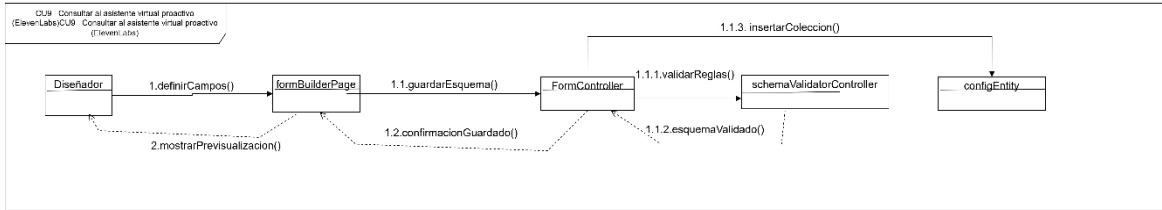
CU7 Iniciar un nuevo trámite desde plataforma web o móvil



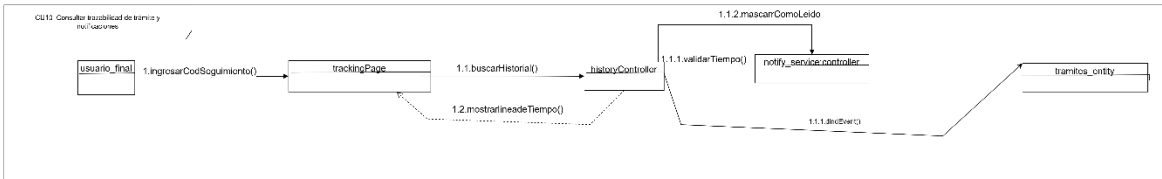
CU8 Gestionar bandeja de trabajo y actualizar estados (SLA)



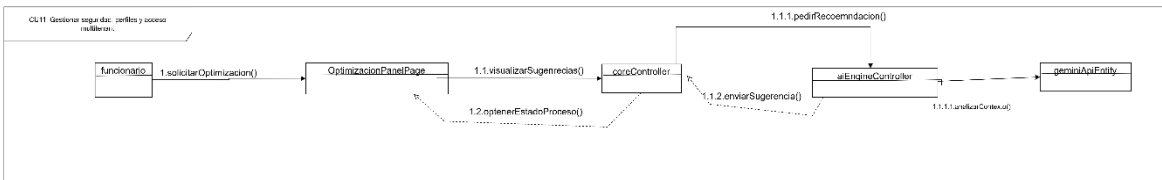
CU9 Consultar al asistente virtual proactivo (ElevenLabs)



CU10 Consultar trazabilidad de trámite y notificaciones

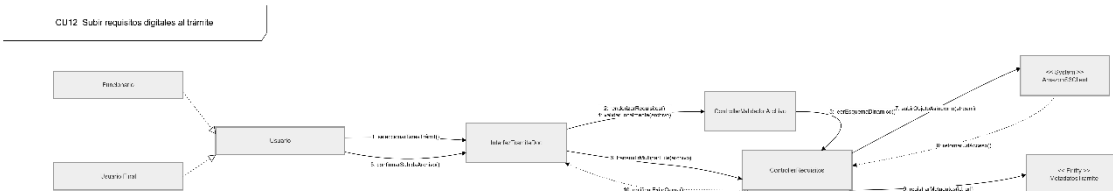


CU11 Gestionar seguridad, perfiles y acceso multitenant

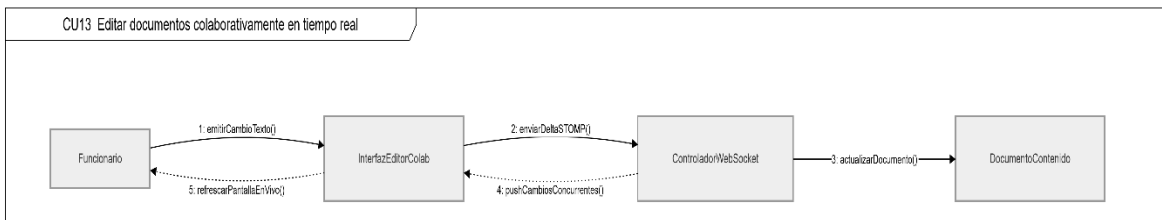


Ciclo # 2

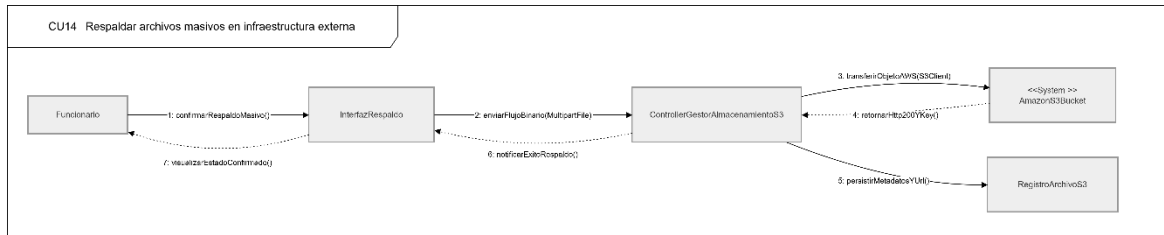
CU12 Subir requisitos digitales al trámite



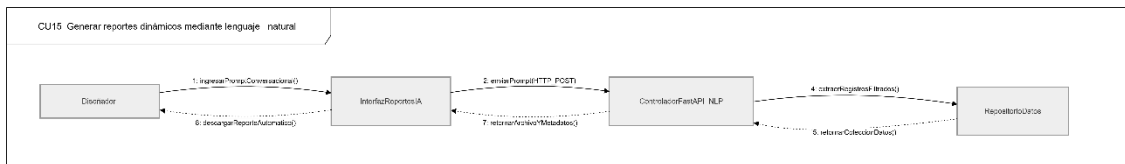
CU13 Editar documentos colaborativamente en tiempo real



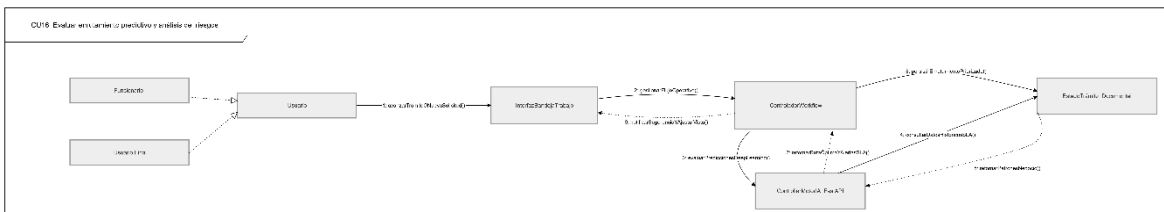
CU14 Respalidar archivos masivos en infraestructura externa



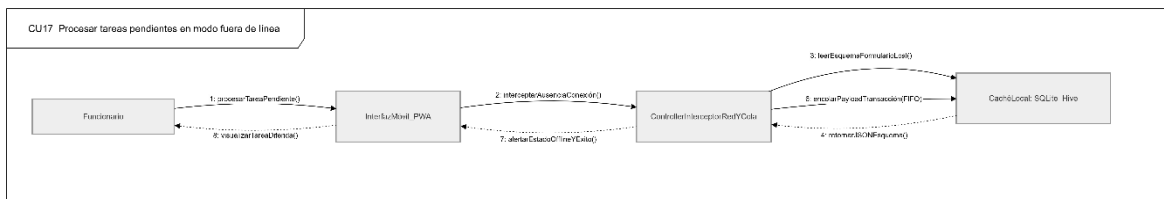
CU15 Generar reportes dinámicos mediante lenguaje natural



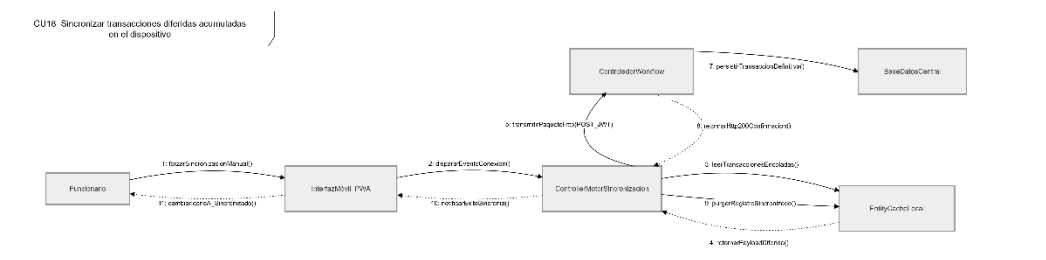
CU16 Evaluar enrutamiento predictivo y análisis de riesgos



CU17 Procesar tareas pendientes en modo fuera de línea



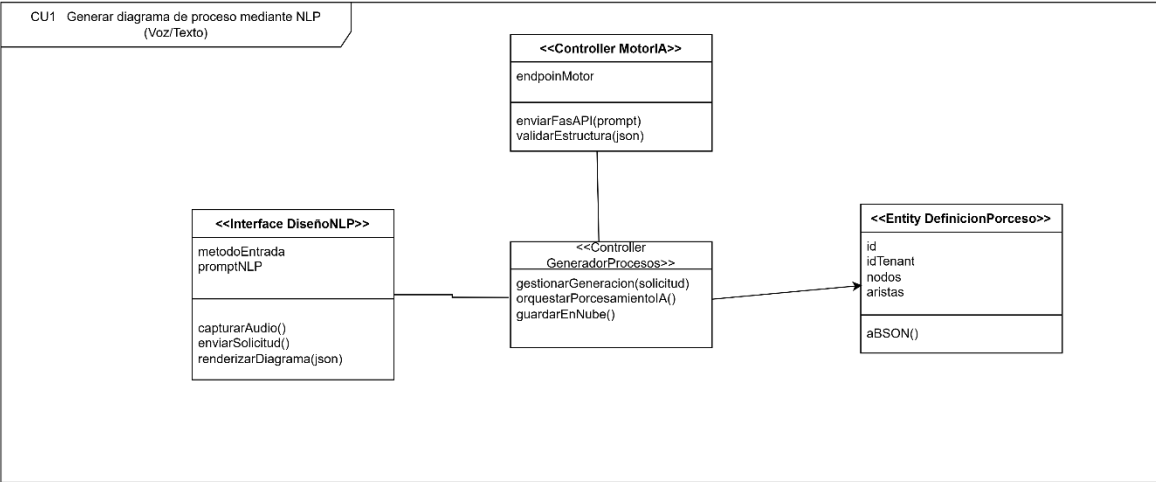
CU18 Sincronizar transacciones diferidas acumuladas en el dispositivo



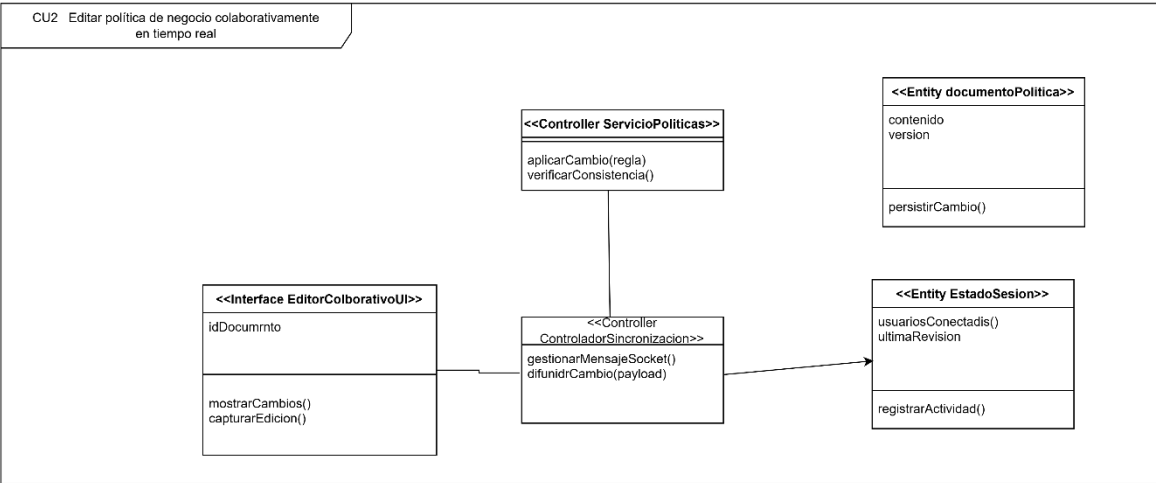
3.3. ANALISIS DE CLASES

Ciclo # 1

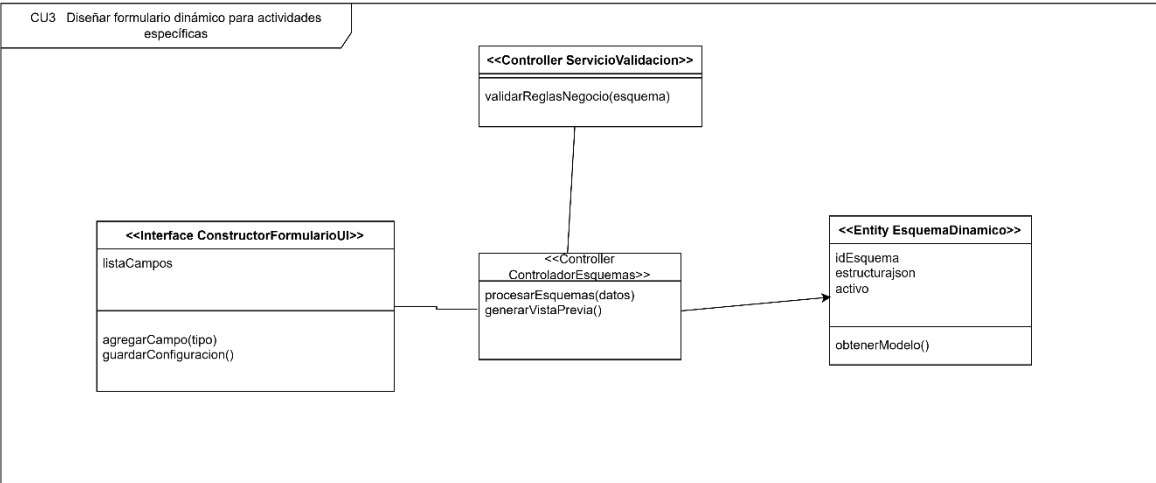
CU1 Generar diagrama de proceso mediante NLP (Voz/Texto)



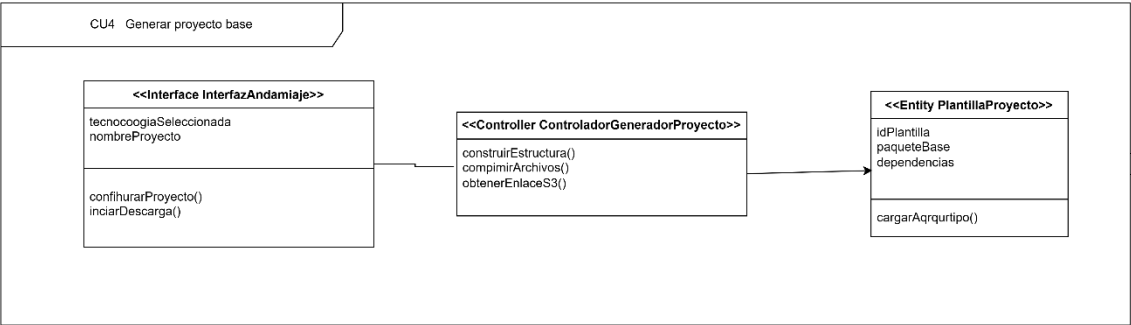
CU2 Editar política de negocio colaborativamente en tiempo real



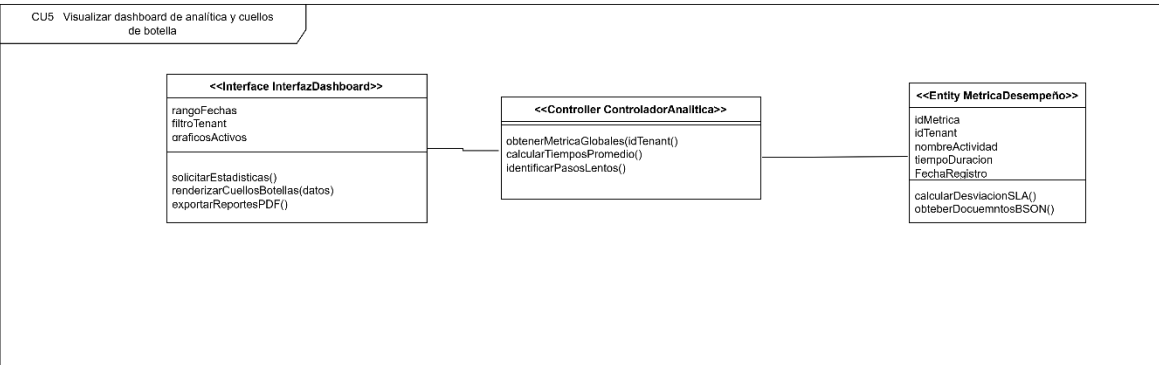
CU3 Diseñar formulario dinámico para actividades específicas



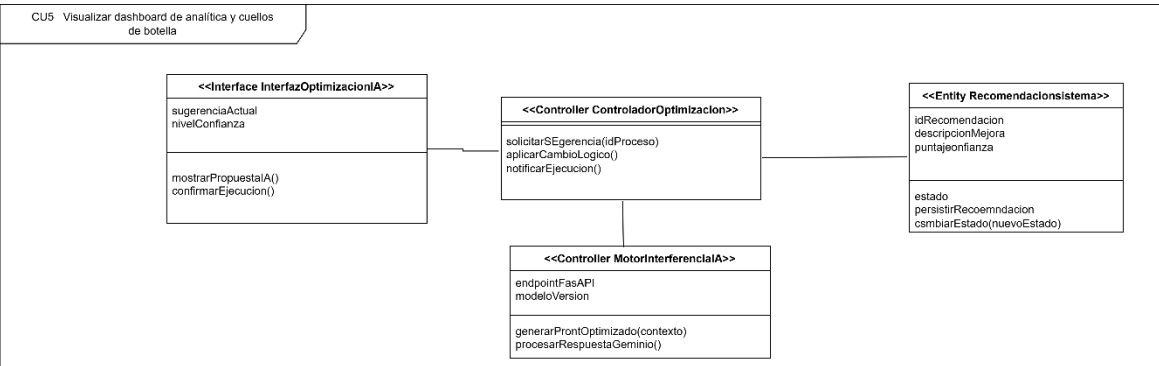
CU4 Generar proyecto base



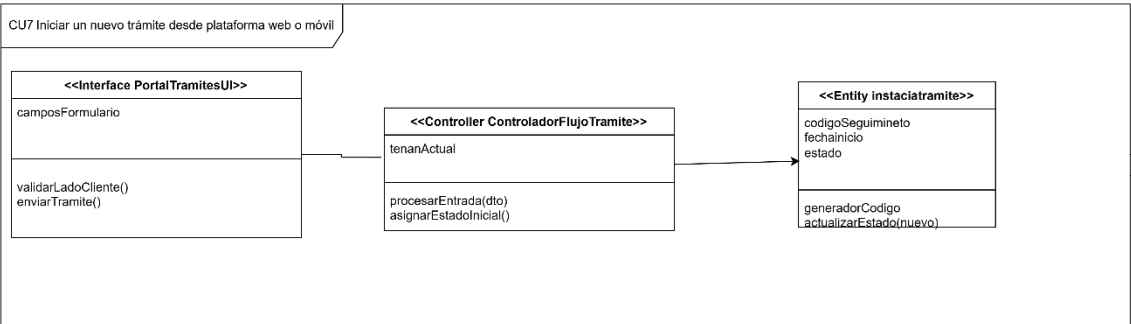
CU5 Visualizar dashboard de analítica y cuellos de botella



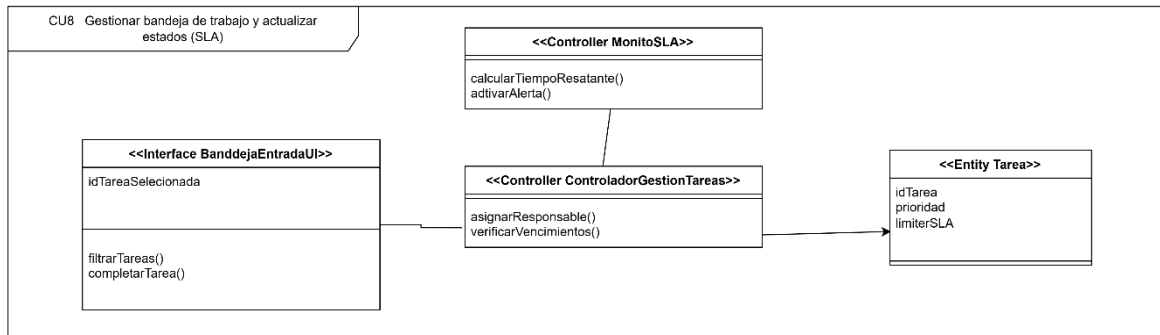
CU6 Aplicar recomendación de optimización sugerida por la IA



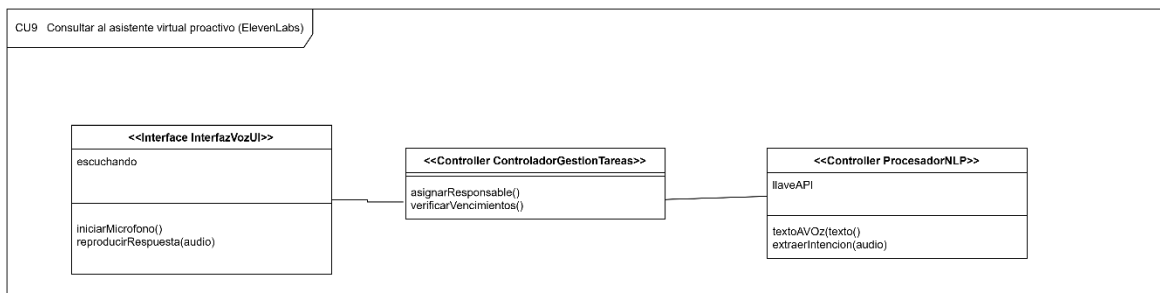
CU7 Iniciar un nuevo trámite desde plataforma web o móvil



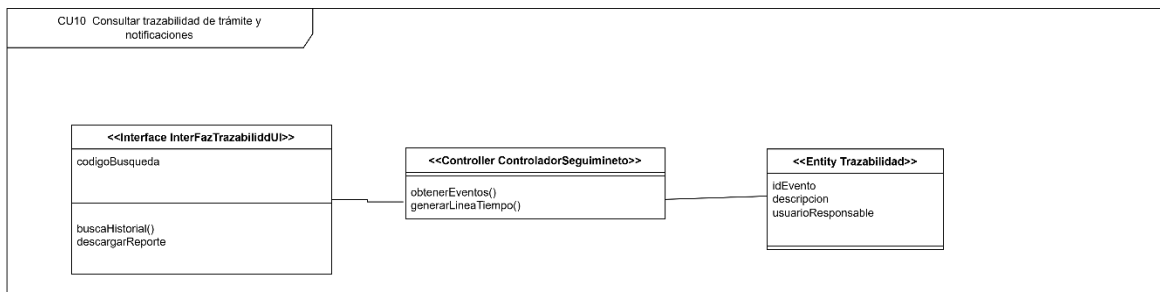
CU8 Gestionar bandeja de trabajo y actualizar estados (SLA)



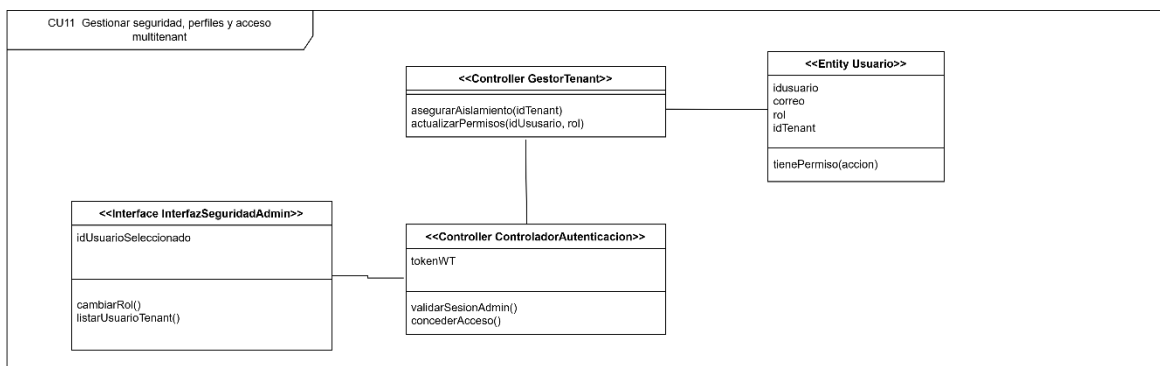
CU9 Consultar al asistente virtual proactivo (ElevenLabs)



CU10 Consultar trazabilidad de trámite y notificaciones

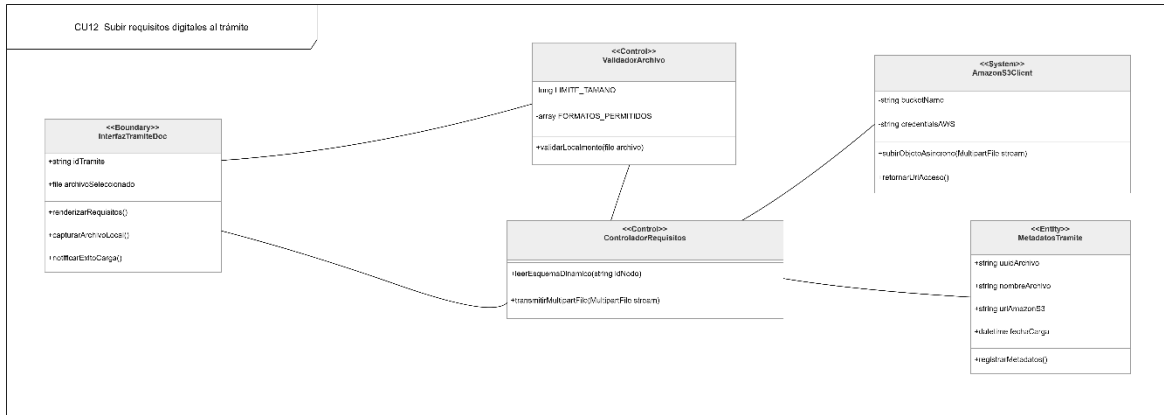


CU11 Gestionar seguridad, perfiles y acceso multitenant



Ciclo # 2

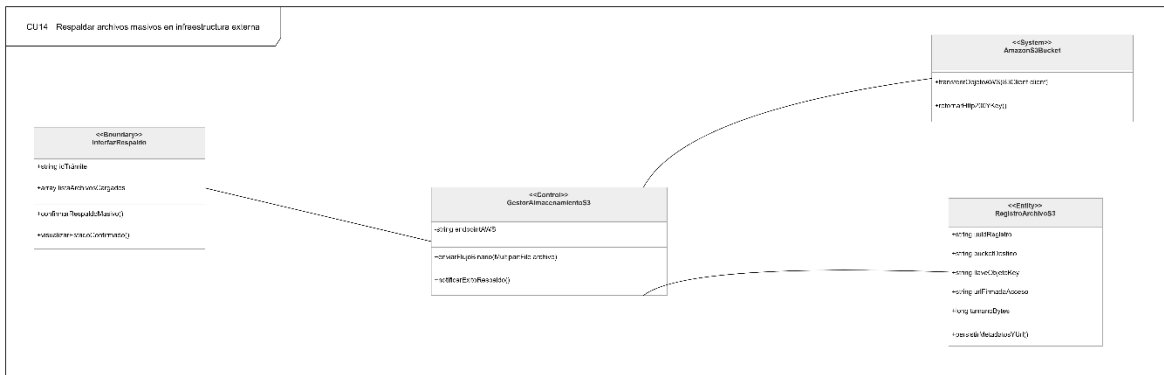
CU12 Subir requisitos digitales al trámite



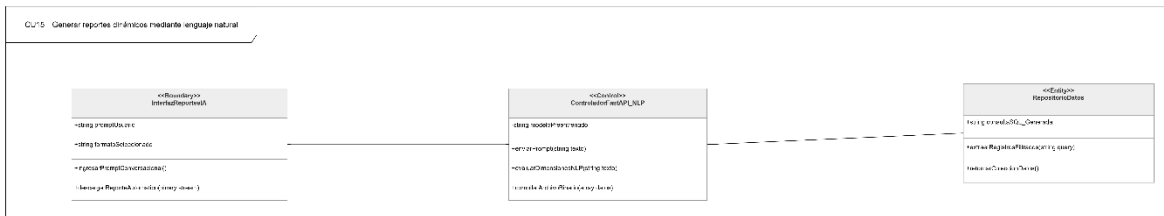
CU13 Editar documentos colaborativamente en tiempo real



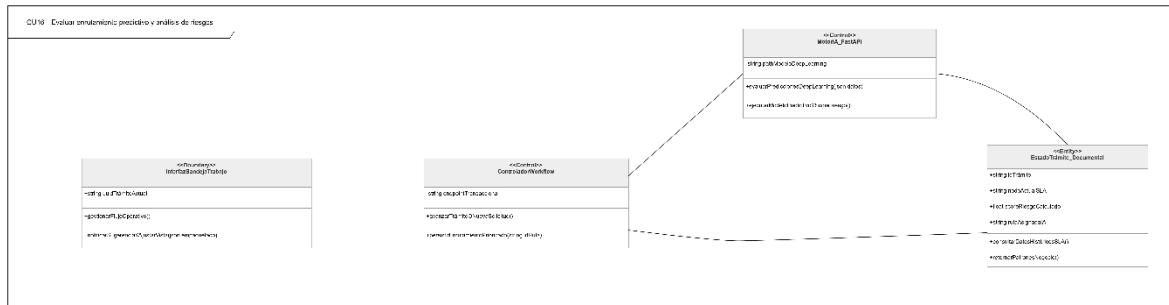
CU14 Respalidar archivos masivos en infraestructura externa



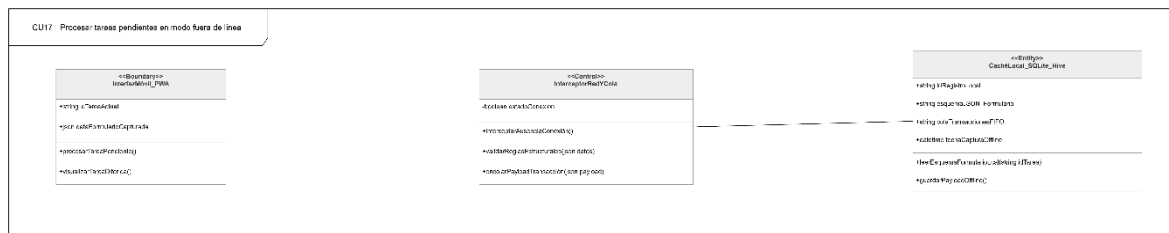
CU15 Generar reportes dinámicos mediante lenguaje natural



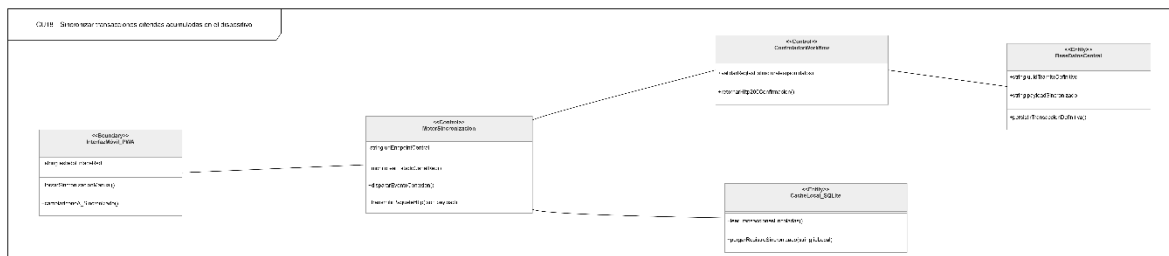
CU16 Evaluar enrutamiento predictivo y análisis de riesgos



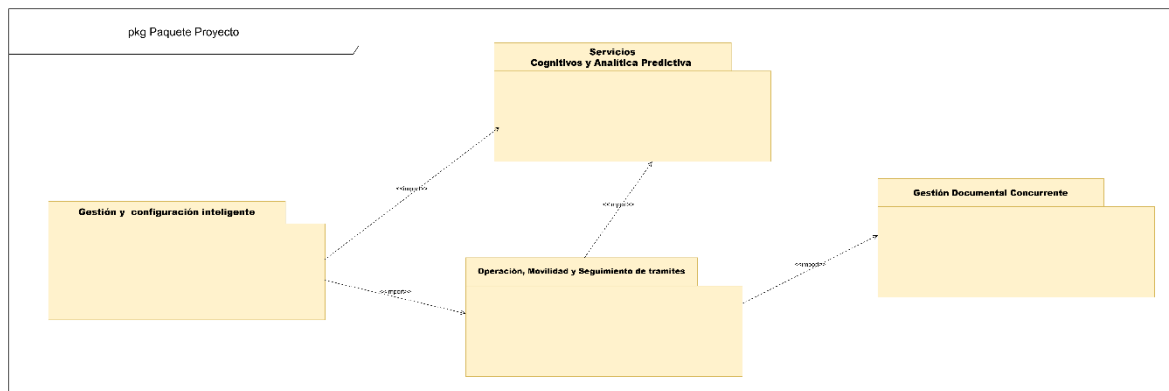
CU17 Procesar tareas pendientes en modo fuera de línea



CU18 Sincronizar transacciones diferidas acumuladas en el dispositivo



3.4. ANALISIS DE PAQUETES

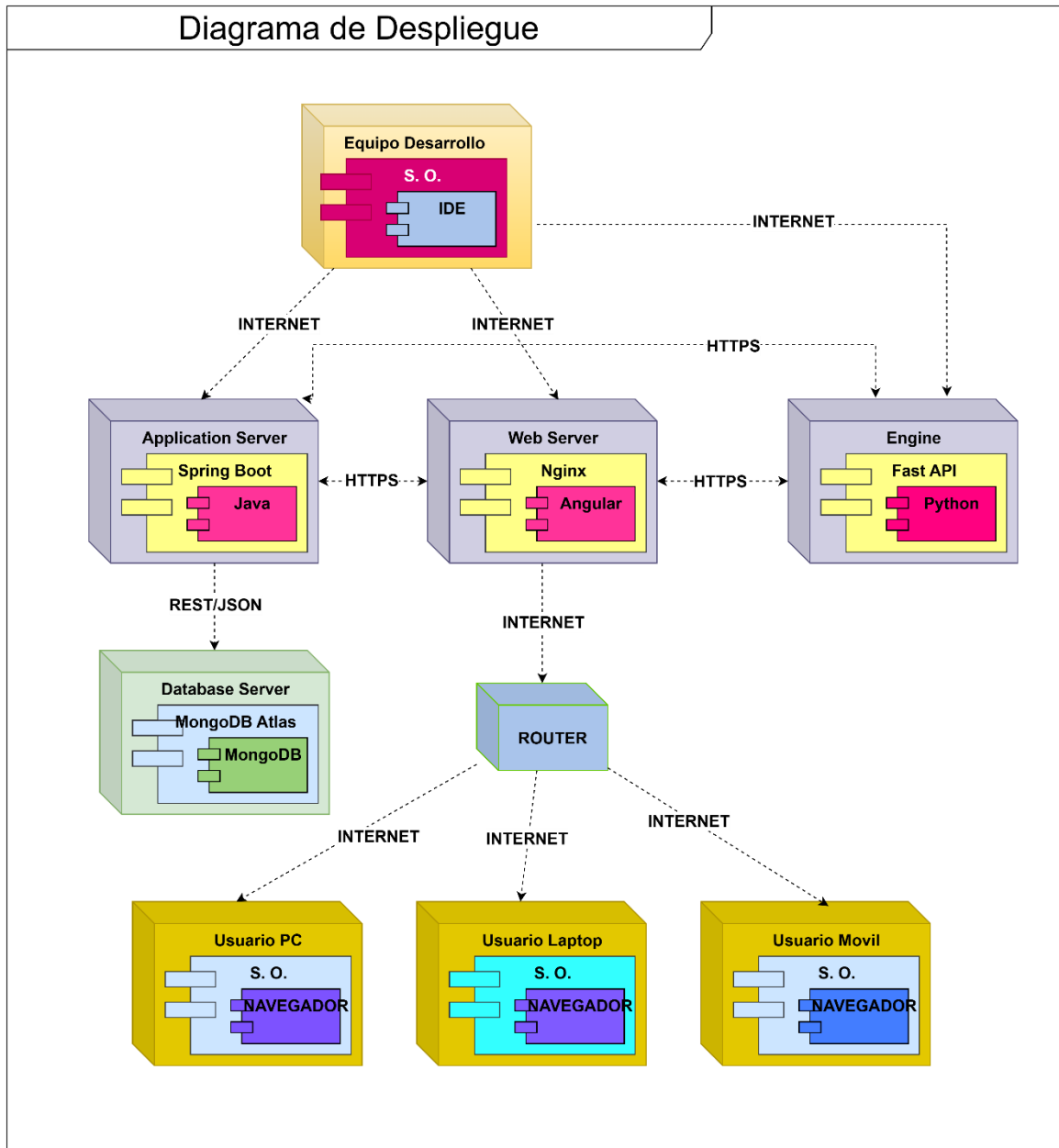


4. FLUJO DE TRABAJO: DISEÑO

4.1. DISEÑO DE ARQUITECTURA

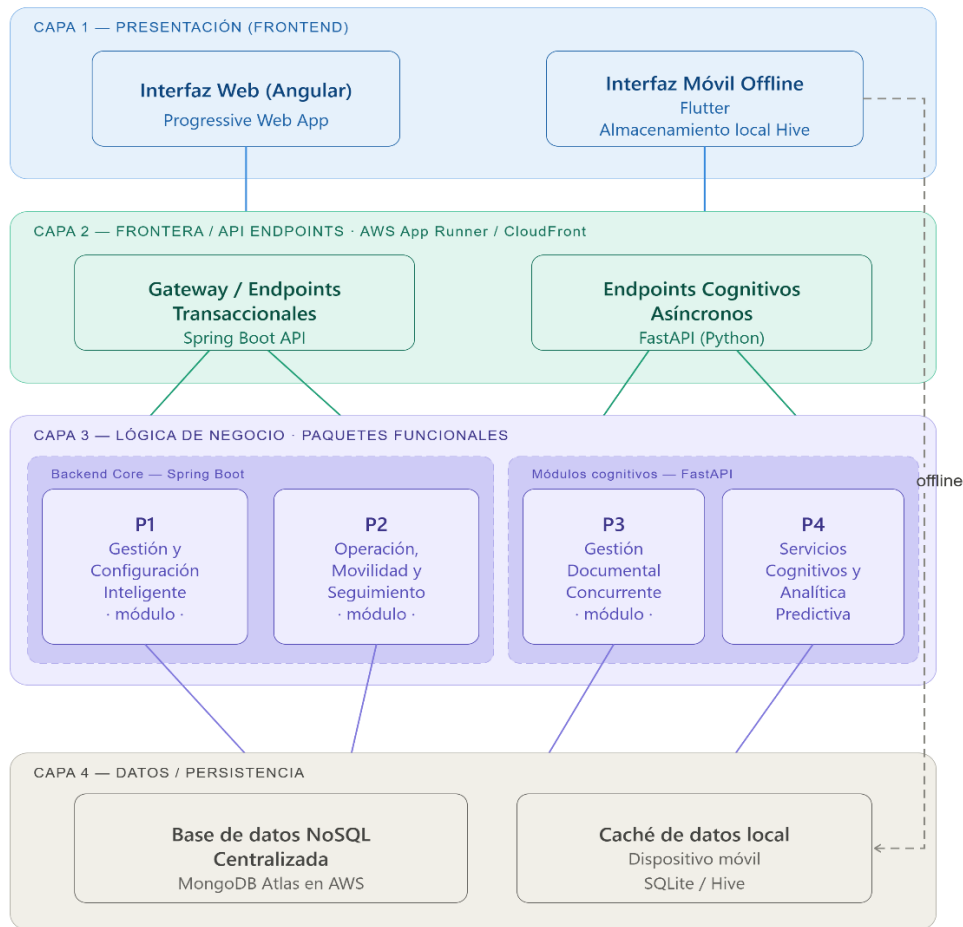
4.1.1. DISEÑO FISICO

DIAGRAMA DE DESPLIEGUE



4.1.2. DISEÑO LOGICO

DIAGRAMA ORGANIZADO EN CAPAS



4.1.3. MODELO C4

Diagrama de contexto del sistema

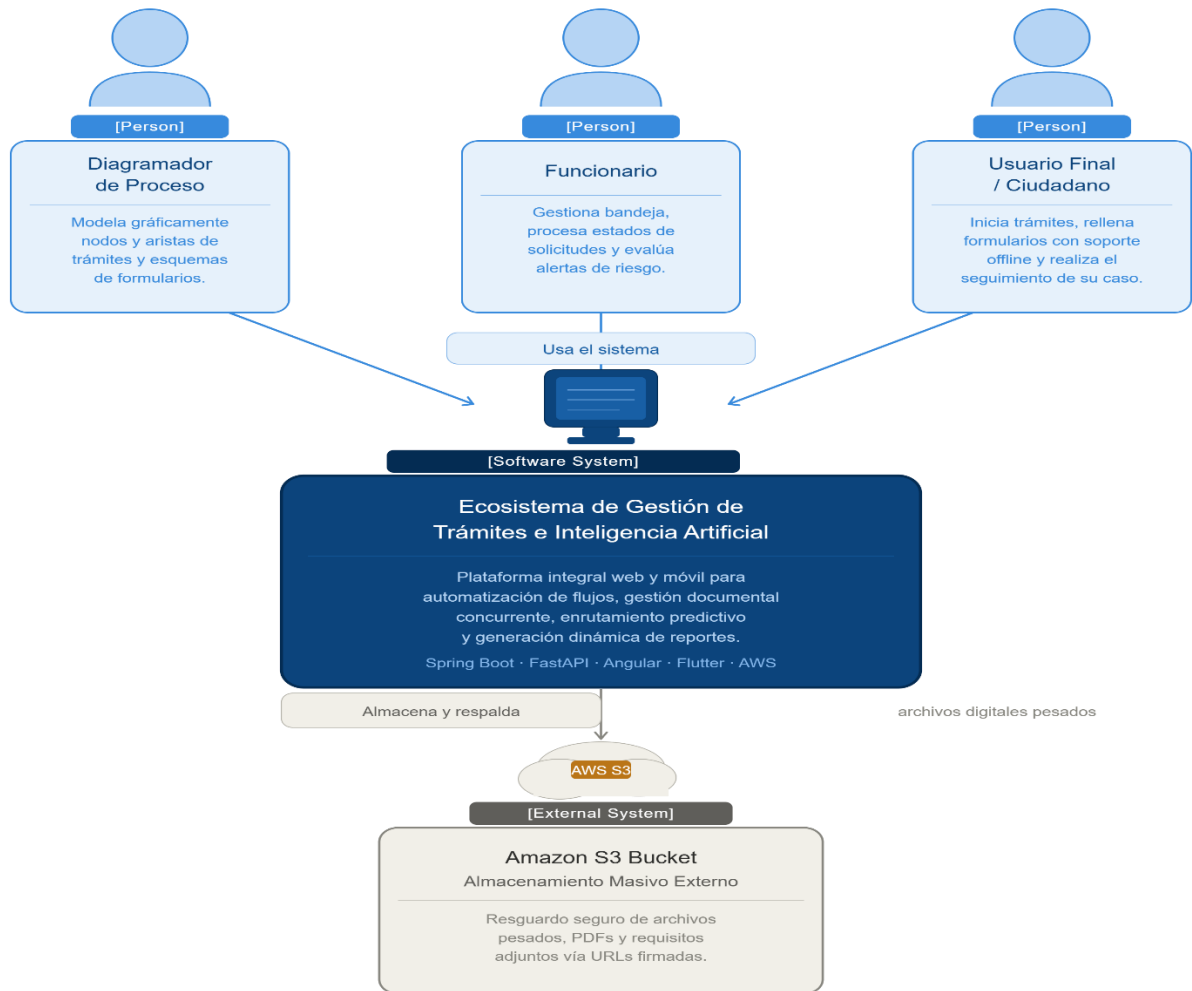


Diagrama de contenedores

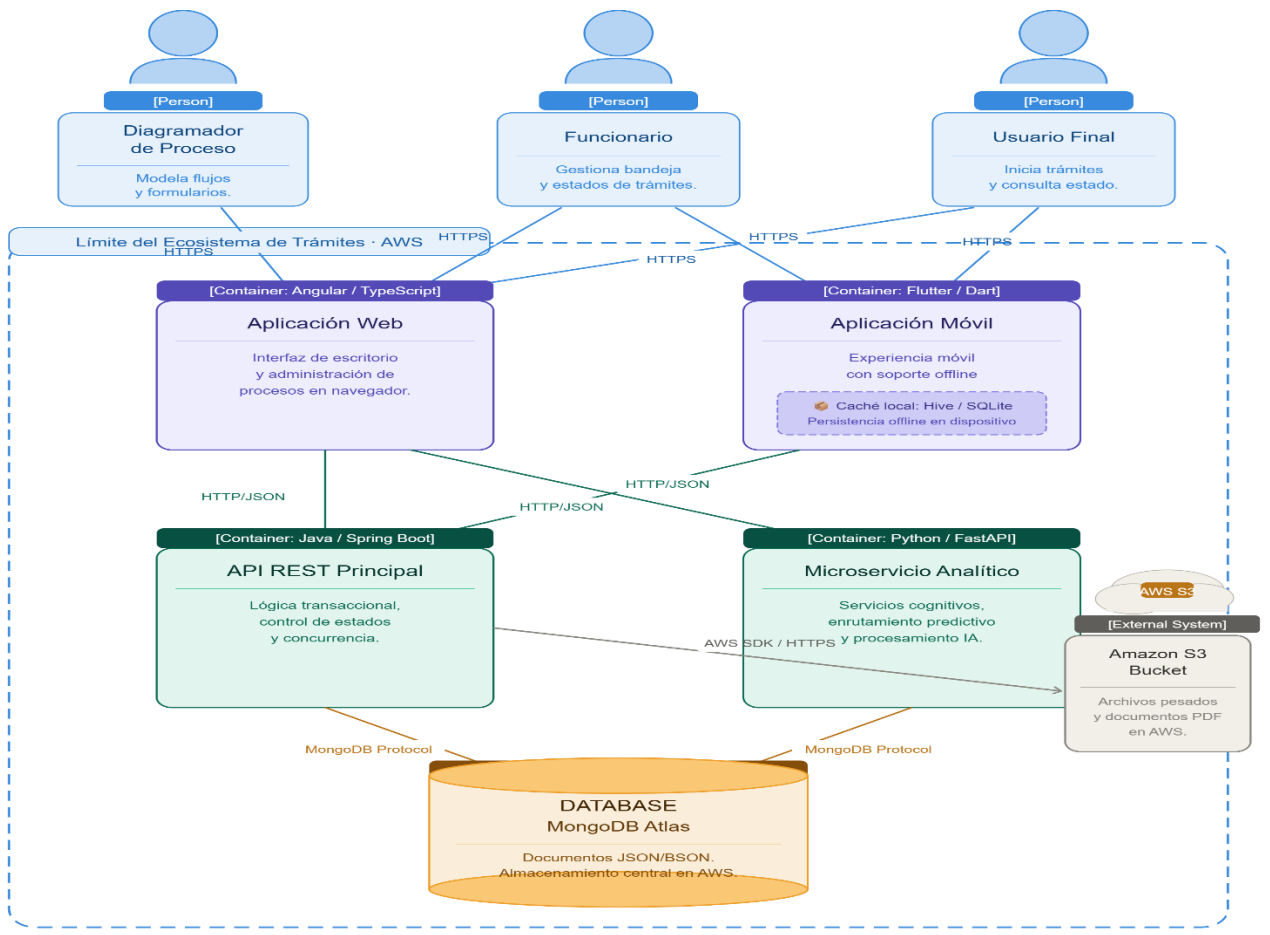


Diagrama de componentes

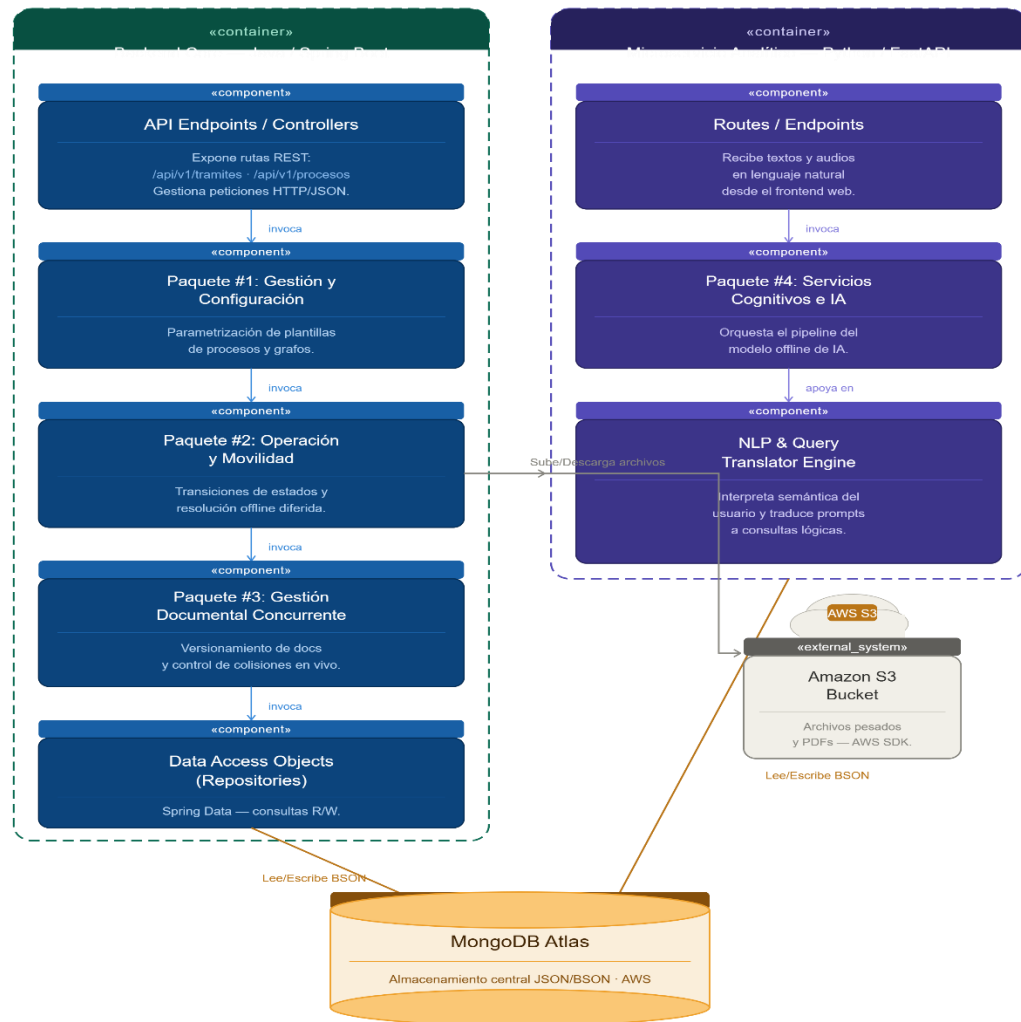
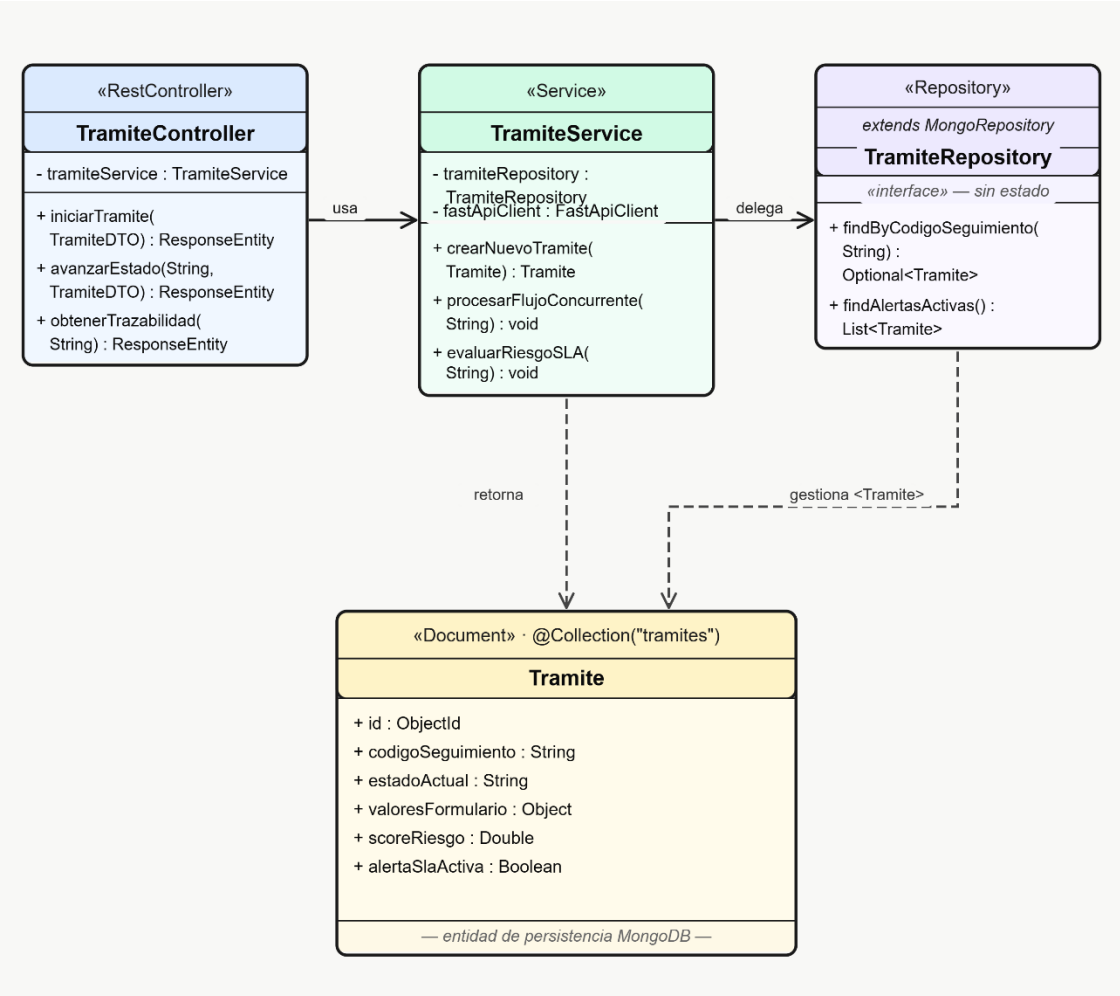
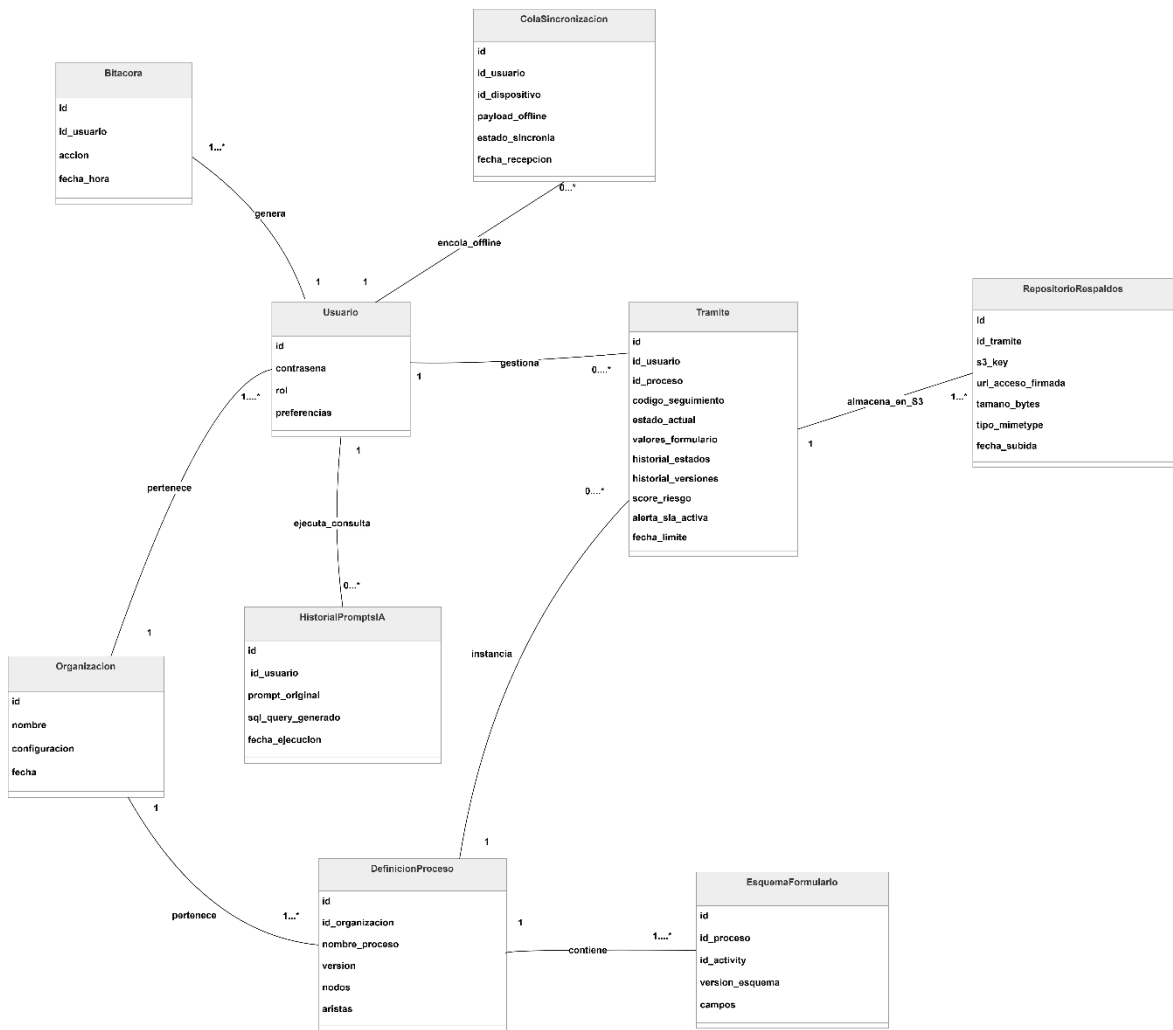


Diagrama de código



5. FLUJO DE TRABAJO: DISEÑO DE DATOS

5.1. DIAGRAMA DE CLASES



5.2. DISEÑO FÍSICO

TABLA DE VOLUMEN

Tabla: Usuario

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador único del usuario (BSON)	12 bytes	No	PK
organizacion_id	ObjectId	Referencia a la organización (Tenant)	12 bytes	No	FK
correo	String	Correo electrónico institucional / login	Variable (coor*)	No	-

contrasena	String	Hash de la contraseña (BCrypt/Argon2)	60 bytes	No	-
rol	String	Rol del sistema (Administrador, Operador)	Variable	No	-
preferencias	Object	Configuración personalizada del usuario	Variable	Sí	-

Tabla : Bitacora

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador único del log	12 bytes	No	PK
id_usuario	ObjectId	Usuario que generó la acción	12 bytes	No	FK
accion	String	Descripción detallada del evento técnico	Variable	No	-
fecha_hora	Date	Marca de tiempo exacta del suceso	8 bytes	No	-

Tabla : Organizacion

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador único de la empresa/institución	12 bytes	No	PK
nombre	String	Nombre legal de la organización	Variable	No	-
configuracion	Object	Parámetros globales del Tenant en la nube	Variable	No	-
fecha	Date	Fecha de alta en el ecosistema	8 bytes	No	-

Tabla : Tramite

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador único del caso/trámite	12 bytes	No	PK
id_usuario	ObjectId	Funcionario responsable actual	12 bytes	No	FK

id_proceso	ObjectId	Plantilla base del workflow	12 bytes	No	FK
codigo_seguimiento	String	Código único indexado para trazabilidad pública	Variable	No	-
estado_actual	String	Nodo o fase donde se encuentra el flujo	Variable	No	-
valores_formulario	Object	Datos dinámicos capturados en los campos JSON	Variable	No	-
historial_estados	Array	Lista de transiciones previas del flujo	Variable	No	-
historial_versiones	Array	Deltas de texto para concurrencia en vivo	Variable	Sí	-
score_riesgo	Double	Índice probabilístico calculado por la IA	8 bytes	No	-
alerta_sla_activa	Boolean	Indicador de retraso o cuello de botella predictivo	1 byte	No	-
fecha_limite	Date	Fecha de expiración del SLA del trámite	8 bytes	No	-

Tabla : Definicion_Proceso

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador único del diseño del proceso	12 bytes	No	PK
id_organizacion	ObjectId	Organización dueña del proceso	12 bytes	No	FK
nombre_proceso	String	Título del procedimiento administrativo	Variable	No	-
version	Integer	Control de versiones del modelador de procesos	4 bytes	No	-
nodos	Array	Lista de estaciones o actividades del grafo	Variable	No	-

aristas	Array	Transiciones y condiciones lógicas del flujo	Variable	No	-
----------------	-------	--	----------	----	---

Tabla: Esquema_Formulario

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador de la estructura del formulario	12 bytes	No	PK
id_proceso	ObjectId	Referencia al proceso padre	12 bytes	No	FK
id_activity	String	ID del nodo específico dentro del grafo	Variable	No	-
version_esquema	Integer	Versión para validación móvil fuera de línea	4 bytes	No	-
campos	Array	Definición JSON de los inputs (inputs, tipos, regex)	Variable	No	-

Tabla : Repositorio_Respaldos

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador del archivo registrado	12 bytes	No	PK
id_tramite	ObjectId	Trámite al que pertenece el adjunto	12 bytes	No	FK
s3_key	String	Dirección o Key del archivo masivo en AWS S3	Variable	No	-
url_acceso_firmada	String	URL temporal de visualización segura	Variable	No	-
tamano_bytes	Long	Peso físico del archivo almacenado	8 bytes	No	-
tipo_mimetype	String	Formato del requisito (pdf, jpeg, docx)	Variable	No	-
fecha_subida	Date	Fecha de ejecución del respaldo masivo	8 bytes	No	-

Tabla: Cola_Sincronizacion

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador transaccional de la sincronía	12 bytes	No	PK
id_usuario	ObjectId	Operador de campo que recolectó la información	12 bytes	No	FK
id_dispositivo	String	UUID de hardware del móvil (Flutter)	Variable	No	-
payload_offline	Object	JSON crudo con los datos capturados sin internet	Variable	No	-
estado_sincronia	String	Control del estado (PENDIENTE, PROCESADO, ERROR)	Variable	No	-
fecha_recepcion	Date	Momento en que el paquete tocó la API REST	8 bytes	No	-

Tabla : Historial_Prompts_IA

Atributo	Tipo de dato	Descripción	Tamaño (aprox)	Nulo	Llave
_id	ObjectId	Identificador único de la consulta analítica	12 bytes	No	PK
id_usuario	ObjectId	Administrador que ejecutó la consulta por voz/texto	12 bytes	No	FK
prompt_original	String	Cadena en lenguaje natural enviada a FastAPI	Variable	No	-
sql_query_generado	String	Código estructurado que derivó de la semántica	Variable	No	-
fecha_ejecucion	Date	Registro temporal de la petición analítica	8 bytes	No	-

SCRIP

```
// Script de inicialización para MongoDB (Mongosh)
// Uso: db = db.getSiblingDB('sistema_tramites_ia');

// 1. Colección: Organizacion
db.createCollection("Organizacion", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "configuracion", "fecha"],
      properties: {
        nombre: { bsonType: "string", description: "Nombre
legal obligatorio" },
        configuracion: { bsonType: "object", description:
"Configuración global del tenant" },
        fecha: { bsonType: "date" }
      }
    }
  }
});

// 2. Colección: Usuario
db.createCollection("Usuario", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["organizacion_id", "correo", "contrasena",
"rol"],
      properties: {
```

```

        organizacion_id: { bsonType: "objectId", description:
"FK obligatoria a Organizacion" },
        correo: { bsonType: "string", pattern: "^.+@.+$",
description: "Debe ser un correo válido" },
        contrasena: { bsonType: "string", description: "Hash
BCrypt" },
        rol: { bsonType: "string", enum: ["Administrador",
"Operador", "Funcionario" ] },
        preferencias: { bsonType: "object" }
    }
}
});

```

// 3. Colección: Bitacora

```

db.createCollection("Bitacora", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["id_usuario", "accion", "fecha_hora"],
            properties: {
                id_usuario: { bsonType: "objectId" },
                accion: { bsonType: "string" },
                fecha_hora: { bsonType: "date" }
            }
        }
    }
});

```

// 4. Colección: Definicion_Proceso

```

db.createCollection("Definicion_Proceso", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_organizacion", "nombre_proceso",
"version", "nodos", "aristas"],
      properties: {
        id_organizacion: { bsonType: "objectId" },
        nombre_proceso: { bsonType: "string" },
        version: { bsonType: "int" },
        nodos: { bsonType: "array", description: "Estructura
del grafo de actividades" },
        aristas: { bsonType: "array", description: "Conexiones
lógicas del grafo" }
      }
    }
  }
});

```

// 5. Colección: Esquema_Formulario

```

db.createCollection("Esquema_Formulario", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_proceso", "id_activity",
"version_esquema", "campos"],
      properties: {
        id_proceso: { bsonType: "objectId" },
        id_activity: { bsonType: "string" },
        version_esquema: { bsonType: "int" },

```

```

        campos: { bsonType: "array", description: "Campos
dinámicos del formulario" }
    }
}
}
});

// 6. Colección: Tramite
db.createCollection("Tramite", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["id_usuario", "id_proceso",
"codigo_seguimiento", "estado_actual", "valores_formulario",
"score_riesgo", "alerta_sla_activa"],
            properties: {
                id_usuario: { bsonType: "objectId" },
                id_proceso: { bsonType: "objectId" },
                codigo_seguimiento: { bsonType: "string" },
                estado_actual: { bsonType: "string" },
                valores_formulario: { bsonType: "object", description:
"Respuestas JSON del formulario" },
                historial_estados: { bsonType: "array" },
                historial_versiones: { bsonType: "array" }, // CU13
Concurrencia
                score_riesgo: { bsonType: "double" }, // CU16 IA
Predictiva
                alerta_sla_activa: { bsonType: "bool" },
                fecha_limite: { bsonType: "date" }
            }
        }
    }
});

```

```

    }
  });

// 7. Colección: Repositorio_Respaldos (CU14 - AWS S3)
db.createCollection("Repositorio_Respaldos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_tramite", "s3_key", "url_acceso_firmada",
        "tamano_bytes", "tipo_mimetype", "fecha_subida"],
      properties: {
        id_tramite: { bsonType: "objectId" },
        s3_key: { bsonType: "string" },
        url_acceso_firmada: { bsonType: "string" },
        tamano_bytes: { bsonType: "long" },
        tipo_mimetype: { bsonType: "string" },
        fecha_subida: { bsonType: "date" }
      }
    }
  }
});

```

```

// 8. Colección: Cola_Sincronizacion (CU18 - Offline Móvil)
db.createCollection("Cola_Sincronizacion", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_usuario", "id_dispositivo",
        "payload_offline", "estado_sincronia", "fecha_recepcion"],

```

```

        properties: {
            id_usuario: { bsonType: "objectId" },
            id_dispositivo: { bsonType: "string" },
            payload_offline: { bsonType: "object" },
            estado_sincronia: { bsonType: "string", enum:
["PENDIENTE", "PROCESADO", "ERROR"] },
            fecha_recepcion: { bsonType: "date" }
        }
    }
});

```

```

// 9. Colección: Historial_Prompts_IA (CU15 - NLP FastAPI)
db.createCollection("Historial_Prompts_IA", {
    validator: {
        $jsonSchema: {
            bsonType: "object",
            required: ["id_usuario", "prompt_original",
"sql_query_generado", "fecha_ejecucion"],
            properties: {
                id_usuario: { bsonType: "objectId" },
                prompt_original: { bsonType: "string" },
                sql_query_generado: { bsonType: "string" },
                fecha_ejecucion: { bsonType: "date" }
            }
        }
    }
});

```



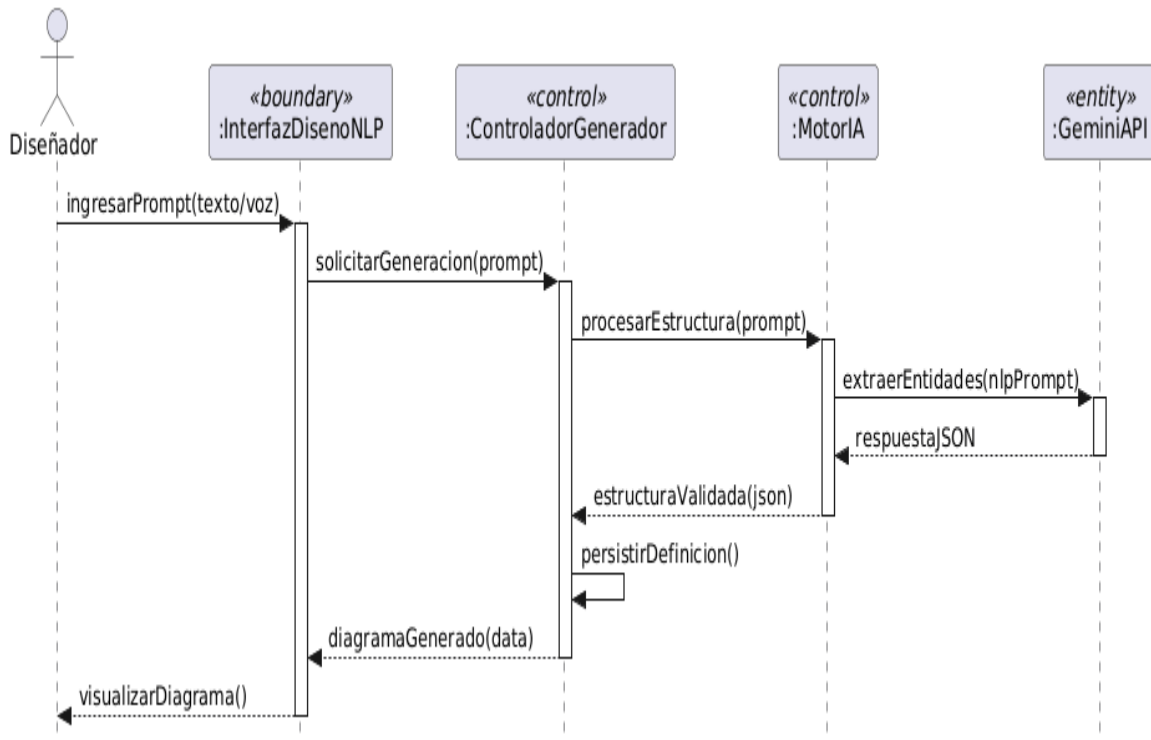
```
// --- CREACIÓN DE ÍNDICES OPTIMIZADOS PARA AWS ---
db.Usuario.createIndex({ "correo": 1 }, { unique: true });
db.Tramite.createIndex({ "codigo_seguimiento": 1 }, { unique: true });
db.Cola_Sincronizacion.createIndex({ "estado_sincronia": 1 });
```

5.3. DISEÑAR CASOS DE USO

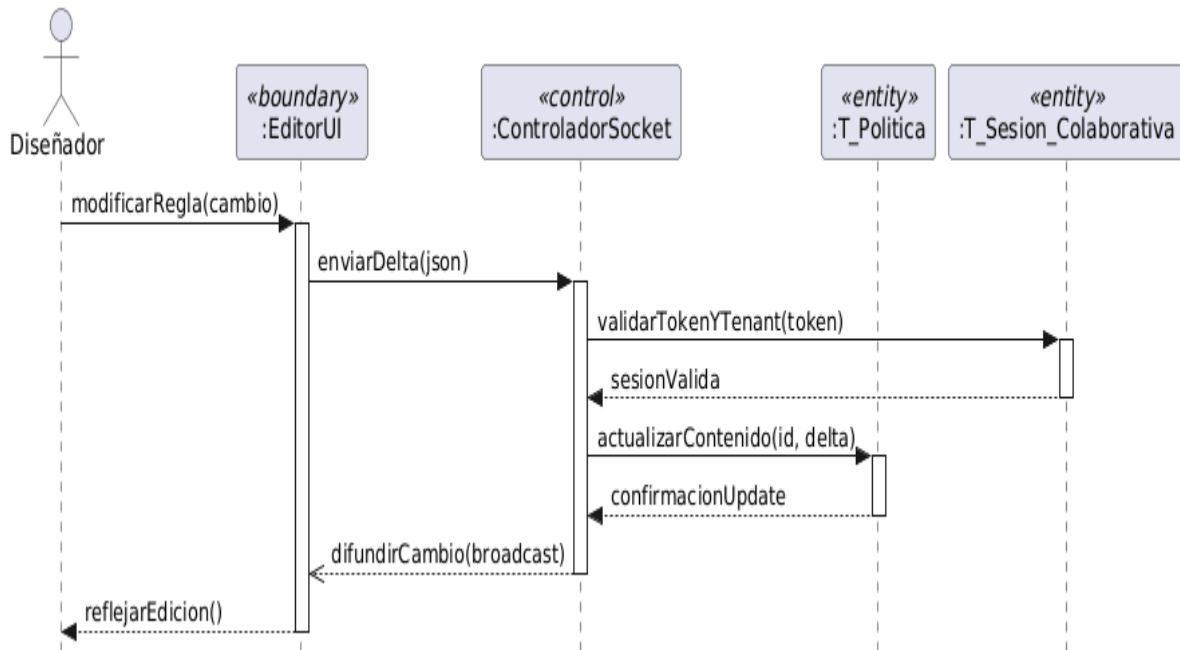
DIAGRAMA DE SECUENCIA

Ciclo # 1

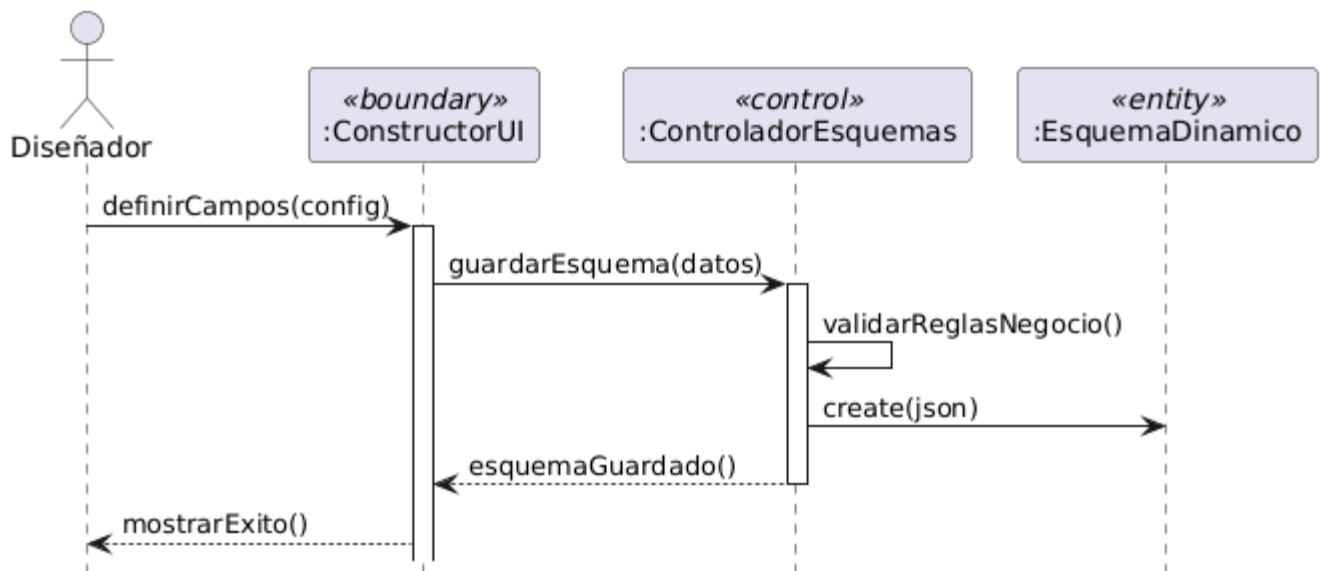
CU1 Generar diagrama de proceso mediante NLP (Voz/Texto)



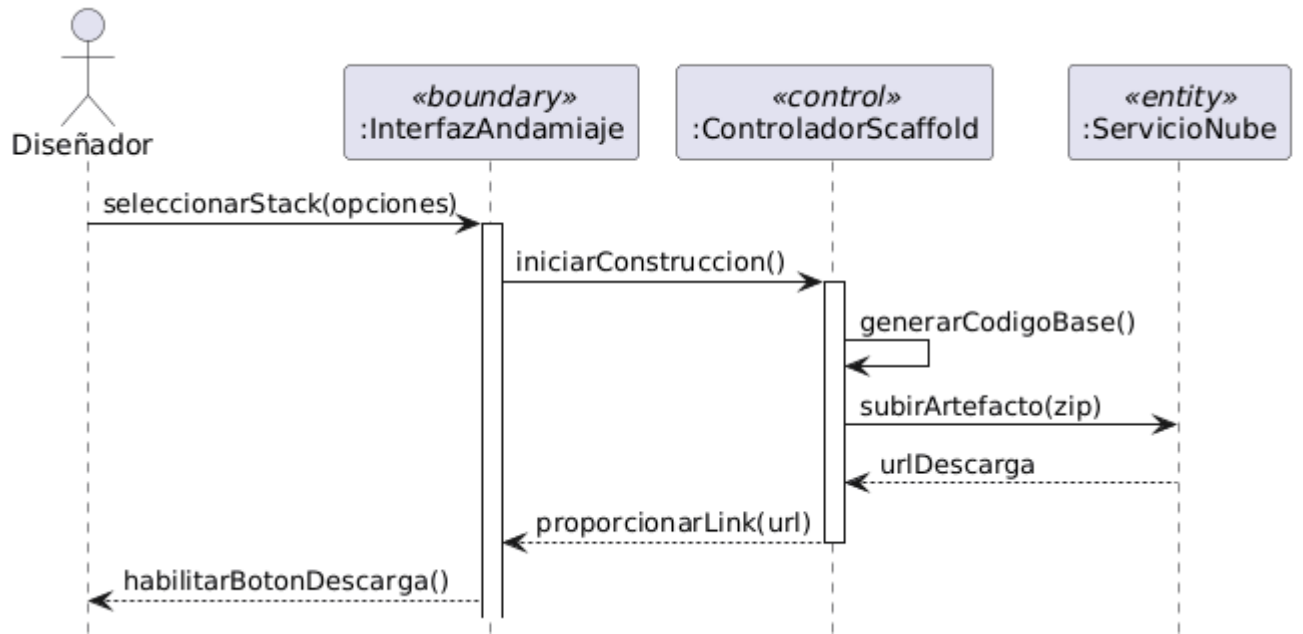
CU2 Editar política de negocio colaborativamente en tiempo real



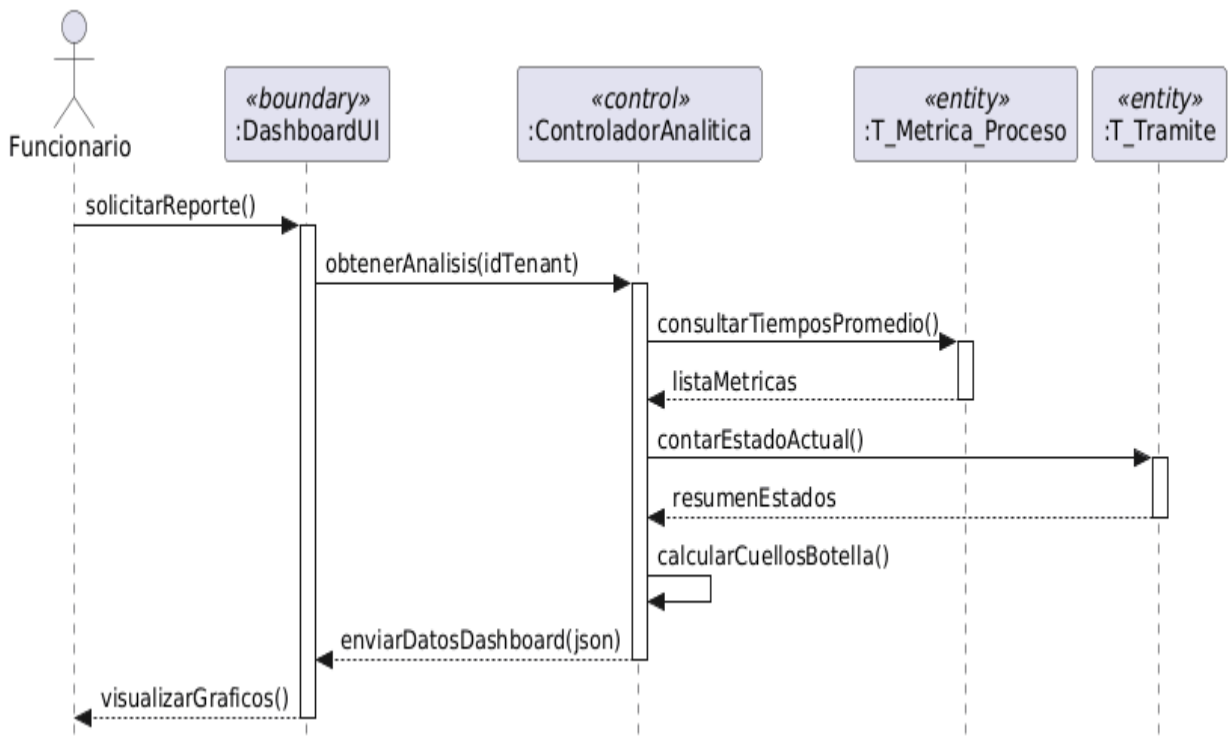
CU3 Diseñar formulario dinámico para actividades específicas



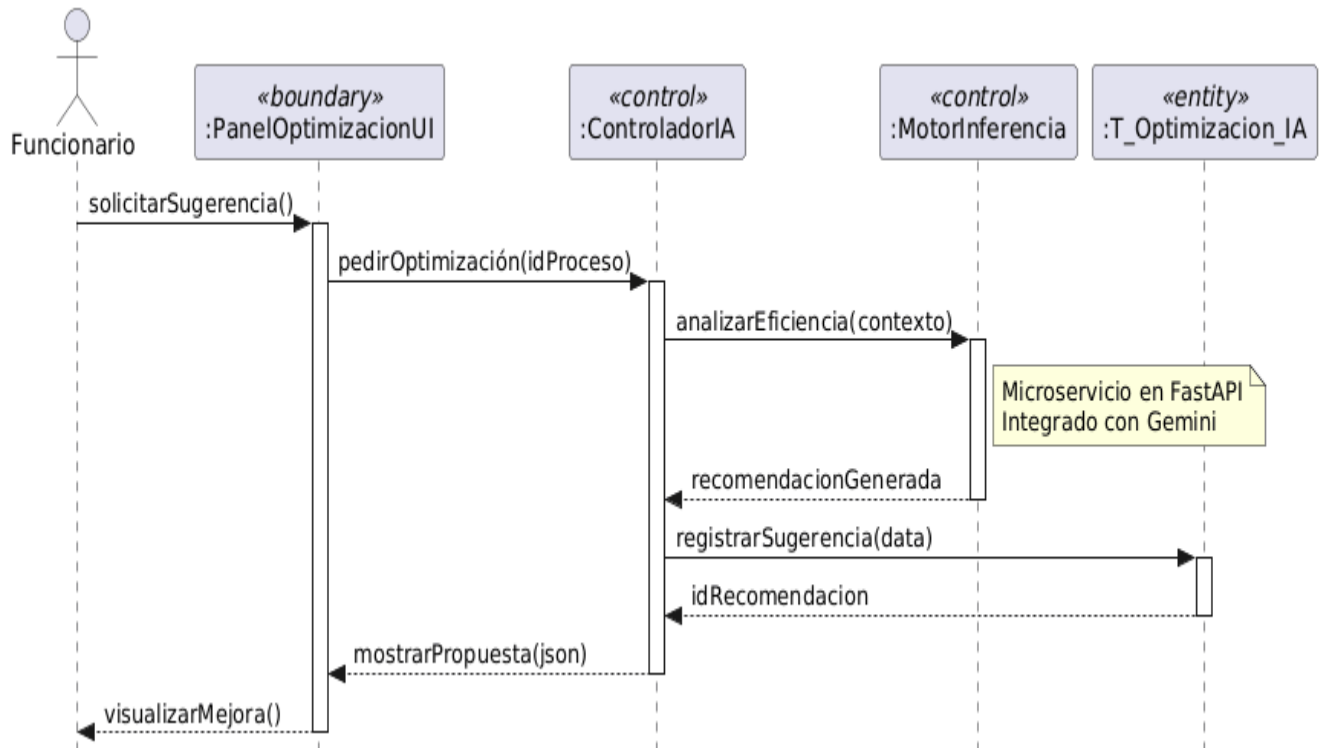
CU4 Generar proyecto base



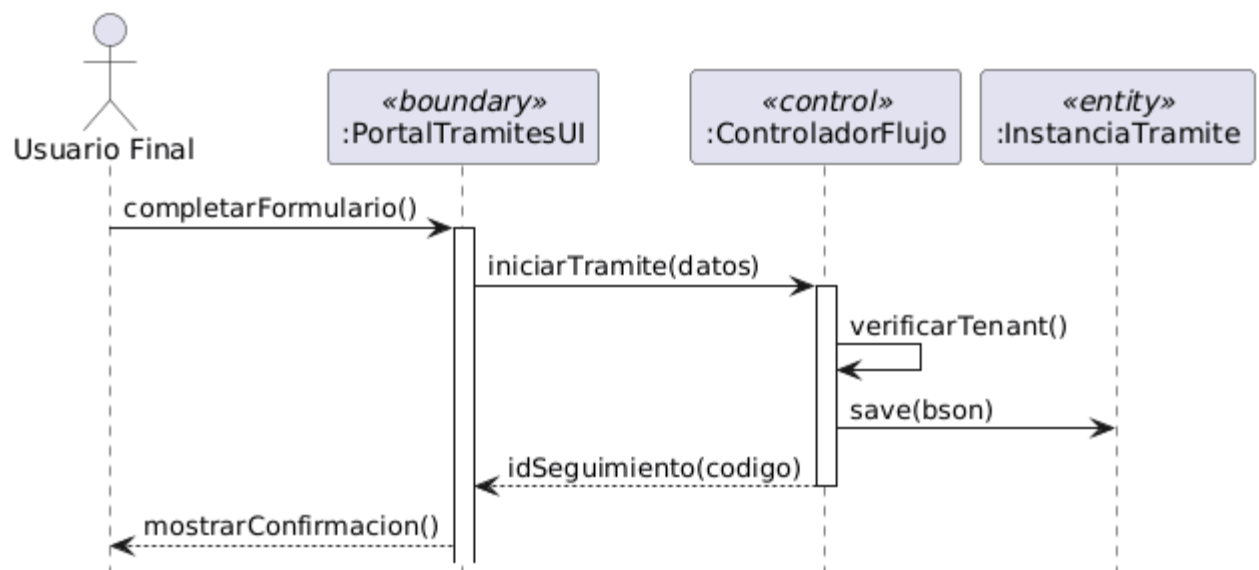
CU5 Visualizar dashboard de analítica y cuellos de botella



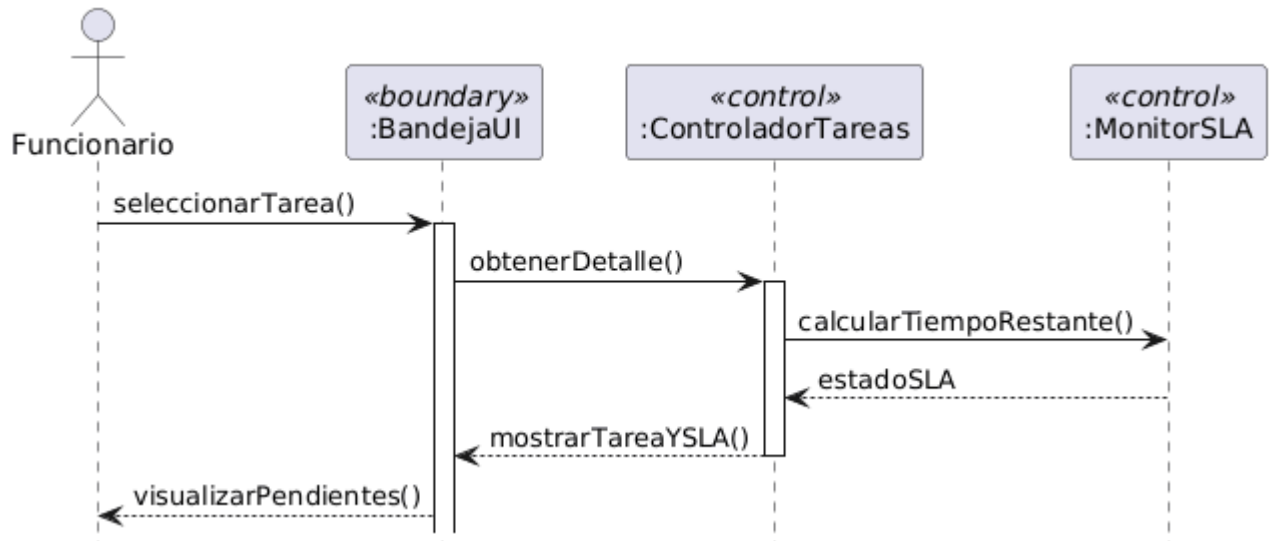
CU6 Aplicar recomendación de optimización sugerida por la IA



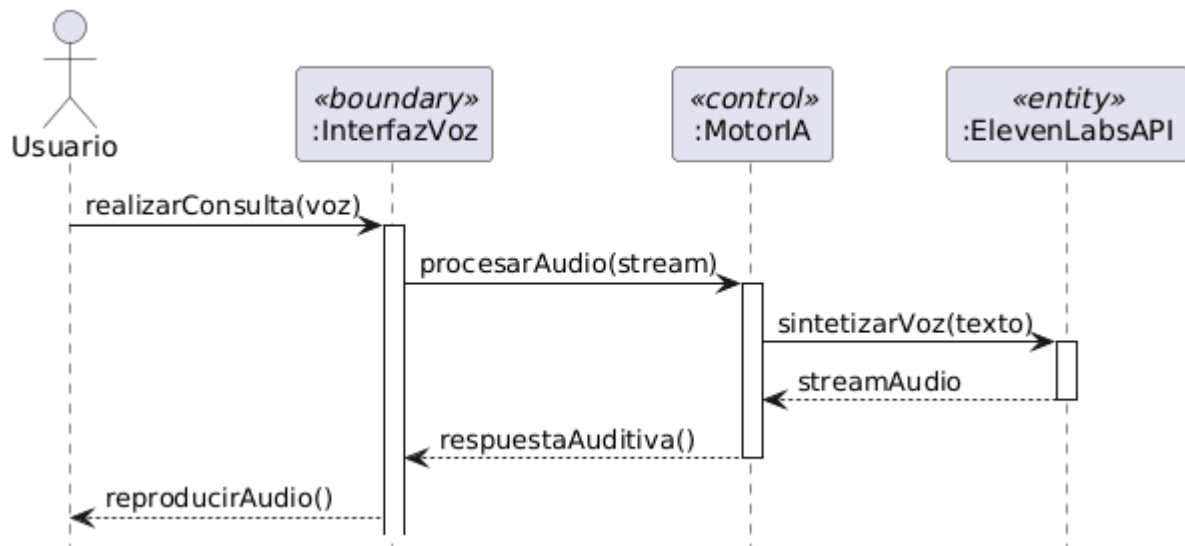
CU7 Iniciar un nuevo trámite desde plataforma web o móvil



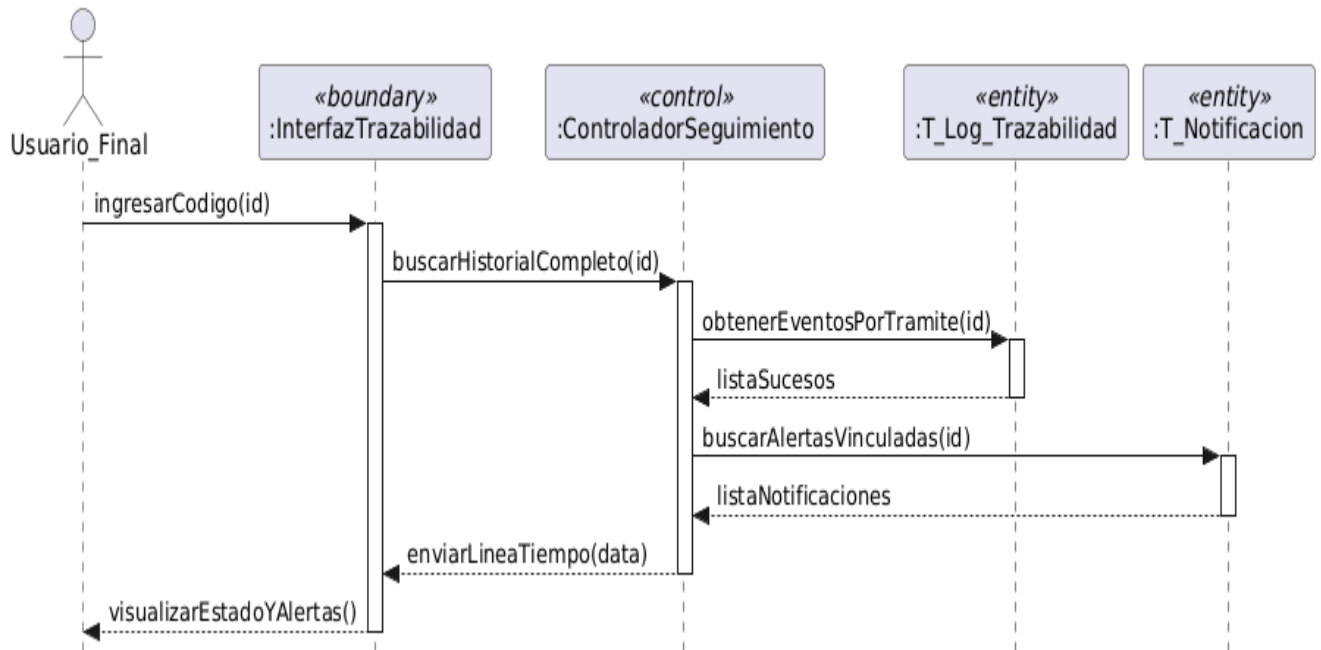
CU8 Gestionar bandeja de trabajo y actualizar estados (SLA)



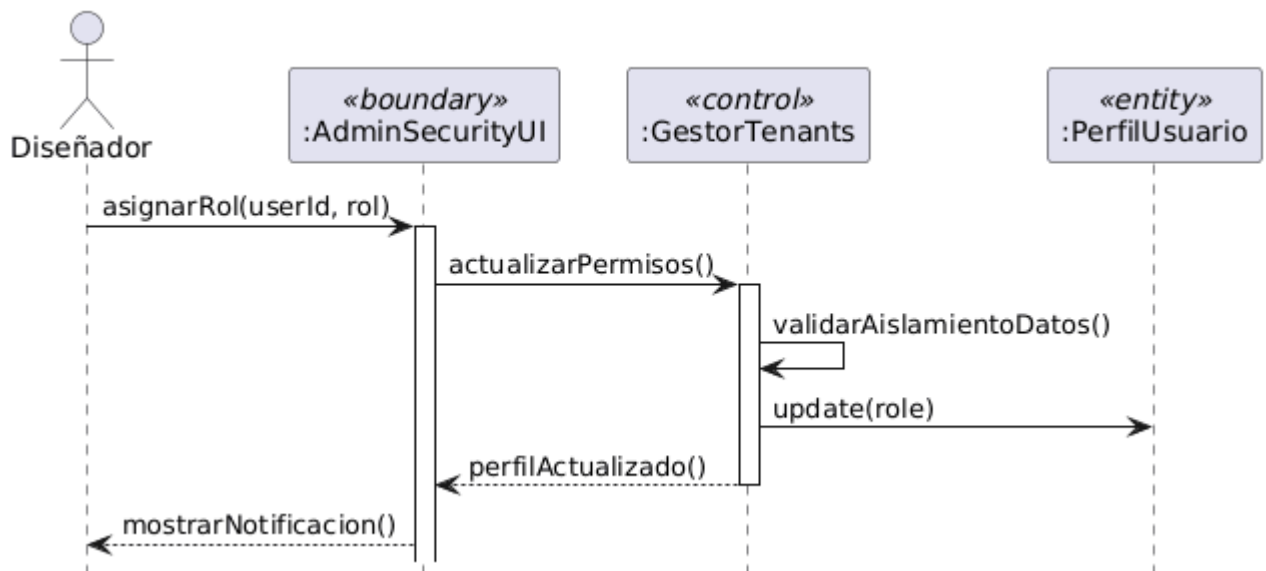
CU9 Consultar al asistente virtual proactivo (ElevenLabs)



CU10 Consultar trazabilidad de trámite y notificaciones

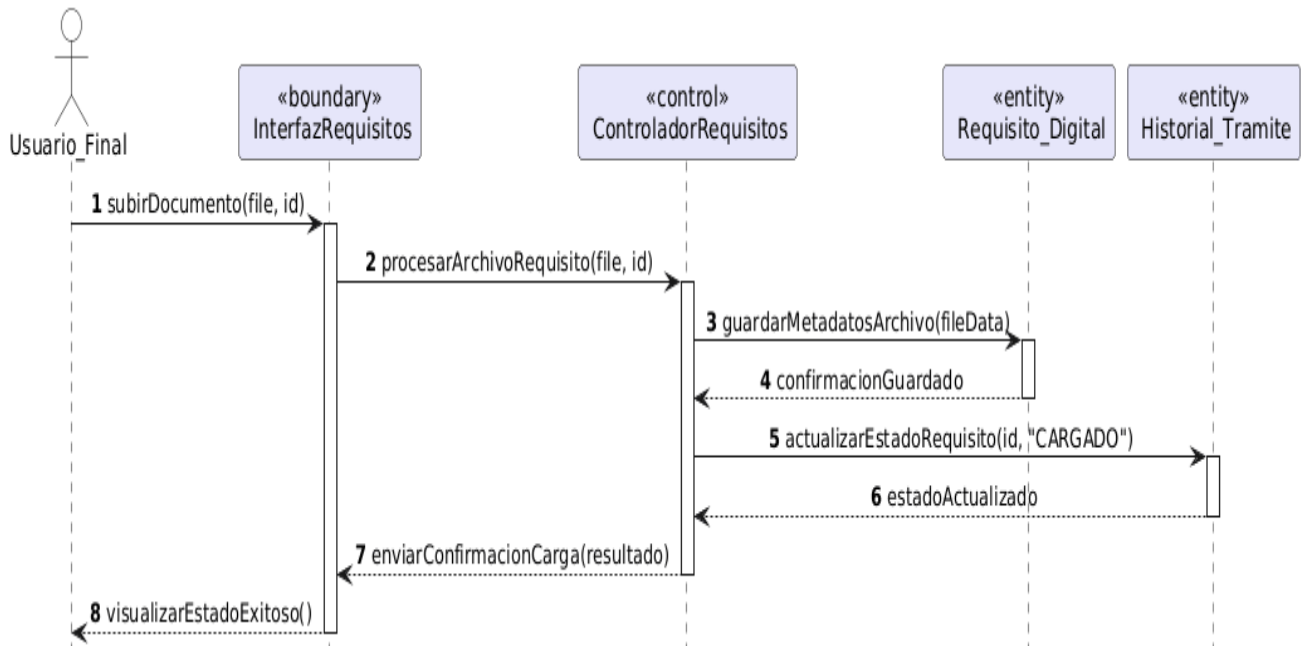


CU11: Gestionar seguridad y multitenant

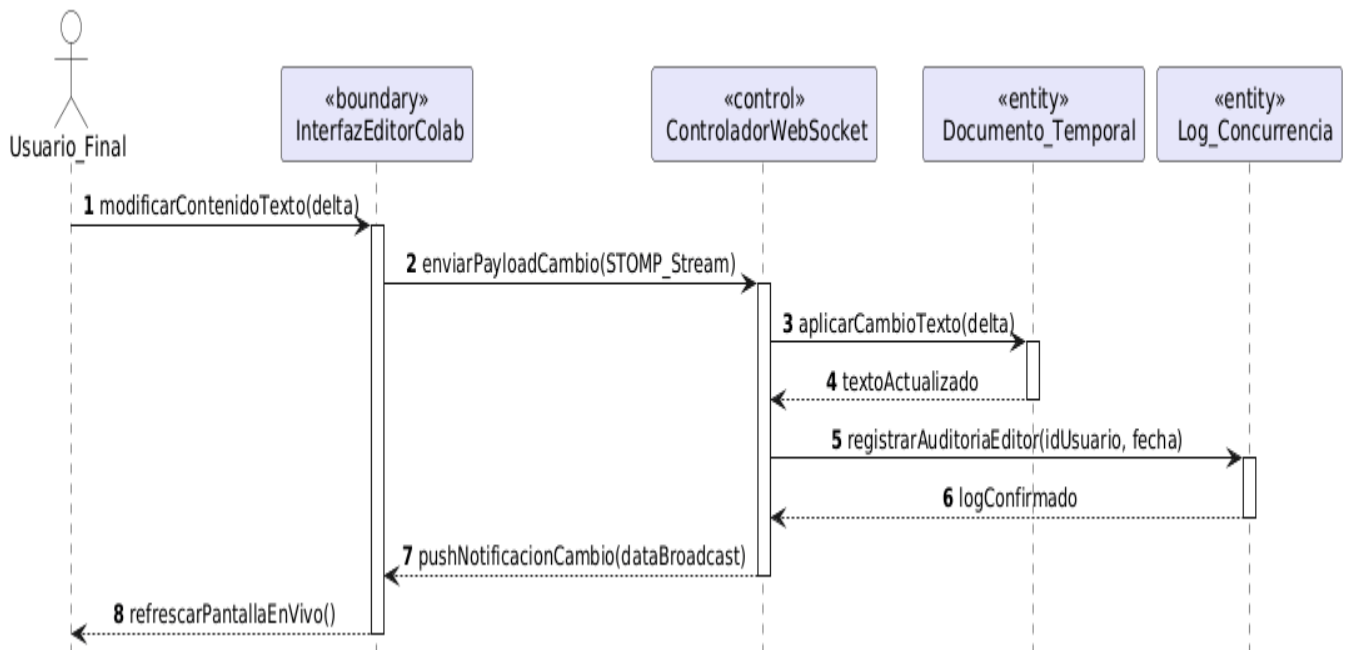


Ciclo # 2

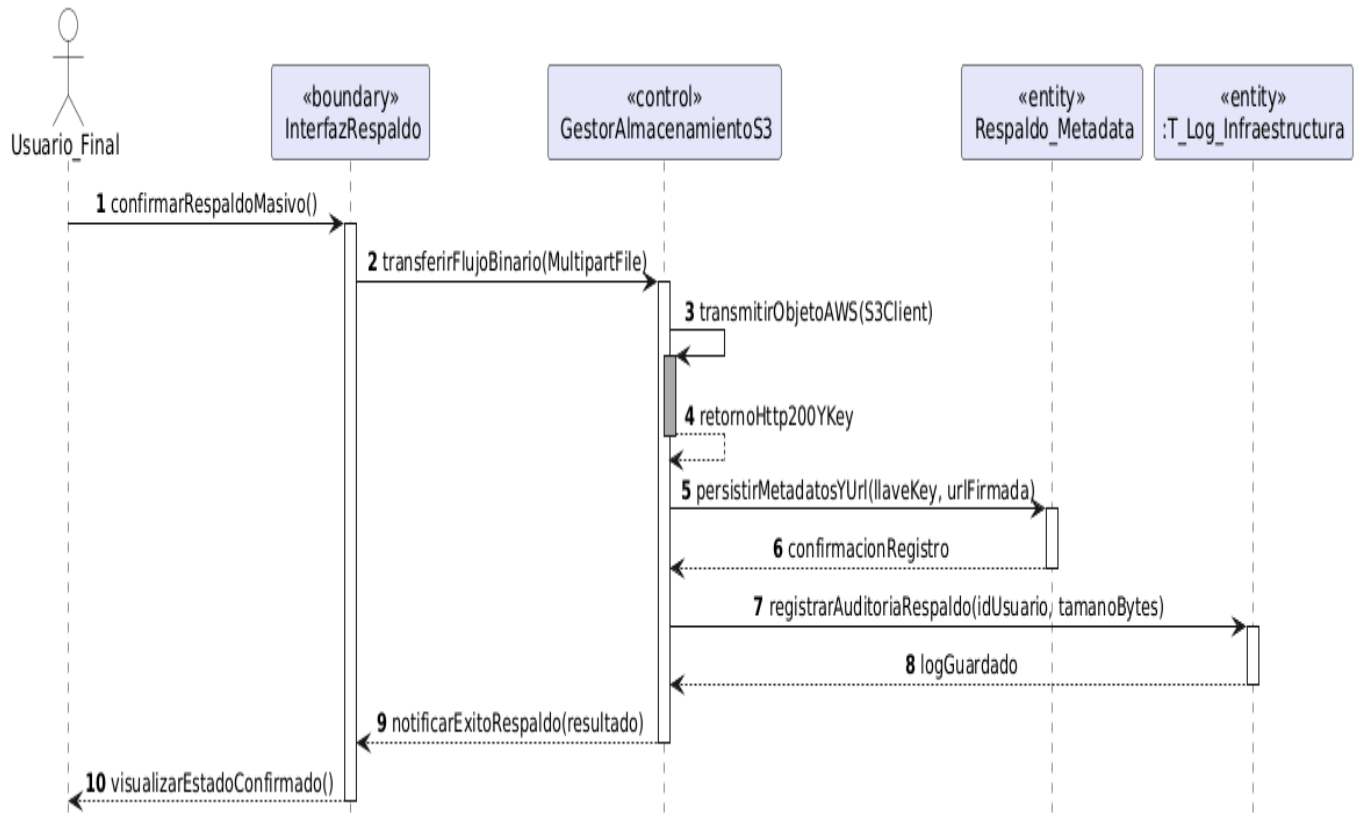
CU12 Subir requisitos digitales al trámite



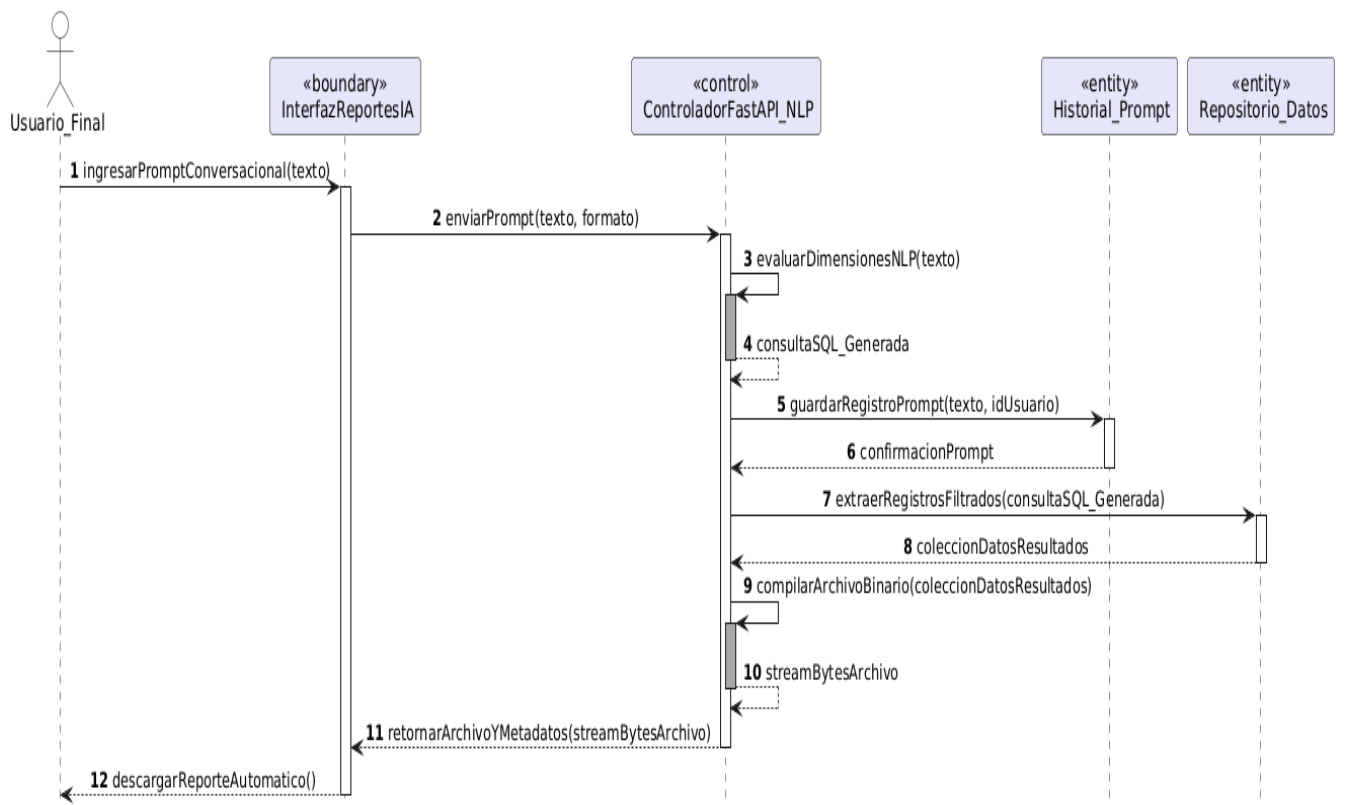
CU13 Editar documentos colaborativamente en tiempo real



CU14 Respalidar archivos masivos en infraestructura externa



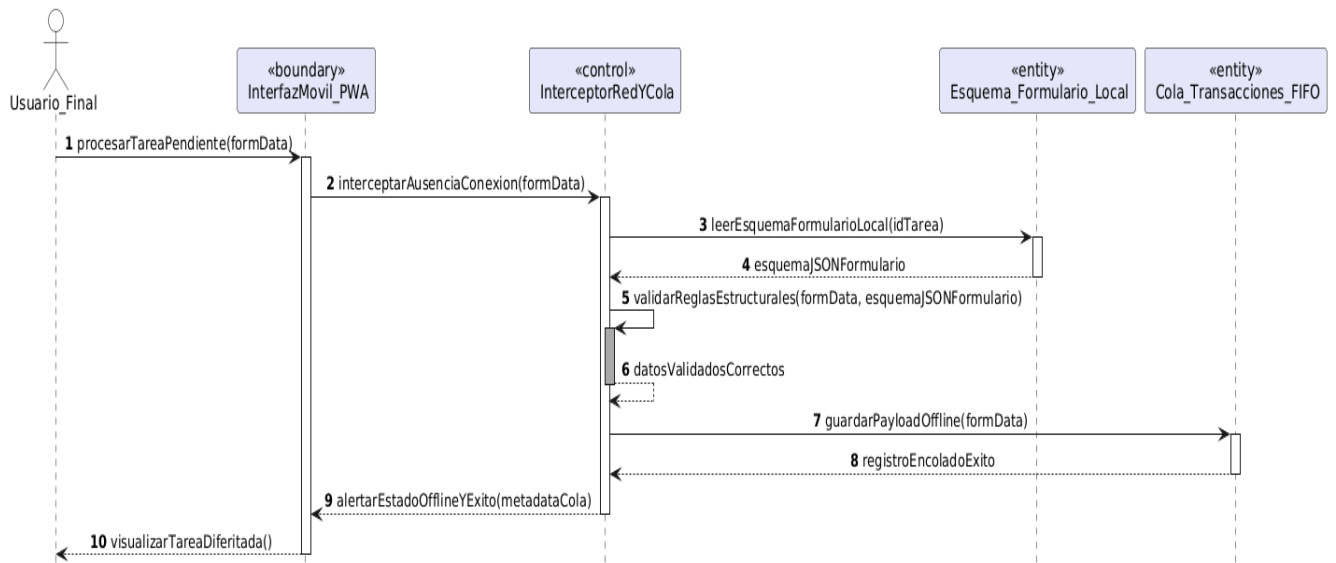
CU15 Generar reportes dinámicos mediante lenguaje natural



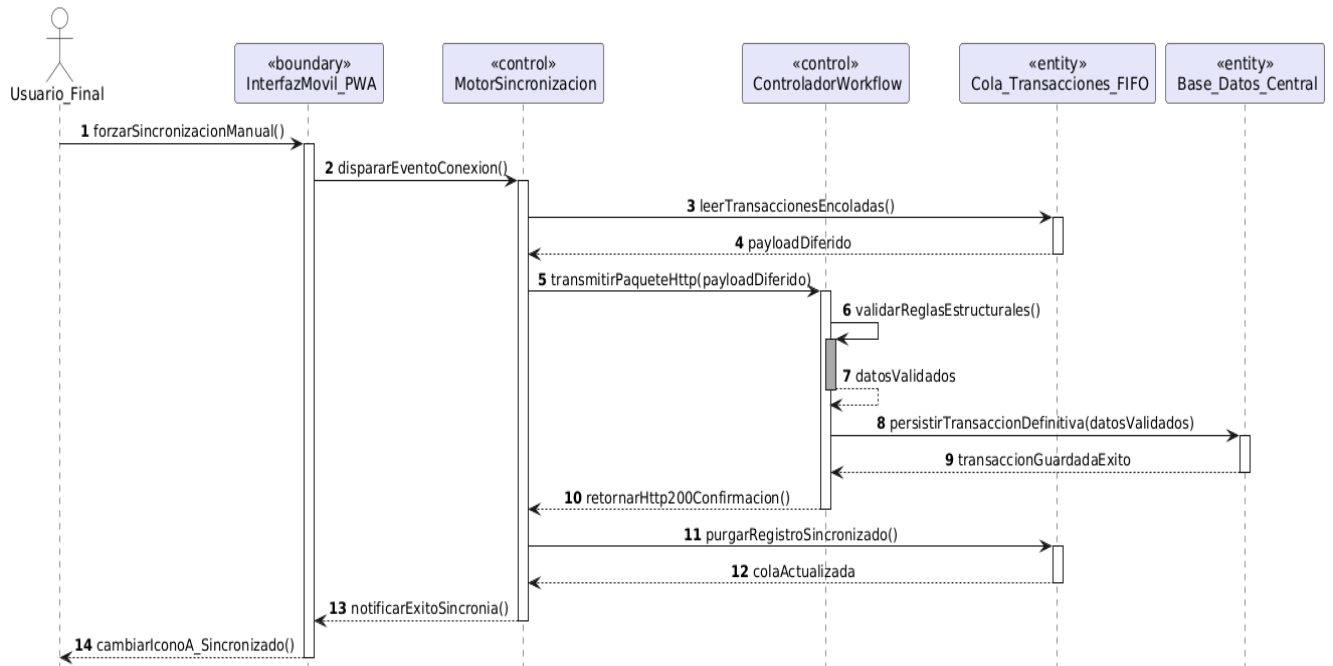
CU16 Evaluar enrutamiento predictivo y análisis de riesgos



CU17 Procesar tareas pendientes en modo fuera de línea



CU18 Sincronizar transacciones diferidas acumuladas en el dispositivo



6.FLUJO DE TRABAJO: IMPLEMENTACION

6.1. Elección de la plataforma de desarrollo de software

6.1.1. Lenguajes de programación

Java

Es el lenguaje de programación orientado a objetos utilizado para el desarrollo del núcleo robusto del backend mediante el framework Spring Boot. Su tipado fuerte y madurez en el ecosistema empresarial son fundamentales para gestionar la arquitectura multitenencia, la persistencia en MongoDB y la orquestación de los flujos de trabajo. Además, permite la implementación eficiente de la herramienta CASE para la generación automática de código, garantizando la escalabilidad y estabilidad del sistema.

Python

Python es un lenguaje de alto nivel reconocido por su versatilidad en el desarrollo backend y su dominio absoluto en el campo de la Inteligencia Artificial. En el sistema, Python actúa como el motor de inteligencia mediante FastAPI, permitiendo la implementación de modelos de procesamiento de lenguaje natural (NLP) para el diseño de procesos y la ejecución de algoritmos de analítica predictiva. Su capacidad para manejar datos multimodales de forma eficiente es clave para la interpretación de los requisitos del usuario.

TypeScript / JavaScript

Es el lenguaje principal utilizado para el desarrollo del frontend web. TypeScript, como superset de JavaScript, añade tipado estático, lo que permite detectar errores durante el desarrollo y mejorar la mantenibilidad del código a largo plazo. Es fundamental para gestionar los eventos de la interfaz en Angular, consumir los servicios de la API de forma segura y garantizar que el renderizado de los formularios dinámicos sea reactivo y preciso.

Dart

Es el lenguaje de programación optimizado para interfaces de usuario, utilizado para la creación de la aplicación móvil mediante el framework Flutter. Dart permite una compilación nativa que asegura un alto rendimiento y baja latencia en dispositivos móviles. Su arquitectura es ideal para gestionar el seguimiento de trámites en tiempo real, las notificaciones push y la integración de funciones proactivas, proporcionando una experiencia fluida tanto en Android como en iOS.

6.1.2. Base de datos

MongoDB

Es el sistema de gestión de base de datos NoSQL orientado a documentos seleccionado para la persistencia de toda la plataforma. A diferencia de las bases de datos relacionales, MongoDB utiliza un esquema flexible basado en documentos BSON (Binary JSON), lo que es fundamental para el almacenamiento de los formularios dinámicos y las definiciones de procesos generados por la Inteligencia Artificial. Su capacidad nativa para manejar estructuras de datos complejas y anidadas permite una consulta eficiente de la trazabilidad de los trámites y garantiza un aislamiento lógico robusto en la arquitectura multitenencia, permitiendo que el sistema escale horizontalmente de manera sencilla.

6.1.3. Sistemas operativos

Windows 11

Se utiliza como el sistema operativo principal para el entorno de desarrollo, diseño y pruebas del software. Ofrece una amplia compatibilidad con los entornos de desarrollo integrados (IDEs) como IntelliJ IDEA y Visual Studio Code, además de permitir la orquestación local de microservicios mediante Docker Desktop. Su ecosistema de herramientas de productividad facilita la gestión de repositorios, el diseño de la base de datos y la simulación de terminales móviles para las pruebas preliminares de la aplicación.

Android / iOS

Son los sistemas operativos móviles de destino donde se ejecuta la aplicación del usuario final diseñada en Flutter. Estos entornos permiten que la aplicación acceda de forma nativa a funciones críticas del dispositivo, como el sistema de notificaciones push, el almacenamiento local seguro y la conectividad a internet de alta velocidad. La capacidad de compilación nativa asegura que la interfaz sea reactiva y que el seguimiento de los trámites en tiempo real funcione con baja latencia, garantizando una experiencia de usuario fluida en la mayoría de los dispositivos del mercado.

6.1.4. Otros

FRAMEWORK Y LIBRERIAS

Spring Boot

Es el framework de código abierto basado en Java utilizado para simplificar el desarrollo de aplicaciones de grado empresarial y microservicios. En el proyecto, actúa como el orquestador principal del backend, facilitando la configuración automática, el manejo de la seguridad mediante Spring Security y la integración fluida con MongoDB a través de Spring Data. Su arquitectura permite gestionar de manera eficiente la lógica multitenencia y garantiza una base sólida para la escalabilidad del sistema.

FastAPI

Es un framework web moderno y de alto rendimiento para construir APIs con Python. Su diseño basado en operaciones asíncronas es fundamental para la integración de los modelos de Inteligencia Artificial (NLP) y servicios de terceros. En el ecosistema del proyecto, FastAPI se encarga de procesar las solicitudes de diseño de procesos (CU-01) y la analítica predictiva, ofreciendo una latencia mínima y una validación de datos automática que acelera el ciclo de respuesta del servidor de IA.

Angular

Es la plataforma y framework de desarrollo basado en TypeScript utilizado para construir la interfaz web del sistema. Su estructura modular y el uso de

componentes permiten crear un panel administrativo y una bandeja de trabajo para el funcionario altamente eficiente. Angular facilita la creación de interfaces reactivas para el diseño de formularios dinámicos y garantiza que la comunicación con las APIs del backend sea segura y tipada, mejorando la robustez de la experiencia de usuario en el navegador.

Flutter

Es el SDK de código abierto desarrollado por Google para la creación de aplicaciones compiladas nativamente para dispositivos móviles desde una única base de código. Gracias a su motor de renderizado de alto rendimiento, permite que la aplicación del usuario final (A3) sea fluida y visualmente atractiva. Es clave para la implementación de interfaces reactivas que muestran la trazabilidad de los trámites en tiempo real y gestionan las interacciones proactivas del sistema.

6.1.5. Entornos de desarrollo (IDEs)

IntelliJ IDEA

Es el entorno de desarrollo integrado de JetBrains seleccionado para la construcción del núcleo del sistema en Java. Su capacidad avanzada para la refactorización de código, la integración nativa con el framework Spring Boot y el soporte especializado para la gestión de dependencias con Maven lo convierten en la herramienta ideal para asegurar la calidad y robustez del backend. Además, facilita la depuración de servicios y la inspección de datos en MongoDB a través de su suite de herramientas de base de datos integrada.

Visual Studio Code

Es un editor de código fuente ligero, extensible y de alto rendimiento utilizado para el desarrollo del frontend web (Angular), los servicios de inteligencia artificial en Python y la lógica móvil en Flutter. Gracias a su ecosistema de extensiones, permite una integración fluida con herramientas de control de versiones como Git, linters para asegurar la limpieza del código y terminales integradas para la gestión de contenedores Docker. Su versatilidad lo posiciona como el eje central para la programación políglota del proyecto.

MongoDB Compass

Es la interfaz gráfica de usuario oficial para la administración, exploración y visualización de la base de datos MongoDB. Se emplea para validar de forma visual la estructura de los documentos BSON, probar consultas complejas y

verificar que las reglas de validación de esquemas (JSON Schema) se apliquen correctamente sobre las colecciones. Su uso es clave para monitorear el rendimiento de la base de datos y asegurar la integridad de la información en la arquitectura multitenencia durante la fase de desarrollo y pruebas.

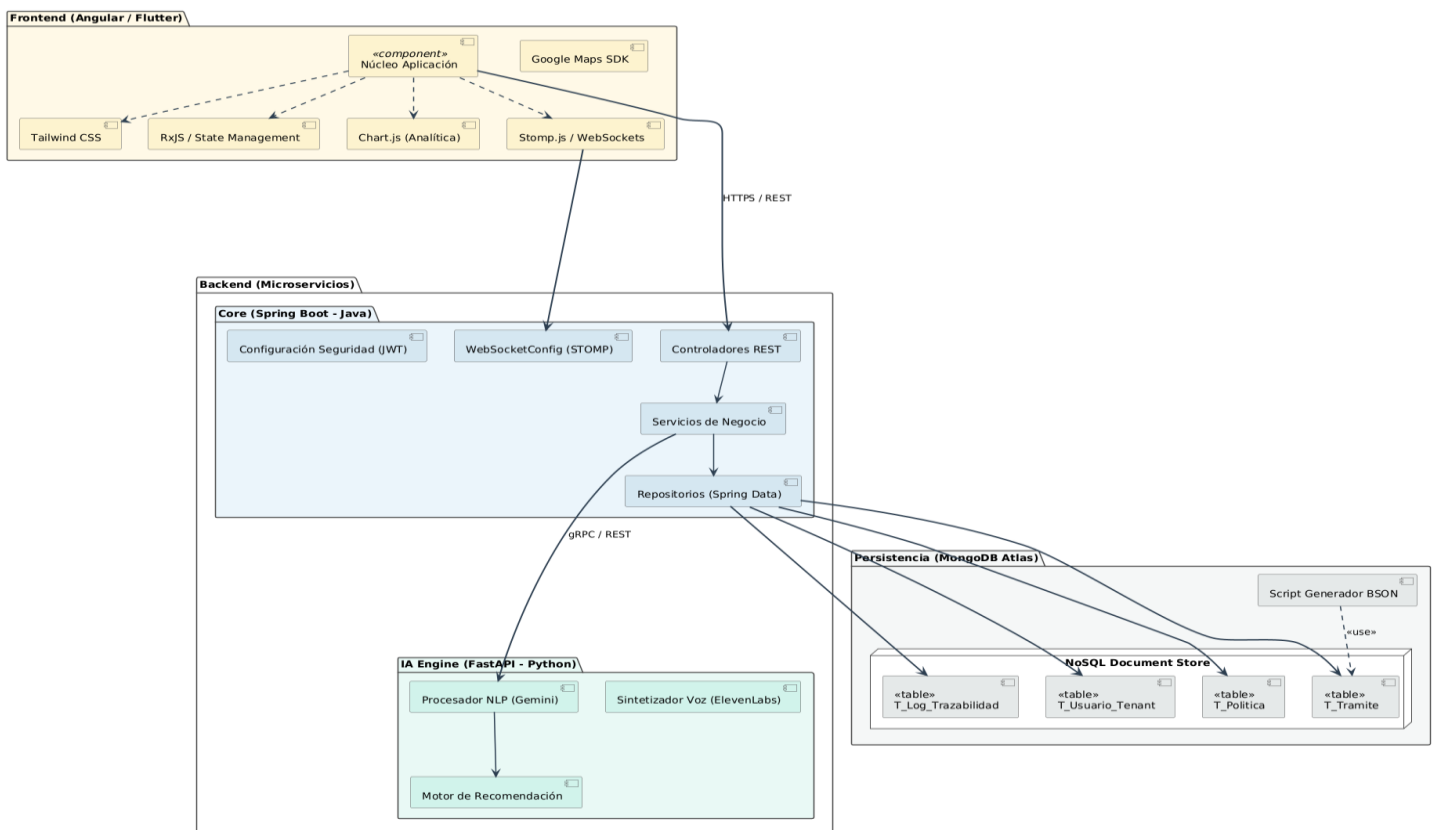
6.1.6. Servidores y alojamiento (AWS)

Amazon Web Services (AWS)

Es la plataforma de servicios de computación en la nube seleccionada para el despliegue y escalado de la infraestructura del sistema. Se utiliza específicamente el servicio Amazon EC2 (Elastic Compute Cloud) para el alojamiento de los microservicios en contenedores y Amazon S3 (Simple Storage Service) para el almacenamiento persistente de documentos y archivos estáticos. La arquitectura de AWS proporciona una infraestructura global con alta tolerancia a fallos, permitiendo que la plataforma gestione de forma segura las peticiones de los usuarios finales y los procesos intensivos de inteligencia artificial con una latencia mínima.

6.2. Implementación de la arquitectura de software

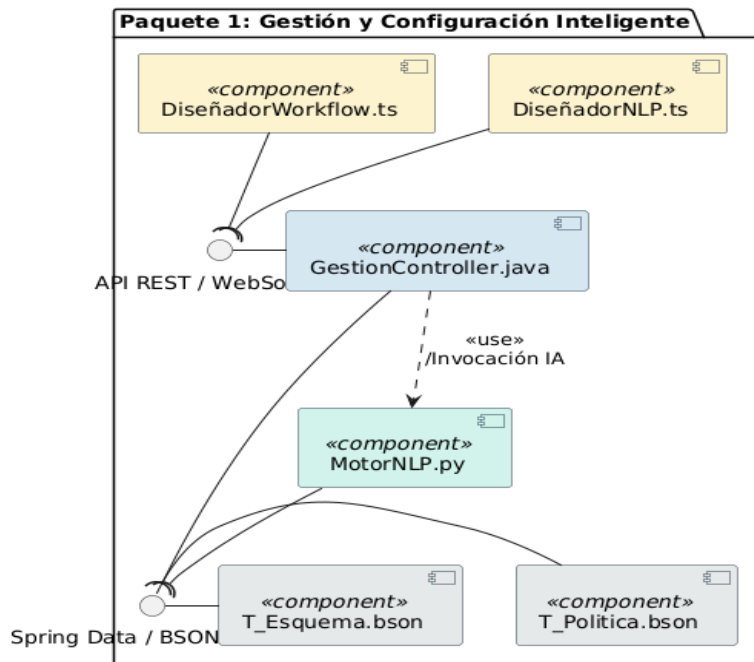
Diagrama de componentes Principal del sistema



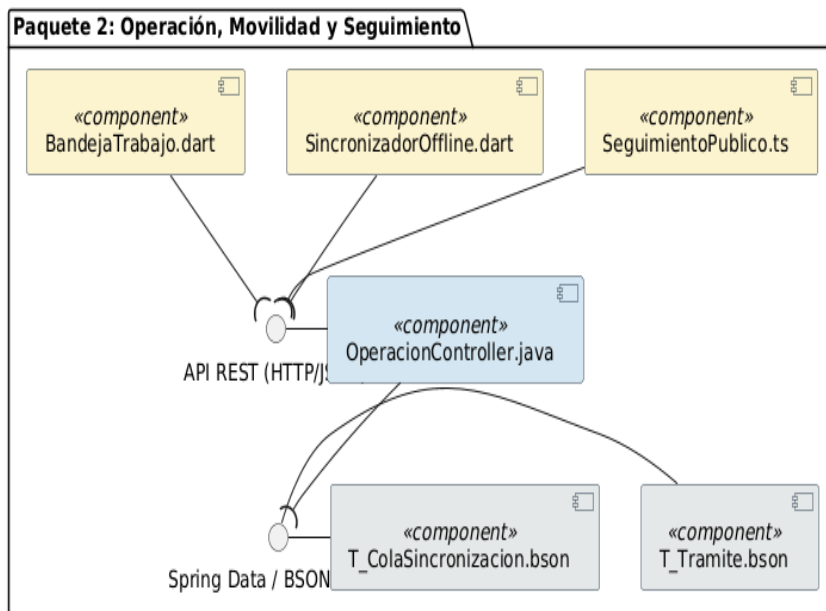
6.3. Implantación e la arquitectura de subsistema

Diagrama de componentes de cada paquete

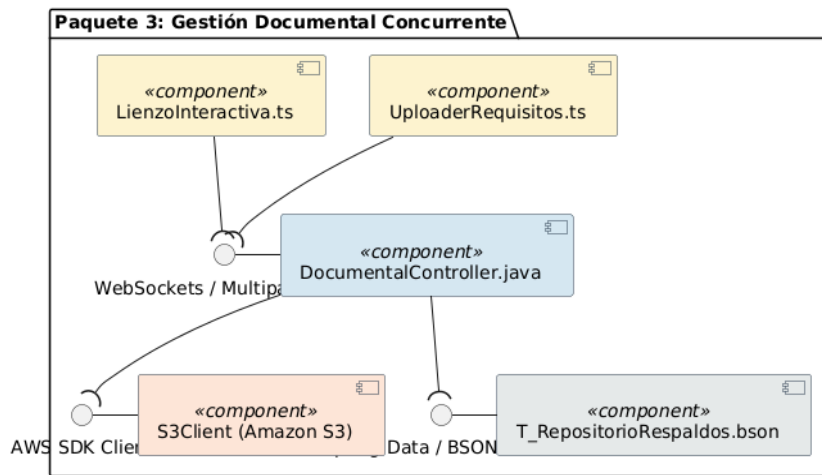
Subsistema: Paquete 1



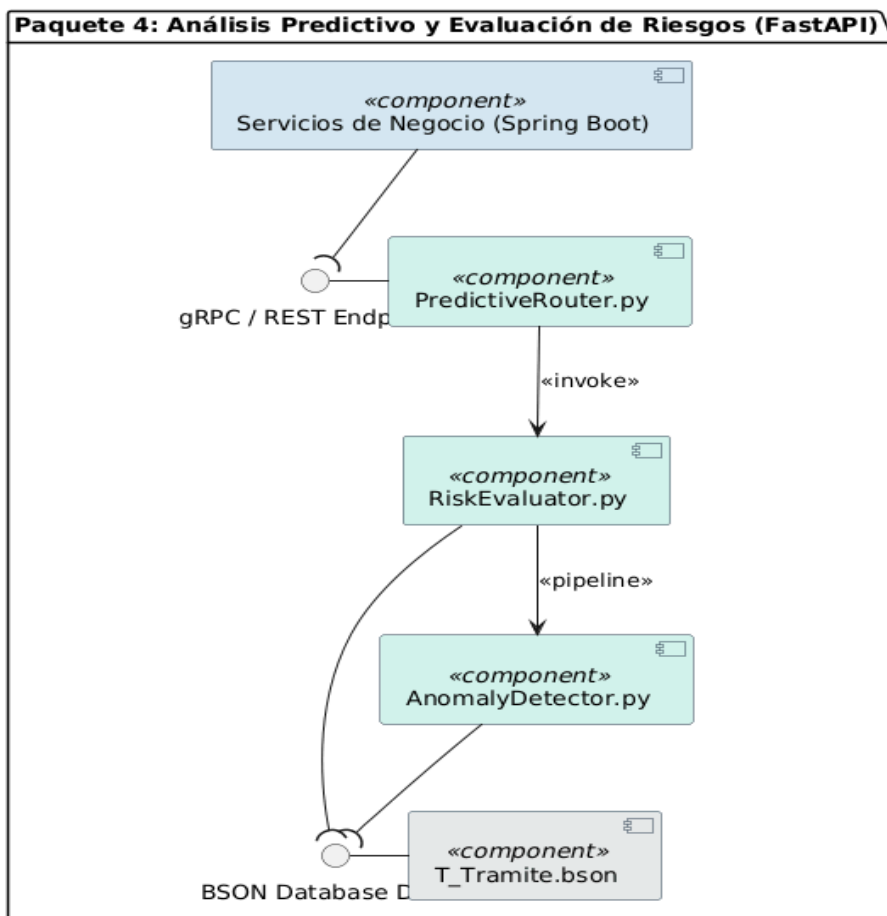
Subsistema: Paquete 2



Subsistema: Paquete 3



Subsistema: Paquete 4



7. CONCLUSION

La implementación de la Plataforma de Gestión Inteligente de Procesos bajo una arquitectura de microservicios políglota (Spring Boot y FastAPI) ha demostrado ser una solución robusta, elástica y de alta disponibilidad para los desafíos actuales de automatización empresarial. A partir del desarrollo, despliegue y pruebas del sistema, se concluye que:

- **Optimización del Diseño Funcional mediante Interfaces Cognitivas:** La integración de Procesamiento de Lenguaje Natural (NLP) a través de los modelos fundacionales de Gemini en el módulo de configuración (P1) reduce drásticamente la fricción técnica en el modelado de flujos. Esto permite que usuarios administrativos automaticen y parametricen grafos de procesos y formularios dinámicos mediante instrucciones conversacionales libres, sin requerir conocimientos en notaciones BPMN estructuradas.
- **Alta Cohesión, Desacoplamiento y Escalabilidad Políglota:** El diseño arquitectónico basado en el Modelo C4 permitió aislar de forma eficiente las responsabilidades del sistema. El núcleo transaccional en Spring Boot garantiza la consistencia del workflow, el control de concurrencia en la edición documental (P3) y la persistencia estructurada. Por otro lado, el microservicio analítico en FastAPI (Python) optimiza el rendimiento del sistema al encapsular de manera independiente las cargas pesadas de inferencia estadística para el enrutamiento predictivo y el análisis de riesgos en tiempo real (P4).
- **Garantía de Aislamiento y Persistencia Concurrente:** La implementación del patrón *Multitenant* a nivel lógico asegura un aislamiento estricto y la confidencialidad de la información entre distintas organizaciones inquilinas. Asimismo, la estrategia de persistencia híbrida —combinando metadatos livianos e indexación elástica en MongoDB Atlas con un esquema de streaming binario diferido hacia el almacenamiento masivo en Amazon S3— optimiza los costos operativos de infraestructura y previene la degradación del rendimiento de la base de datos central.
- **Resiliencia Operativa en Escenarios de Desconexión:** Los mecanismos de tolerancia a fallos desarrollados para la capa de movilidad (P2) demuestran que el uso de cachés criptográficas locales y colas de transacciones con enfoque FIFO (First-In, First-Out) garantizan la continuidad de la operación en campo bajo condiciones de red nula o intermitente, restableciendo la consistencia global del sistema de forma

automática y asíncrona una vez detectado un canal de comunicación estable.

- **Evolución hacia Sistemas Colaborativos Proactivos:** La incorporación de flujos bidireccionales concurrentes mediante WebSockets (STOMP) combinada con asistentes de voz (ElevenLabs) y motores predictivos de Deep Learning, transforma la plataforma de un software pasivo de registro en un ecosistema proactivo de asistencia. El sistema no solo previene cuellos de botella al calcular dinámicamente los scores de riesgo frente al vencimiento de acuerdos de nivel de servicio (SLA), sino que optimiza la toma de decisiones críticas y la asignación prioritaria de recursos en la organización.

8. RECOMENDACIÓN

Para garantizar la evolución, el mantenimiento preventivo y la sostenibilidad de la Plataforma de Gestión Inteligente de Procesos en el entorno productivo de la FICCT-UAGRM, se recomienda:

- **Optimización y Eficiencia de Costos en la Infraestructura Cloud (AWS):** Implementar políticas de auto-escalado (Auto Scaling Groups) configuradas con métricas agresivas basadas en el uso de CPU y memoria para los contenedores transaccionales. Asimismo, se sugiere la adopción de instancias programables de bajo costo (AWS Spot Instances) para el clúster analítico de FastAPI encargado de la inferencia pesada de los modelos de Deep Learning, disminuyendo significativamente los costos operativos mensuales en la nube sin comprometer la alta disponibilidad ni degradar la experiencia de usuario.
- **Gobernanza de Datos y Maduración Continua de Modelos Cognitivos (MLOps):** Establecer un pipeline continuo de captura y curación de datos para alimentar los almacenes de entrenamiento del motor NLP (Gemini). Se debe priorizar la recopilación de variaciones lingüísticas, modismos locales y terminología técnica/administrativa propia de los flujos burocráticos y académicos de Bolivia, optimizando los hiperparámetros de los modelos para mitigar sesgos semánticos y elevar la precisión en la traducción automática de los diagramas y reportes conversacionales.

- **Robustecimiento del Esquema de Seguridad y Gobierno de Identidades:** Fortalecer el módulo de gestión de accesos (P1) mediante la integración obligatoria de mecanismos de autenticación multifactor (MFA), basados en contraseñas de un solo uso por tiempo (TOTP) o llaves criptográficas de hardware. Esta capa adicional resguarda de manera estricta los privilegios de los perfiles con rol de Diseñador o Administrador de cada organización (Tenant), blindando el sistema contra vectores de ataque por ingeniería social o compromiso de credenciales.
- **Monitoreo de Infraestructura y Observabilidad en Tiempo Real:** Configurar herramientas centralizadas de recolección de métricas y logs distribuidos (tales como AWS CloudWatch, Prometheus o Grafana) con enfoque en el monitoreo del ciclo de vida de los trámites. La implementación de alertas tempranas sobre el comportamiento de la base de datos documental y el estado de la cola transaccional offline facilitará al equipo de soporte técnico la detección proactiva de anomalías antes de que afecten los acuerdos de nivel de servicio (SLA) de la facultad.
- **Estrategias de Desacoplamiento de Base de Datos (Data Sharding):** Considerar a mediano plazo la segmentación física de los datos (Database Sharding) en MongoDB Atlas utilizando el identificador único de la organización (tenant_id) como llave de partición primaria. A medida que el número de instituciones e inquilinos concurrentes escale en el ecosistema, esta práctica técnica garantizará que las operaciones I/O permanezcan distribuidas de manera uniforme, evitando cuellos de botella en la persistencia centralizada y aislando por completo la carga operativa de cada nodo organizativo.

9. BIBLIOGRAFIA

Libros y Literatura Científica

- Brown, S. (2018). *Software Architecture for Developers: Visualise, document and explore your software architecture*. Leanpub.
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.
- Jacobson, I., Booch, G., y Rumbaugh, J. (2000). *El Proceso Unificado de Desarrollo de Software*. Addison-Wesley.
- Jacobson, I., Booch, G., y Rumbaugh, J. (2006). *El Lenguaje Unificado de Modelado* (2a. ed.). Addison-Wesley.

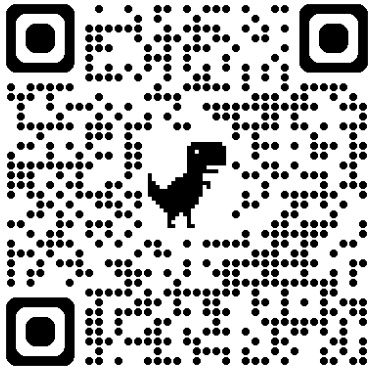
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Documentación Técnica y Recursos Digitales

- Amazon Web Services. (s.f.). ¿Qué es Java?. <https://aws.amazon.com/es/what-is/java/>
- Amazon Web Services. (s.f.). ¿Qué es el almacenamiento elástico en la nube con Amazon S3?. <https://aws.amazon.com/es/s3/>
- FastAPI. (s.f.). *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. <https://fastapi.tiangolo.com/>
- Flutter Dev. (s.f.). *Build apps for any screen: Flutter documentation*. <https://docs.flutter.dev/>
- Google Cloud. (s.f.). *Gemini Large Language Models API reference documentation*. <https://cloud.google.com/vertex-ai/docs/generative-ai/model-reference/gemini>
- JetBrains. (s.f.). *IntelliJ IDEA: El IDE de Java capaz y ergonómico*. <https://www.jetbrains.com/es-es/idea/features/>
- MongoDB. (s.f.). *MongoDB Atlas: Cloud Document Database Service*. <https://www.mongodb.com/cloud/atlas>
- Postman. (s.f.). *The Collaboration Platform for API Development*. <https://www.postman.com/>
- Spring Framework. (s.f.). *Why Spring: Modern, Productive, Agile, Lightweight, and Secure Java*. <https://spring.io/why-spring>
- Spring Initializr. (s.f.). *Bootstrap your Spring Boot application metadata*. <https://start.spring.io/>
- Tailwind Labs. (s.f.). *Tailwind CSS: A utility-first CSS framework for rapid UI development*. <https://tailwindcss.com/>
- United States National Institute of Standards and Technology (NIST). (s.f.). *Role-Based Access Control (RBAC) Standards*. <https://csrc.nist.gov/projects/role-based-access-control>

10. CODIGO

10.1. repositorio de github



<https://github.com/beimarM1/segundo-parcial-sw1.git>

10.2. proyecto desplegado



<https://enterprise-diagrammer.netlify.app/login>

10.3. apk

<https://drive.google.com/file/d/12R36L1XxYWCeNaOX-xOx4eecpH7FWK8Y/view?usp=sharing>

